

INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

OmniAI Gateway

Relatório Final de Projeto

Guilherme Coutinho

Nº 50467

A50467@alunos.isel.pt

André Nunes

Nº 51766

A51766@alunos.isel.pt

Orientador: Pedro Félix

pedro.felix@isel.pt

Engenharia Informática e de Computadores

7 de julho de 2026

Resumo

O desenvolvimento de aplicações que utilizam Inteligência Artificial Generativa (Gen AI) tem crescido de forma exponencial. No entanto, a ausência de um protocolo comum na comunicação com Modelos de Linguagem de Grande Escala (LLMs) obriga estas aplicações a integrarem diferentes interfaces de acesso e implementações particulares para a invocação de ferramentas externas. Atualmente cada fornecedor (como **OpenAI** [1], **Anthropic** [2] e **Gemini** [3]) impõe as suas próprias estruturas de dados, ciclos de vida de *streaming* e padrões para a invocação de ferramentas (*tool calling*) [4] [5]. Para resolver esta fragmentação tecnológica, este projeto propõe o desenvolvimento do **OmniAI SDK**, um *Software Development Kit* construído em **Kotlin Multiplatform (KMP)** [6]. O objetivo principal desta biblioteca é fornecer a infraestrutura base necessária para instanciar e configurar de forma rápida uma **OmniAI Gateway** (um ponto de entrada centralizado dedicado ao acesso de inferência, segurança, observabilidade e outros). O **OmniAI SDK** é baseado nos princípios da Arquitetura Hexagonal (*Ports and Adapters*) [7], este isola a lógica de domínio e suporta a comunicação bidirecional compatível com os padrões das APIs de referência, nomeadamente **OpenAI** [1], **Anthropic** [2] e **Gemini** [3]. O sistema implementa uma *pipeline* extensível e configurável, inspirada no modelo de filtros e cadeia de filtros dos **Servlets** [8] [9], que executa *interceptors* pré-construídos para autorização, autenticação, métricas, *rate limiting*, *circuit breaker* e *fallback*, podendo ser adicionados mais *interceptors* customizados pelo utilizador do **OmniAI SDK**. Adicionalmente, o *Software Development Kit* prepara um módulo **MCP Broker** para evoluir a **OmniAI Gateway** de uma *gateway* de inferência e torná-la numa *AI Gateway* completa, adicionando a gestão de ferramentas externas à **OmniAI Gateway** utilizando o **Model Context Protocol (MCP)** [10]. Distribuído de forma multiplataforma para ambientes **JVM**, via **Maven**, e **Node.js/TypeScript**, via **NPM**, o **OmniAI SDK** foi validado através da construção de uma **OmniAI Gateway**, um *Proof of Concept* que utiliza a biblioteca para demonstrar a criação de uma *gateway* real. A avaliação demonstra que a solução minimiza o acoplamento tecnológico e o esforço de integração, garantindo resiliência, escalabilidade e centralização na utilização de clientes de IA, como o **Claude Code** [11], entre outros.

Abstract

The development of applications leveraging Generative Artificial Intelligence (Gen AI) has grown exponentially. However, the lack of a common protocol for communicating with Large Language Models (LLMs) forces these applications to integrate different access interfaces and custom implementations for invoking external tools. Currently, each provider (such as **OpenAI** [1], **Anthropic** [2], and **Gemini** [3]) imposes its own data structures, *streaming* lifecycles, and patterns for *tool calling* [4] [5]. To address this technological fragmentation, this project proposes the development of the **OmniAI SDK**, a *Software Development Kit* built in **Kotlin Multiplatform** (KMP) [6]. The main objective of this library is to provide the necessary foundational infrastructure to quickly instantiate and configure an **OmniAI Gateway** (a centralized entry point dedicated to inference access, security, observability, and others).

Based on the principles of Hexagonal Architecture (*Ports and Adapters*) [7], the **OmniAI SDK** isolates the domain logic and supports bidirectional communication compatible with the standards of reference APIs, namely **OpenAI** [1], **Anthropic** [2], and **Gemini** [3]. The system implements an extensible and configurable *pipeline*, inspired by the **Servlets** filter and filter chain model [8] [9], which executes pre-built *interceptors* for authorization, authentication, metrics, *rate limiting*, *circuit breaking*, and *fallback*. Additionally, custom *interceptors* can be added by the **OmniAI SDK** user.

Furthermore, the *Software Development Kit* includes an **MCP Broker** module to evolve the **OmniAI Gateway** from an inference *gateway* into a full-fledged *AI Gateway*, by adding external tool management capabilities using the **Model Context Protocol** (MCP) [10]. Distributed as a multiplatform solution for **JVM** environments via **Maven**, and for **Node.js/TypeScript** via **NPM**, the **OmniAI SDK** was validated through the construction of an **OmniAI Gateway**, a *Proof of Concept* that utilizes the library to demonstrate the creation of a real-world *gateway*. The evaluation demonstrates that the solution minimizes technological coupling and integration effort, ensuring resilience, scalability, and centralization when using AI clients, such as **Claude Code** [11], among others.

Índice

1	Introdução	5
1.1	Contexto	5
1.2	Motivação e Definição do Problema	6
1.3	Objetivos e Proposta de Solução	7
1.4	Estrutura do Documento	8
2	Estudo Prévio	9
2.1	Análise das APIs de Inferência	9
2.1.1	Construção do Pedido	10
2.1.2	Processamento da Resposta	12
2.1.3	Divergências na Padronização de Esquemas de Validação	13
2.1.4	Conclusão do Estudo das APIs de Inferência	14
2.2	O protocolo MCP	14
2.2.1	Arquitetura do protocolo	15
2.2.2	Formato da Comunicação	15
2.2.3	Descoberta de capacidades	16
2.2.4	Mecanismos de Transporte	17
2.2.5	Autenticação e Autorização	18
2.2.6	Ferramentas de Inspeção	19
2.2.7	Protótipo Desenvolvido	19
2.3	Estudo de Soluções Semelhantes	19
3	Arquitetura e Desenho do Sistema	21
3.1	Modelação do Domínio Comum	23
3.1.1	Visão Global do Domínio	23
3.1.2	Estrutura do Domínio Comum	24
3.1.3	Mapa Tipificado e Extensibilidade do Domínio	28
3.2	Arquitetura do Sistema	30
3.2.1	Core	30
3.2.2	Contratos	31
3.2.3	Adaptadores de Entrada	31
3.2.4	Adaptadores de Saída	31
3.2.5	Camada de Comunicação HTTP	32
3.2.6	Dispatcher	32
3.3	Organização do <i>SDK</i>	32

4	Implementação e Evolução Técnica	34
4.1	Tecnologias e Metodologias Utilizadas	34
4.2	A Primeira Iteração da <i>Gateway</i>	34
4.3	Decisões de Desenho e Alternativas Consideradas	35
4.4	Evolução Arquitetural: De <i>Inference Service</i> a <i>Dispatcher</i>	36
4.5	Implementação da Camada HTTP	36
4.6	Implementação do <code>HttpTransportClient</code> em Ktor	37
4.7	Implementações Inbound e Outbound	38
4.8	Tradutores de Entrada (Inbound)	38
4.8.1	OpenAI Inbound Translator	38
4.8.2	Anthropic Inbound Translator	40
4.8.3	Gemini Inbound Translator	41
4.9	Tradutores de Saída (Outbound)	43
4.9.1	OpenAI Outbound Translator	43
4.9.2	Anthropic Outbound Translator	45
4.9.3	Gemini Outbound Translator	46
4.10	Evolução para a Pipeline de Interceptors	48
4.11	Arquitetura Interna dos Interceptors	50
4.11.1	Interceptor de Telemetria e Observabilidade	50
4.11.2	Interceptor de Autenticação	52
4.11.3	Interceptor de Autorização	54
4.11.4	Interceptor de <i>Rate Limiting</i>	55
4.11.5	Interceptor de <i>Circuit Breaker</i>	56
4.11.6	Interceptor de <i>Fallback</i>	57
4.11.7	Interceptor de <i>Logging</i> e <i>Tracing</i>	58
4.12	Distribuição Multiplataforma	58
5	MCP Broker e Ferramentas Externas	60
5.1	Visão Geral do Sistema	60
5.2	Modelos de Domínio e DTOs	61
5.2.1	DTOs de Configuração	61
5.2.2	Modelos de Domínio	62
5.3	Configuração Declarativa em YAML	62
5.3.1	Descrição de Ferramentas REST	63
5.3.2	Contextos Estáticos como Recursos	63
5.3.3	Proxy de Servidores MCP Externos	64
5.4	Arranque e Inicialização do MCP Broker	64
5.5	Hot-Reload de Configuração	65
5.6	DSL de Configuração Programática	65
5.7	Fluxo de Execução	66
6	Provas de Conceito e Validação	67
6.1	Validação do <i>Proof of Concept</i> da OmniAI Gateway	67
6.2	Validação com REST Client	68
6.3	Validação de Segurança com Logto	68
6.4	Validação de Observabilidade	68
6.5	Validação com Aplicações Cliente	68

6.5.1	Execução do Claude Code com autenticação Logto	69
6.6	Limites da Validação	69
7	Conclusões e Trabalho Futuro	70
7.1	Conclusões	70
7.2	Melhorias Futuras e Escalabilidade	71
A	Exemplos JSON das APIs de Inferência	72
A.1	Pedidos (<i>Requests</i>)	72
A.1.1	OpenAI (formato compatível)	72
A.1.2	Anthropic	74
A.1.3	Google Gemini	75
A.2	Respostas (<i>Responses</i>)	77
A.2.1	OpenAI (formato compatível)	77
A.2.2	Anthropic	80
A.2.3	Google Gemini	82
B	Histórico de Planeamento e Riscos	85
B.1	Riscos	85
B.2	Planeamento	87

Lista de Figuras

1.1	Acoplamento direto entre cliente e um fornecedor de inferência	6
1.2	Multiplicação de conexões com chaves de API descentralizadas	6
1.3	Cliente decide trocar de fornecedor	6
1.4	Centralização com OmniAI Gateway	7
2.1	Estruturas JSON esquemáticas e simplificadas de pedidos para cada fornecedor.	11
2.2	Estruturas JSON esquemáticas e simplificadas de respostas para cada fornecedor.	13
2.3	Arquitetura simplificada do MCP	15
2.4	Fluxo de comunicação MCP - Adaptado da Documentação do MCP [12]	17
3.1	Arquitetura de alto nível: OmniAI SDK integrada numa Gateway, nomeadamente uma OmniAI Gateway	22
3.2	Visão da integração entre Ktor , <i>inbounds</i> , <i>pipeline</i> , <i>interceptors</i> , <i>outbounds</i> e HTTP client no OmniAI SDK	22
3.3	Fluxo de normalização entre contratos externos e domínio comum (OmniAi-SDK)	23
3.4	Estrutura de classes do CommonRequest	24
3.5	Estrutura de classes do CommonResponse	26
3.6	Utilização da TypedMap para transportar atributos extensíveis	30
4.1	Arquitetura de Tradução e Fluxo de Dados entre Contratos e Domínio Comum	38
4.2	Visão conceptual da <i>pipeline</i> de <i>interceptors</i>	49
4.3	Fluxo interno de processamento e lógica de observabilidade para pedidos unários e <i>streaming</i> .	51
4.4	Observabilidade e métricas operacionais da OmniAI Gateway	52
4.5	Sequência OIDC : descoberta, JWKS e cache de chaves	53
4.6	Fluxo interno do <i>interceptor</i> de autenticação	54
4.7	Arquitetura do <i>interceptor</i> de autorização: separação entre PEP e PDP	55
5.1	Visão geral do MCP Broker: integração planeada com o servidor de autorização, registo de ferramentas via YAML, execução de pedidos HTTP a APIs externas e proxy de servidores MCP externos.	61
5.2	Hierarquia dos DTOs de configuração.	62
5.3	Modelos de domínio e a substituição das estruturas YAML por equivalentes em JSON. . .	62
6.1	Cenários de validação do <i>Proof of Concept</i>	67

Capítulo 1

Introdução

1.1 Contexto

Os Modelos de Linguagem de Grande Escala (*Large Language Models – LLMs*) [13] tornaram-se componentes relevantes no desenvolvimento de aplicações baseadas em inteligência artificial. Um LLM é um modelo treinado com grandes volumes de texto, capaz de interpretar linguagem natural, gerar respostas e apoiar tarefas como sumarização, classificação, programação assistida, pesquisa semântica. Uma funcionalidade também oferecida por estes modelos é o *tool calling* [4] [5]. O *tool calling* permite que os modelos não se limitem ao seu conhecimento estático, permitindo-lhes descrever um pedido de interação com sistemas externos através de uma resposta estruturada. Este pedido obedece a um formato específico, definido previamente no momento em que as ferramentas são enviadas à API de inferência.

O ciclo de vida de um modelo divide-se em duas fases principais: treino e inferência. O treino corresponde ao processo de aprendizagem a partir de grandes conjuntos de dados, exigindo elevados recursos computacionais. A inferência corresponde à utilização do modelo já treinado para gerar respostas a novos pedidos. Atualmente, a maioria das aplicações integra LLMs através de APIs de inferência disponibilizadas por fornecedores externos. A comunicação com estas APIs é normalmente realizada usando o protocolo **HTTP** com representação em JSON, tanto nos pedidos como nas respostas. Antes de serem processados pelos modelos, os textos são convertidos em *tokens*, pequenas unidades de informação que permitem ao modelo representar e manipular sequências de entrada e saída [14]. Como a resposta é gerada de forma incremental, muitas APIs suportam *streaming*, permitindo enviar fragmentos da resposta ao cliente à medida que são produzidos.

Existem vários fornecedores relevantes, incluindo **OpenAI** [1], **Anthropic** [2], **Google Gemini** [3] e **Groq** [15]. Embora todos disponibilizem APIs **HTTP** e estruturas JSON, cada fornecedor define interfaces próprias, campos específicos, regras de autenticação e modelos distintos para *tool calling*. A integração direta com vários fornecedores obriga, por isso, as aplicações a conhecerem detalhes internos de cada API.

Ao mesmo tempo, o uso de LLMs evoluiu de interações simples para sistemas baseados em agentes (*agentic workflows*) [16]. Nestes sistemas, o modelo não se limita a devolver texto, pode também devolver instruções para chamadas a ferramentas externas, interpretar resultados intermédios e produzir uma resposta final com base nesses dados. O **Model Context Protocol (MCP)** surgiu como proposta de normalização para a comunicação entre modelos, aplicações e ferramentas externas [10].

1.2 Motivação e Definição do Problema

As Figuras 1.1, 1.2 e 1.3 ilustram o acoplamento direto entre clientes e fornecedores de inferência, bem como a dificuldade que os clientes tem em trocar de fornecedor devido a todo acoplamento que existe.

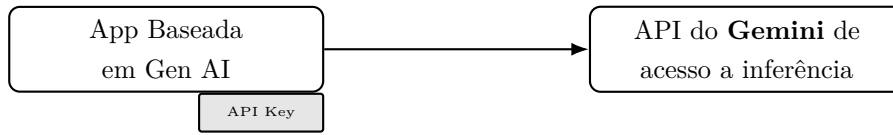


Figura 1.1: Acoplamento direto entre cliente e um fornecedor de inferência

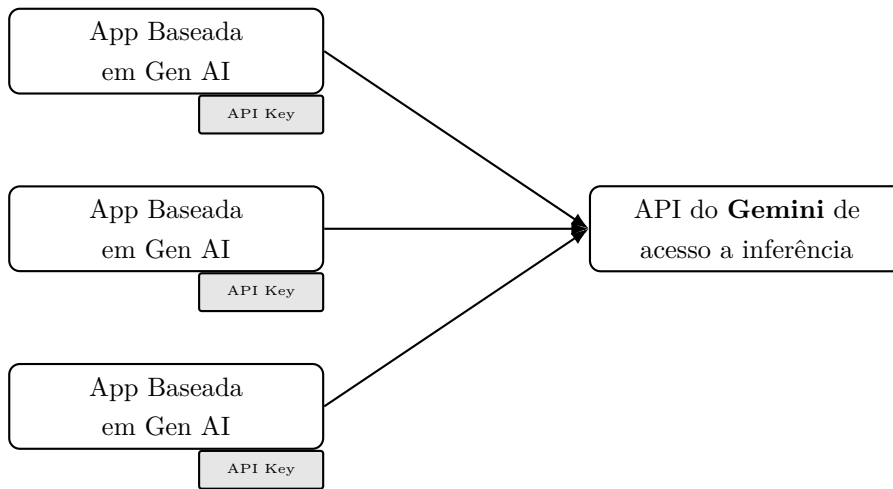


Figura 1.2: Multiplicação de conexões com chaves de API descentralizadas

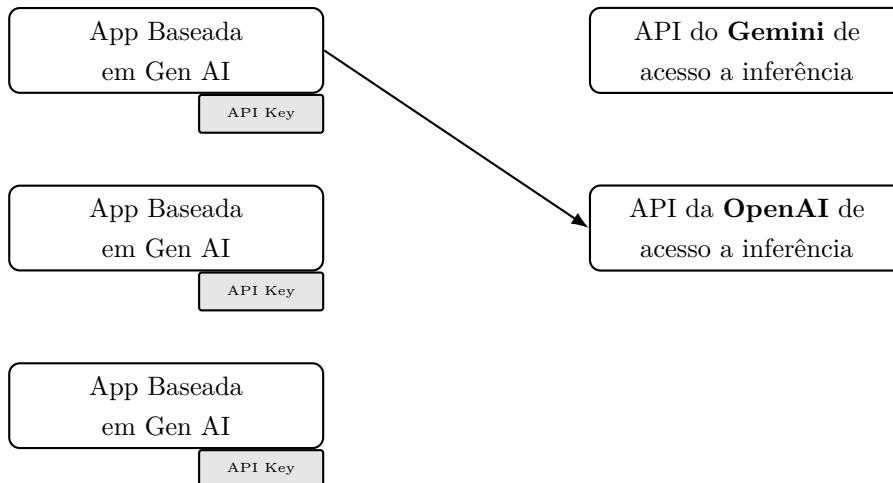


Figura 1.3: Cliente decide trocar de fornecedor

A existência de múltiplos fornecedores de LLMs e a ausência de um protocolo comum implicam diferentes interfaces de acesso aos modelos, estruturas específicas de pedidos e respostas, modelos distintos de contabilização de *tokens*, custos variáveis e implementações particulares para invocação de funções externas (ver Figuras 1.1–1.3). Por exemplo, a API da **OpenAI** [1] organiza a interação em torno de *chat completions* e *tool calls*; a API da **Anthropic** [2] utiliza uma arquitetura baseada em mensagens

e blocos de conteúdo; e a API do **Gemini** [3] estrutura a geração através de contents, parts, function declarations e configurações de geração próprias.

Estas diferenças criam acoplamento entre as aplicações cliente e os fornecedores, denominado de *vendor lock-in*. Sempre que uma aplicação pretende trocar de modelo, suportar mais do que um fornecedor, adicionar um mecanismo de *fallback* ou integrar ferramentas externas, torna-se necessário duplicar ou reimplementar lógica de tradução, autenticação, tratamento de erros e interpretação de respostas. Esta duplicação ou reimplementação aumenta a complexidade e dificulta a evolução do sistema, um problema recorrente em sistemas que combinam lógica de aplicação com integrações externas especializadas [17].

Para além da fragmentação dos contratos, existem preocupações operacionais e de segurança. Uma aplicação que comunica diretamente com vários fornecedores precisa de gerir chaves de API, quotas, limites de utilização, métricas, auditoria, políticas de acesso e proteção contra falhas de terceiros. Sem uma camada intermédia, a implementação destas lógicas fica fragmentada por diferentes componentes, tornando mais difícil aplicar regras consistentes de autenticação, autorização, observabilidade e resiliência.

O problema central que este projeto visa resolver é, assim, a ausência de uma camada de abstração capaz de separar as aplicações cliente das particularidades de cada fornecedor de LLMs, mantendo simultaneamente suporte para segurança, monitorização, *tool calling* e integração dinâmica com ferramentas externas.

1.3 Objetivos e Proposta de Solução

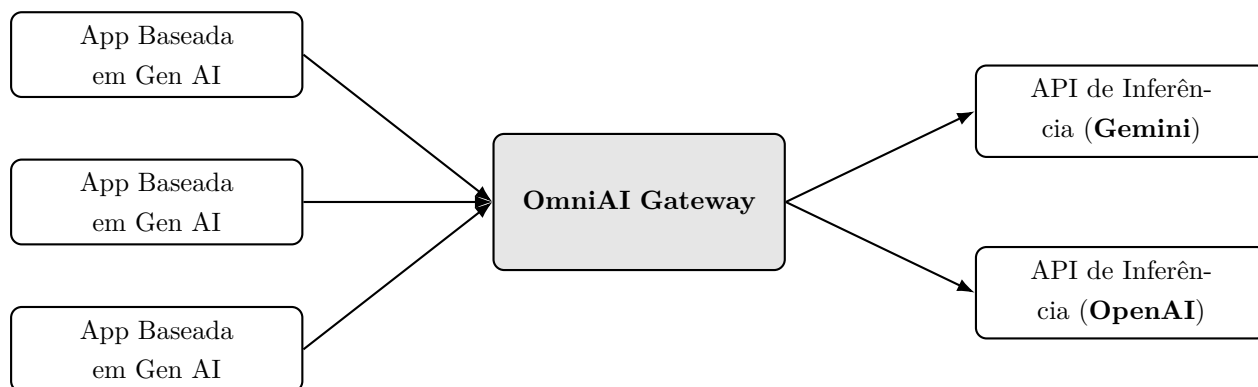


Figura 1.4: Centralização com **OmniAI Gateway**

A solução proposta consiste numa peça de infraestrutura intermediária que se posiciona entre as aplicações baseadas em Inteligência Artificial Generativa e as diversas APIs de inferência fornecidas pelo mercado. O objetivo é centralizar o acesso aos modelos através de uma **Gateway de IA**. Para permitir que a instanciação desta Gateway seja configurável e facilmente adaptável a diferentes cenários, a solução materializou-se na construção do **OmniAI SDK**, uma biblioteca reutilizável que abstrai a lógica de integração. A **OmniAI Gateway** instanciada a partir deste SDK atua assim como ponto de entrada para as aplicações cliente baseadas em Gen AI, centralizando o acesso a APIs de inferência, ferramentas externas e mecanismos globais de segurança e operação. Esta abordagem aproxima-se do padrão de API **Gateway** [18] e, no contexto específico de modelos de IA, do padrão de AI **Gateway** enquanto camada centralizada de acesso a modelos [19,20]. Embora existam soluções relacionadas, como LiteLLM e OpenRouter [21,22], o foco deste projeto está na construção de um **SDK** extensível, configurável e validado por uma **Gateway Proof of Concept** própria.

O **OmniAI SDK** foi desenvolvido em **Kotlin Multiplatform**, permitindo partilhar o domínio e vários componentes entre *targets* diferentes e preparar a distribuição para **JVM** e **JavaScript/Node.js** [23]. Esta decisão torna possível disponibilizar a biblioteca em ecossistemas distintos: via **Maven** para aplicações **JVM/Kotlin** e via **NPM** para ambientes **Node.js/TypeScript**.

A arquitetura do **OmniAI SDK** foi organizada segundo os princípios da Arquitetura Hexagonal, também conhecida como *Ports and Adapters* [7]. O domínio comum fica isolado de contratos externos, enquanto os *inbound adapters* traduzem pedidos de clientes compatíveis com **OpenAI**, **Anthropic** ou **Gemini** para o domínio comum, e os *outbound adapters* traduzem o modelo comum para chamadas reais aos fornecedores.

Para tornar a solução configurável e escalável, foi criada uma *pipeline* interna de *interceptors*, inspirada no modelo de filtros e cadeia de filtros dos **Servlets** [8] [9]. Esta *pipeline* permite injetar comportamentos específicos tanto antes como após a chamada ao fornecedor de inferência ou ao próximo *interceptor* na *pipeline*, incluindo autenticação, autorização, métricas, *rate limiting*, *circuit breaker*, *fallback* e *logging*.

Para garantir que a **OmniAI Gateway** atue como uma peça de infraestrutura completa de inteligência artificial, a arquitetura incorpora um **MCP Broker**. Este módulo permite a exposição e centralização de múltiplas ferramentas externas através do **Model Context Protocol (MCP)** [10].

1.4 Estrutura do Documento

Este documento encontra-se organizado de forma progressiva, partindo do problema de integração e avançando até à validação da solução construída. O Capítulo 2 apresenta o estudo prévio: primeiro a análise comparativa das APIs de inferência e depois o estudo do **Model Context Protocol**. O Capítulo 3 descreve a arquitetura e desenho do sistema para suporte à inferência, incluindo a visão de alto nível e a modelação do domínio comum. O Capítulo 4 foca-se na implementação técnica e evolução do sistema, detalhando a Arquitetura Hexagonal, os vários adaptadores e portas, a *pipeline* de *interceptors*, a DSL, bem como os mecanismos de segurança, observabilidade e resiliência. O Capítulo 5 é dedicado exclusivamente ao **MCP Broker** e à integração com ferramentas externas, detalhando a arquitetura, desenho e implementação deste componente. O Capítulo 6 apresenta as provas de conceito efetivamente realizadas na **OmniAI Gateway** (*Proof of Concept*). O Capítulo 7 sintetiza as conclusões, limitações e linhas de evolução futura. O Apêndice B documenta o histórico de planeamento e identifica os principais riscos considerados durante a fase de desenvolvimento.

Capítulo 2

Estudo Prévio

2.1 Análise das APIs de Inferência

O estudo inicial comparou 3 APIs de inferência de utilização comum, sendo estas a **OpenAI** [1], **Anthropic** [2] e **Gemini** [3], para este estudo, utilizou-se o **Ollama** para analisar a API da **Anthropic**, o **Groq** para a da **OpenAI**, e o **AI Studio** da própria Google para avaliar a API do **Gemini**.

Este estudo não teve apenas um papel de descrever as APIs, mas sim servir como base e ponto de partida para observar os conceitos que podiam ser normalizados no **OmniAI SDK** e os pontos onde a normalização teria de preservar extensões específicas de cada fornecedor. Para esse efeito, foram executados e documentados pedidos focados apenas em formato textual de pedidos de geração simples, pedidos com *streaming*, pedidos com *tool calling*, respostas com contabilização de *tokens* e casos de configuração de geração.

O estudo das APIs de inferência mostrou que estas partilham diversos conceitos semelhantes, mas permitem a utilização dos mesmos de maneira diferente. A **OpenAI** organiza a interação em torno de `chat/completions`, com `messages`, `choices`, `tool_calls` e eventos incrementais baseados em `delta` [1]. A **Anthropic** usa uma API de `messages`, onde o `system prompt` está no nível de topo, o campo `max_tokens` é obrigatório, e o conteúdo da resposta é uma lista de blocos como `text`, `tool_use` ou `thinking` [2]. A API do **Gemini** modela o pedido como `contents` compostos por `parts`, agrupa parâmetros em `generationConfig`, representa ferramentas por `functionDeclarations` e introduz conceitos próprios como `thought signatures` e `thinkingConfig` [3]. Ao longo deste capítulo as diferenças e semelhanças vão ser apresentadas em mais detalhe.

2.1.1 Construção do Pedido

Construção do Pedido	OpenAI	Anthropic	Gemini
Autenticação	Cabeçalho <code>Authorization: Bearer <TOKEN></code> .	Cabeçalho <code>x-api-key</code> e obrigatoriedade de <code>anthropic-version</code> .	Cabeçalho <code>x-goog-api-key</code> ou parâmetro <code>?key=</code> no URI.
URI (<i>Endpoint</i>)	Fixo por tarefa (e.g., <code>.../v1/chat/completions</code>).	Fixo para mensagens (<code>.../v1/messages</code>).	Dinâmico, acoplado modelo e método (<code>.../models/{model}:{method}</code>).
Identificação do Modelo	Definido via propriedade <code>"model"</code> na raiz do <i>payload</i> JSON.	Definido via propriedade <code>"model"</code> na raiz do <i>payload</i> JSON.	Injetado dinamicamente no próprio URI de chamada HTTP .
Representação do Histórico	<i>Array messages</i> , composto por objetos com <code>role</code> e <code>content</code> .	<i>Array messages</i> , composto por objetos com <code>role</code> e <code>content</code> .	<i>Array contents</i> , composto por objetos com <code>role</code> e <code>parts</code> .
Corpo da Mensagem (Payload)	O campo <code>content</code> aceita uma <i>string</i> simples ou um <i>array</i> multimodal.	O campo <code>content</code> exige estritamente um <i>array</i> de blocos estruturados.	Os dados estão contidos num <i>array parts</i> composto por objetos tipados.
Instruções de Sistema	Tratadas como mensagem inicial com <code>role: "system"</code> .	Declaradas na propriedade isolada <code>"system"</code> na raiz do <i>payload</i> .	Declaradas no objeto isolado <code>"system_instruction"</code> na raiz.
Parâmetros de Geração	Propriedades soltas na raiz (e.g., <code>temperature</code> , <code>max_tokens</code>).	Propriedades soltas na raiz (<code>"max_tokens"</code> é obrigatório).	Agrupados num único objeto configuracional (<code>"generation Config"</code>).
Declaração de Ferramentas	Propriedade <code>tools[].function</code> contendo <code>parameters</code> .	Propriedade <code>tools[]</code> contendo <code>input_schema</code> .	Propriedade <code>tools[].functionDeclarations</code> contendo <code>parameters</code> .
Ativação de Streaming	Propriedade <code>"stream": true</code> na raiz do corpo JSON.	Propriedade <code>"stream": true</code> na raiz do corpo JSON.	URI distinto (<code>streamGenerateContent</code>) com parâmetro <code>?alt=sse</code> ; não existe campo no corpo.
Configuração de Raciocínio	Suporte nativo e automático na família de modelos <code>o1/o3</code> ; sem campo dedicado de ativação.	Objeto <code>"thinking"</code> na raiz, com <code>type</code> e <code>budget_tokens</code> para controlar o orçamento.	Objeto <code>thinkingConfig</code> dentro de <code>generationConfig</code> , com opções como <code>thinkingLevel</code> e <code>include_thoughts</code> .
Saídas Estruturadas	Controladas via <code>"response_format"</code> na raiz (e.g., <code>{"type": "json_object"}</code>).	Implementadas forçando uma ferramenta dedicada via <code>tool_choice</code> para extrair dados.	Definidas via <code>responseMimeType</code> e <code>responseJsonSchema</code> dentro de <code>generationConfig</code> .

Tabela 2.1: Mapeamento da estrutura de pedidos e configuração (*Requests*).

Com base nos pedidos executados durante o estudo, identificou-se os campos JSON mais relevantes de cada fornecedor. A Figura 2.1 indentifica as estruturas dessas mensagens, evidenciando as diferenças de organização e de nomenclatura entre os três fornecedores. Os exemplos JSON completos dos pedidos encontram-se no Apêndice A.1.

OpenAI Request	Anthropic Request	Gemini Request
<pre>{ "model": "gpt-4o", "messages": [{ "role": "system", "content": "..." }, { "role": "user", "content": "..." }], "temperature": 0.7, "max_tokens": 150, "tools": [{ "type": "function", "function": { "name": "func", "parameters": {...} } }], "tool_choice": "auto", "stream": false }</pre>	<pre>{ "model": "claude-3-5-sonnet", "system": "...", "messages": [{ "role": "user", "content": [{ "type": "text", "text": "..." }] }], "max_tokens": 150, "temperature": 0.7, "tools": [{ "name": "func", "input_schema": {...} }], "tool_choice": { "type": "auto" } }</pre>	<pre>URI: models/{model}:{method} { "contents": [{ "role": "user", "parts": [{"text": "..."}] }], "systemInstruction": { "parts": [{"text": "..."}] }, "generationConfig": { "temperature": 0.7, "maxOutputTokens": 150 }, "tools": [{ "functionDeclarations": [{ "name": "func", "parameters": {...} }] }] }</pre>

Figura 2.1: Estruturas JSON esquemáticas e simplificadas de pedidos para cada fornecedor.

A análise dos campos de pedido revela que, embora os três fornecedores partilhem conceitos equivalentes, cada um organiza o corpo JSON de forma distinta. A **OpenAI** [1] e a **Anthropic** [2] colocam a maioria dos parâmetros de configuração como propriedades na raiz do objeto, enquanto a **Gemini** agrupa-os dentro de `generationConfig` [3]. A diferença mais notável está na identificação do modelo. Neste caso, a **OpenAI** e a **Anthropic** incluem-na no corpo do pedido, enquanto a **Gemini** a incorpora diretamente no URI do pedido **HTTP**, tornando o *endpoint* dinâmico ou seja, o próprio URL da chamada varia consoante o modelo que se pretende utilizar. O *system prompt* também diverge consideravelmente: a **OpenAI** trata-o como uma mensagem com o papel `system` [1], a **Anthropic** eleva-o a uma propriedade de topo [2], e a **Gemini** define-o através de um objeto `system_instruction` separado [3]. Estas diferenças tornam evidente que uma tradução superficial dos nomes dos campos seria insuficiente para construir uma abstração e um domínio comum que seja capaz de cobrir os três fornecedores de inferência.

2.1.2 Processamento da Resposta

Processamento da Resposta	OpenAI	Anthropic	Gemini
Estrutura Raiz da Resposta	Objeto <code>choices[]</code> contendo <code>message</code> com <code>role</code> e <code>content</code> .	Diretamente na raiz: <code>role</code> , <code>content[]</code> , <code>stop_reason</code> e <code>usage</code> .	Objeto <code>candidates[]</code> contendo <code>content</code> com <code>role</code> e <code>parts[]</code> .
Invocação pelo Modelo (<i>Tool Call</i>)	Retorna <code>tool_calls[]</code> com <code>arguments</code> em <i>string</i> JSON serializada.	Retorna um bloco <code>type: "tool_use"</code> com <code>input</code> em JSON nativo.	Retorna uma <code>part</code> contendo <code>functionCall</code> com <code>args</code> (JSON nativo).
Resposta da Ferramenta	Adicionada como nova mensagem com <code>role: "tool"</code> e <code>tool_call_id</code> .	Adicionada sob <code>role: "user"</code> num bloco <code>type: "tool_result"</code> com <code>tool_use_id</code> .	Adicionada como nova mensagem contendo <code>functionResponse</code> com <code>name</code> e <code>response</code> .
Razão de Paragem	Campo <code>finish_reason</code> com valores como <code>"stop"</code> , <code>"tool_calls"</code> ou <code>"length"</code> .	Campo <code>stop_reason</code> com valores como <code>"end_turn"</code> , <code>"tool_use"</code> ou <code>"max_tokens"</code> .	Campo <code>finishReason</code> com valores como <code>"STOP"</code> ou <code>"MAX_TOKENS"</code> (notação em maiúsculas).
Saídas Estruturadas	Controladas via propriedade <code>"response_format"</code> na raiz.	Implementadas forçando uma ferramenta dedicada via <code>tool_choice</code> .	Definidas via <code>responseMimeType</code> e <code>responseJsonSchema</code> .
Formato de Eventos <i>Streaming</i>	Eventos SSE prefixados por <code>data: </code> , contendo fragmentos de texto no objeto <code>delta</code> , terminando com <code>[DONE]</code> .	Eventos SSE fortemente tipados (e.g., <code>content_block_delta</code>), exigindo um agregador de estado complexo no lado do cliente.	Eventos SSE contendo fragmentos do objeto completo, frequentemente exigindo a concatenação de texto extraído via <code>parts</code> .
Estrutura de Raciocínio (<i>Reasoning</i>)	Os <i>tokens</i> gerados ficam maioritariamente ocultos (na família o1/o3), não sendo expostos como texto regular no campo <code>content</code> .	Devolvido explicitamente no <i>array</i> <code>content</code> como um bloco do tipo <code>thinking</code> , fisicamente separado do bloco <code>text</code> final.	Emitido através de blocos de pensamento nativos ou visível a nível de metadados na agregação de <code>thoughtsTokenCount</code> .
Contabilização de <i>Tokens</i>	Objeto <code>usage</code> que já inclui o somatório total pronto a usar em <code>total_tokens</code> .	Objeto <code>usage</code> contendo apenas <code>input_tokens</code> e <code>output_tokens</code> (total tem de ser calculado pelo cliente).	Objeto <code>usageMetadata</code> com <code>promptTokenCount</code> , <code>candidatesTokenCount</code> e <code>totalTokenCount</code> .
Rastreabilidade da Resposta	Identificadores <code>id</code> (e.g., <code>chatcmpl-...</code>) e metadados como <code>system_fingerprint</code> e <code>timestamp (created)</code> .	Campo <code>id</code> com prefixo padronizado (e.g., <code>msg...</code>) e identificação estrutural <code>type: "message"</code> .	Metadados na raiz, incluindo <code>responseId</code> e a versão exata do modelo em <code>modelVersion</code> .

Tabela 2.2: Mapeamento da estrutura de respostas, formatação de eventos e rastreabilidade.

À semelhança dos pedidos, a estrutura JSON das respostas foi esquematizada para evidenciar as diferenças e semelhanças de cada resposta de cada fornecedor. A Figura 2.2 apresenta essa comparação estrutural. Os exemplos JSON completos das respostas encontram-se no Apêndice A.2.

OpenAI Response	Anthropic Response	Gemini Response
<pre>{ "id": "chatcmpl-...", "choices": [{ "index": 0, "message": { "role": "assistant", "content": "...", "tool_calls": [{ "id": "call_...", "type": "function", "function": { "name": "func", "arguments": "{\\\"...\\\"}" } }] }, "finish_reason": "stop" }], "usage": { "prompt_tokens": 10, "completion_tokens": 20 } }</pre>	<pre>{ "id": "msg-...", "role": "assistant", "content": [{ "type": "text", "text": "..." }, { "type": "tool_use", "id": "toolu_...", "name": "func", "input": {...} }], "stop_reason": "end_turn", "usage": { "input_tokens": 10, "output_tokens": 20 } }</pre>	<pre>{ "candidates": [{ "content": { "role": "model", "parts": [{ "text": "..."}, { "functionCall": { "name": "func", "args": {...} } }] }, "finishReason": "STOP" }], "usageMetadata": { "promptTokenCount": 10, "candidatesTokenCount": 20 } }</pre>

Figura 2.2: Estruturas JSON esquemáticas e simplificadas de respostas para cada fornecedor.

A análise às respostas mostra que, embora os conceitos base sejam idênticos, a forma como cada API organiza os dados é muito diferente. A interface da **OpenAI** integra a resposta dentro de um *array* denominado **choices**, que contém um objeto **message** [1]; a interface da **Anthropic** expõe o conteúdo diretamente na raiz através de um *array* **content** com blocos tipados [2]; e a interface da **Gemini** utiliza **candidates** com **parts** e conceitos adicionais, como **thoughtSignature** [3].

Um aspeto particularmente relevante é a representação dos argumentos em chamadas de ferramenta: a **OpenAI** serializa-os como uma *string* JSON dentro do campo **arguments**, obrigando a uma desserialização adicional [1], enquanto a **Anthropic** e a **Gemini** devolvem JSON nativo nos campos **input** [2] e **args** [3], respetivamente. A contabilização de *tokens* também difere na nomenclatura e granularidade: a API do **Gemini**, por exemplo, expõe um campo adicional **thoughtsTokenCount** que permite distinguir os *tokens* de raciocínio dos *tokens* de conteúdo efetivo [3].

2.1.3 Divergências na Padronização de Esquemas de Validação

Para além das diferenças do corpo JSON, é importante referir a divergência na forma como cada fornecedor padroniza a tipagem de dados estruturados, nomeadamente na declaração de ferramentas (*tools*) e nas saídas estruturadas (*structured outputs*).

Enquanto a **OpenAI** [24] e a **Anthropic** [25] se apoiam nativamente na norma *JSON Schema* [26] para definir propriedades e parâmetros, a API do **Gemini** adota um subconjunto da especificação *OpenAPI 3.0* [27] (através do seu objeto **Schema** [28]). Isto reforça que a construção de uma camada de abstração exige não só o mapeamento de chaves, mas também a tradução e compatibilização entre diferentes definições de esquemas.

2.1.4 Conclusão do Estudo das APIs de Inferência

A realização deste estudo envolveu dificuldades práticas que merecem ser documentadas. Para a API da **OpenAI**, o acesso direto à API oficial implica custos de utilização, pelo que os testes foram realizados utilizando a API da **Groq** [15], que oferece um plano gratuito e implementa uma interface compatível com o formato da **OpenAI** (`/v1/chat/completions`). Adicionalmente, foram executados testes complementares contra uma instância local de **Ollama** [29], que também expõe um *endpoint* compatível com o formato **OpenAI**. Para a API da **Anthropic**, a situação foi semelhante: a API oficial requer um plano de subscrição pago, pelo que os testes foram realizados utilizando o **Ollama**, que disponibiliza um *endpoint* compatível com o formato da **Anthropic** (`/v1/messages`). Embora o **Ollama** implemente corretamente as interfaces de compatibilidade para pedidos simples, de *streaming* e com parâmetros de configuração, algumas funcionalidades mais avançadas, nomeadamente o *tool calling* estruturado e a geração de blocos de raciocínio (*thinking*) apresentaram limitações quando executadas em modelos de pequena dimensão, como o `llama3.2:1b`. Nesses casos, os exemplos de resposta foram ajustados para refletir com fidelidade o comportamento documentado das APIs oficiais. A API do **Gemini** foi a única cujos testes foram executados diretamente contra a API oficial, nomeadamente através do **AI Studio** [30], uma vez que este disponibiliza um nível de acesso gratuito.

Do ponto de vista técnico, o estudo permitiu concluir que as três APIs partilham um conjunto de conceitos fundamentais, como as mensagens, papéis, ferramentas, *streaming* e contabilização de *tokens*, mas implementam-nos de formas estruturalmente diferentes. As diferenças manifestam-se na nomenclatura dos campos, na organização hierárquica do JSON, na localização dos parâmetros de configuração e nos mecanismos de ativação de funcionalidades como *streaming* e raciocínio. Contudo, apesar destas similaridades conceituais, ainda existem campos proprietários em cada API de fornecedor de inferência. Esta análise permitiu perceber que uma abstração que pretenda cobrir estes três fornecedores precisa de ir além de uma simples tradução de nomes, exigindo uma compreensão das diferenças semânticas e estruturais observadas nos pedidos e respostas reais.

2.2 O protocolo MCP

O **Model Context Protocol (MCP)** é um protocolo desenvolvido pela Anthropic com o objetivo de normalizar a forma como aplicações consumidoras de IA, nomeadamente Agentes, disponibilizam contexto externo às suas APIs de inferência [31]. Em vez de cada integração definir uma interface própria para expor ferramentas, dados ou operações, o **MCP** estabelece uma interface comum entre aplicações, clientes MCP e servidores MCP, permitindo reutilizar as mesmas capacidades em diferentes ambientes sem acoplamento direto a um fornecedor específico.

Antes do surgimento do **MCP**, cada plataforma disponibilizava mecanismos próprios para expor ferramentas externas. Embora estes mecanismos oferecessem funcionalidades semelhantes, apresentavam interfaces incompatíveis entre si, obrigando os programadores a implementar adaptações específicas para cada API. Esta situação dificultava a reutilização de integrações, aumentava o esforço de desenvolvimento e criava um forte acoplamento entre as aplicações e os fornecedores de inferência.

Alem de padronizar as integrações, o protocolo mitiga o problema do isolamento das APIs de inferência. Os modelos de linguagem operam apenas sobre o contexto recebido, sem acesso nativo a sistemas externos ou a outras fontes de dados dinâmicas. O **MCP** resolve esta limitação ao oferecer uma arquitetura que permite à aplicação consultar ficheiros e aceder a serviços externos para, essencialmente, fornecer contexto ao modelo.

2.2.1 Arquitetura do protocolo

O protocolo segue uma arquitetura cliente-servidor composta por três participantes: a aplicação consumidora de IA, que integra um ou mais clientes **MCP**; o cliente **MCP**, que estabelece e mantém as ligações com os servidores; e o servidor **MCP**, que expõe capacidades como ferramentas, recursos e *prompts* [12,32,33]. Nesta arquitetura, os servidores assumem o papel de fornecedores de contexto e funcionalidades externas. O cliente atua como o intermediário que descobre as funcionalidades que o servidor tem para oferecer e executa as operações necessárias a pedido da aplicação principal. Ao contrário das APIs de inferência, responsáveis pela geração e interpretação de respostas, o **MCP** define exclusivamente a comunicação com fontes externas de contexto. Esta separação permite isolar o acesso a ferramentas do pedido de inferência, tornando ambas as componentes independentes.

A Figura 2.3 ilustra esta separação de responsabilidades. Do lado da inferência, a aplicação comunica com o fornecedor através de uma API de inferência. Do lado do contexto externo, o cliente **MCP** estabelece ligação a um ou mais servidores **MCP**, cada um responsável por disponibilizar um conjunto de capacidades específicas.

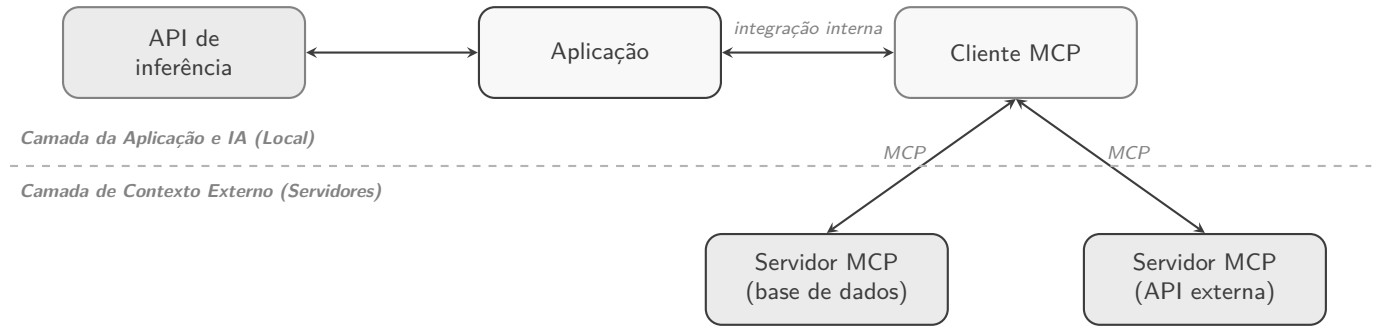


Figura 2.3: Arquitetura simplificada do MCP

Assim, o cliente não necessita de conhecer previamente quais destas capacidades existem. Após estabelecer ligação ao servidor, pode solicitar dinamicamente a lista de ferramentas, recursos e *prompts* disponibilizados, adaptando automaticamente o seu comportamento às funcionalidades de contexto oferecidas pelo servidor.

2.2.2 Formato da Comunicação

No contexto do MCP, todas as interações entre o cliente e o servidor, desde o transporte de contexto externo até à negociação e invocação de ferramentas são realizadas através da troca de mensagens **JSON-RPC 2.0** [34]. O JSON-RPC é um protocolo de Chamada de Procedimento Remoto (RPC, *Remote Procedure Call*) que utiliza o formato de dados JSON nas suas estruturas de comunicação.

Este protocolo assenta em três pilares fundamentais. Em primeiro lugar, destaca-se por possuir uma forte interoperabilidade, sendo amplamente suportado e independente da linguagem de programação, garantindo que clientes e servidores consigam comunicar sem terem problemas de compatibilidade.

Em segundo lugar, a sua estrutura simples adequa-se ao modelo do MCP. A especificação contém três tipos de mensagens distintos: os *Requests*, que representam pedidos iniciados por qualquer uma das partes que aguardam uma resposta; as *Responses*, que transportam o resultado ou erro emitido em resposta a um pedido; e as *Notifications*, que servem para mensagens assíncronas que não requerem resposta como o envio contínuo de eventos ou atualizações.

Por fim, ao utilizar na sua estrutura o formato JSON, como referido anteriormente, o conteúdo das mensagens, tal como a estrutura de parâmetros necessários para executar uma ferramenta, pode ser

validado de forma direta e padronizada com recurso a JSON Schema.

2.2.3 Descoberta de capacidades

Uma das principais características do protocolo é a negociação dinâmica de capacidades (*capability discovery*), realizada durante a fase de inicialização da sessão. O cliente inicia este processo enviando um *request* JSON-RPC, no qual declara a versão do protocolo suportada e as suas próprias capacidades. O servidor responde indicando a versão de protocolo acordada e o conjunto de categorias de primitivas que disponibiliza como *tools*, *resources* e *prompts*, podendo ainda especificar funcionalidades adicionais associadas a cada uma, como a emissão de notificações de alteração (**listChanged**) ou o suporte para subscrição de atualizações de recursos (**subscribe**). É importante notar que nesta negociação, o servidor indica apenas *que tipos* de funcionalidades suporta, sem ainda enumerar os elementos concretos que disponibiliza [32].

Só depois de concluída a inicialização (confirmada através de uma notificação **initialized** enviada pelo cliente) é que este pode consultar o conteúdo efetivo de cada categoria previamente anunciada. Esta descoberta detalhada é feita através de três operações definidas pelo protocolo: **tools/list**, **resources/list** e **prompts/list**, que devolvem, respetivamente, a lista de ferramentas, recursos e *prompts* disponíveis, incluindo os respetivos nomes, descrições e esquemas de parâmetros (no caso das ferramentas e *prompts*, definidos através de *JSON Schema*). A invocação efetiva de uma ferramenta, a leitura de um recurso ou a obtenção de um *prompt* são depois realizadas pelas operações correspondentes, **tools/call**, **resources/read** e **prompts/get**, respetivamente [32,33]. Desta forma, diferentes servidores podem disponibilizar conjuntos distintos de funcionalidades sem que seja necessário alterar a lógica da aplicação cliente, que se limita a reagir às capacidades e aos elementos anunciados em cada fase.

Quando a resposta da API de inferência indica a necessidade de executar uma ação externa, a aplicação utiliza o cliente MCP para selecionar a ferramenta apropriada (entre as previamente obtidas via **tools/list**), enviar os parâmetros necessários ao servidor e receber posteriormente o resultado da operação. A Figura 2.4 representa o fluxo completo de comunicação, desde o estabelecimento da sessão até à execução de uma ferramenta.

Para além das mensagens de pedido-resposta, o protocolo define ainda um segundo tipo de mensagem JSON-RPC, as *notifications*, que permitem ao servidor informar o cliente de eventos assíncronos como a alteração da lista de ferramentas disponíveis (**notifications/tools/list_changed**) ou a atualização de um recurso subscrito sem que seja necessário um pedido prévio por parte deste, dado não exigirem resposta.

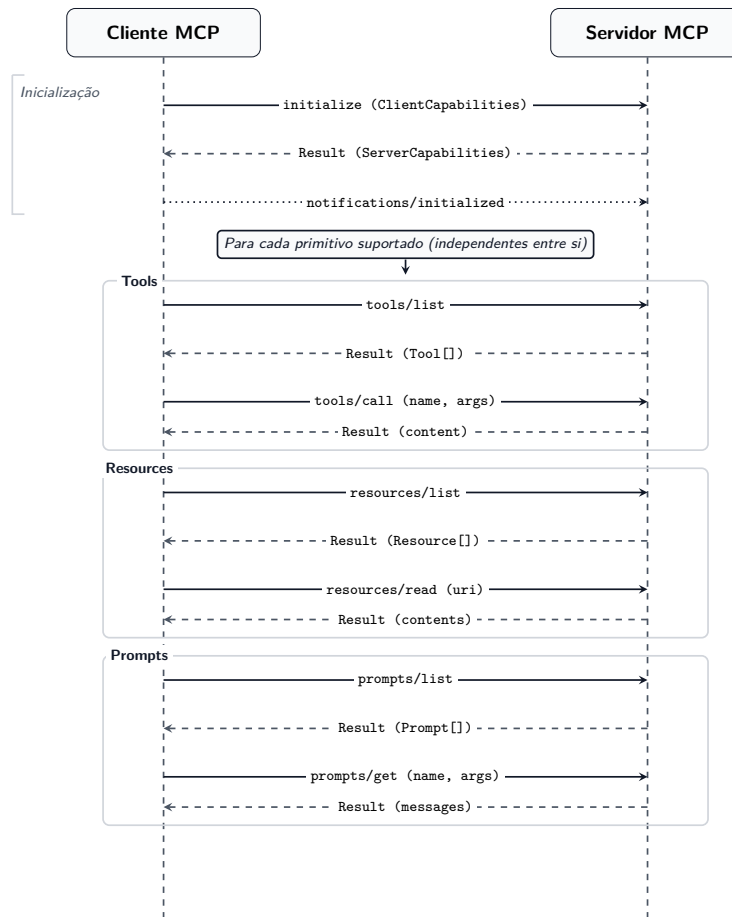


Figura 2.4: Fluxo de comunicação MCP - Adaptado da Documentação do MCP [12]

2.2.4 Mecanismos de Transporte

O protocolo define o formato das mensagens e o ciclo de comunicação entre cliente e servidor, mas permite que essa comunicação seja realizada através de diferentes mecanismos de transporte. A versão atual da especificação define dois mecanismos oficiais: **STDIO** e **Streamable HTTP** [12], cujas diferenças estão resumidas na Tabela 2.3.

Característica	STDIO	Streamable HTTP
Âmbito principal	Comunicação local entre a aplicação e um processo servidor iniciado na mesma máquina.	Comunicação remota ou distribuída entre clientes e servidores acessíveis por rede.
Canal de comunicação	Fluxos padrão do processo: <code>stdin</code> para pedidos e <code>stdout</code> para respostas.	Um único endpoint HTTP (suporta <code>POST</code> e <code>GET</code>) para troca de mensagens JSON-RPC.
Modelo operacional	O cliente lança e controla o ciclo de vida do processo servidor.	O servidor é executado de forma independente e pode ser partilhado por vários clientes.
Sessão e estado	Adequado a sessões locais simples, normalmente associadas a uma instância da aplicação.	Adequado a sessões remotas, autenticação, infraestrutura partilhada e cenários multiutilizador.
<i>Streaming</i>	Não aplicável: a troca ocorre diretamente pelos fluxos do processo, sem depender de mecanismos de rede.	Opcional: o mesmo <i>endpoint</i> responde de forma direta (JSON) ou, quando necessário, converte-se num <i>stream</i> SSE para enviar múltiplas mensagens ou notificações, sem recorrer a <i>endpoints</i> adicionais.
Caso de uso típico	Aplicações desktop e ferramentas locais que precisam de integrar servidores MCP sem expor rede.	Servidores remotos, integrações partilhadas, ambientes <i>cloud</i> e arquiteturas distribuídas.

Tabela 2.3: Comparação entre os dois mecanismos de transporte oficiais do MCP.

O transporte **STDIO** destina-se principalmente à comunicação entre processos executados localmente. Neste modo, o cliente inicia o processo correspondente ao servidor MCP e toda a comunicação é realizada através do *standard input* e *standard output*. Esta abordagem apresenta uma implementação simples e elimina a necessidade de configurar ligações de rede, sendo por isso adequada a aplicações de ambiente de trabalho e ferramentas executadas no mesmo ambiente da aplicação cliente.

O transporte **Streamable HTTP** destina-se à comunicação entre processos distribuídos através da rede. Neste caso, as mensagens JSON-RPC são trocadas através de um único *endpoint* HTTP, que aceita pedidos `POST` e `GET`. A resposta do servidor pode ser devolvida diretamente ou, quando há necessidade de enviar várias mensagens ou notificações, a ligação é mantida aberta e passa a funcionar como um *stream* SSE, sem que seja necessário recorrer a um *endpoint* adicional. Este mecanismo substitui o transporte *HTTP+SSE*, definido na versão da especificação de novembro de 2024 (2024-11-05), que assentava em dois *endpoints* distintos, um dedicado à ligação SSE (`/sse`) e outro ao envio de mensagens (`/messages`) [12]. A partir da revisão de março de 2025 (2025-03-26), esta estrutura foi simplificada para um único *endpoint*, sem eliminar o uso de SSE, que passou a funcionar apenas como mecanismo opcional de *streaming* sobre a mesma ligação.

Embora a especificação atual privilegie estes dois mecanismos, diversas implementações disponibilizam suporte adicional para outros transportes, nomeadamente **WebSocket**. Apesar de não constituir um mecanismo oficial do protocolo, o WebSocket é frequentemente utilizado em aplicações que necessitam de comunicação bidirecional persistente e de baixa latência.

2.2.5 Autenticação e Autorização

A abordagem do protocolo MCP à autenticação e à autorização depende do mecanismo de transporte utilizado. No caso de comunicações locais através de **STDIO**, a especificação não define um mecanismo de autenticação próprio, uma vez que assume que a segurança é assegurada pelo sistema operativo. Neste cenário, o controlo de acesso é garantido pelas permissões do utilizador e pelo isolamento do processo que

executa o servidor MCP.

Em contrapartida, quando a comunicação ocorre através de **Streamable HTTP**, tipicamente em ambientes distribuídos ou remotos, a especificação recomenda a utilização do padrão **OAuth 2.1**. Neste modelo, o servidor MCP desempenha o papel de *Resource Server*, aceitando apenas pedidos autenticados com *access tokens* válidos, previamente obtidos pelo cliente junto de um *Authorization Server* (AS).

Além da autenticação, o MCP utiliza *OAuth scopes*, ou seja, identificadores que representam permissões, como ler um ficheiro ou executar uma funcionalidade para gerir o que cada agente de IA pode fazer. Isto permite configurar quais as ferramentas e os dados que são disponibilizados ao cliente, garantindo que o agente tem apenas acesso aos recursos estritamente necessários para executar a tarefa.

2.2.6 Ferramentas de Inspeção

Durante o desenvolvimento do protocolo foram igualmente utilizadas ferramentas que facilitam a validação de implementações. Entre estas destaca-se o **MCP Inspector**, ferramenta oficial de desenvolvimento distribuída através do ecossistema Node.js [35].

Esta ferramenta permite estabelecer ligação a um servidor **MCP**, visualizar interativamente as capacidades disponibilizadas, invocar ferramentas, consultar recursos e analisar as mensagens JSON-RPC trocadas durante toda a sessão.

2.2.7 Protótipo Desenvolvido

Para complementar o estudo da especificação foi desenvolvido um protótipo utilizando a biblioteca oficial `io.modelcontextprotocol:kotlin-sdk` [36]. O objetivo deste protótipo consistiu em compreender o ciclo completo de funcionamento do protocolo e avaliar a maturidade da biblioteca para futura integração na arquitetura da **OmniAI**.

O trabalho realizado incluiu a implementação de um servidor **MCP**, responsável pelo registo de ferramentas e restantes capacidades disponibilizadas, e de um cliente **MCP**, responsável por estabelecer ligação ao servidor, realizar o processo de inicialização, descobrir dinamicamente as capacidades disponíveis e invocar as ferramentas registadas.

Durante o desenvolvimento foram explorados os diferentes mecanismos de transporte suportados pela biblioteca, tendo sido igualmente utilizado o **MCP Inspector** [35] para validar o correto funcionamento da comunicação entre cliente e servidor e verificar a troca de mensagens definida pela especificação.

2.3 Estudo de Soluções Semelhantes

Antes de iniciarmos o desenvolvimento deste projeto, realizámos uma pesquisa sobre as **Gateways de IA** já existentes no mercado. O objetivo deste estudo prévio foi analisar as soluções disponíveis e compreender as diferentes abordagens utilizadas. Desta pesquisa, destacaram-se as seguintes soluções semelhantes:

- **LiteLLM** [21]: Uma solução amplamente adotada que atua como um *proxy* central para unificar chamadas a dezenas de fornecedores utilizando o formato da **OpenAI**. Destaca-se por fornecer funcionalidades prontas de balanceamento de carga e controlo de orçamentos.
- **RouteLLM** [37]: Uma *framework* focada no roteamento inteligente de pedidos de inferência. A sua principal proposta de valor é a otimização de custo e latência, reencaminhando pedidos mais simples para fornecedores mais baratos e rápidos, e reservando configurações de maior capacidade para tarefas mais complexas.

- **Cloudflare AI Gateway** [38]: Integrado na rede global da Cloudflare, este serviço posiciona a gestão das chamadas na *edge*, proporcionando benefícios significativos a nível de latência e de *caching* de respostas sem exigir gestão de infraestrutura.
- **OpenRouter** [22]: Funciona como um ponto de unificação que disponibiliza acesso a varios modelos através de uma única API. A sua principal vantagem é a interoperabilidade e a facilidade de mudar entre diferentes fornecedores, conseguido ainda ter um mercado centralizado de modelos, o que simplifica drasticamente a gestão de credenciais e a faturação para os utilizadores.

Para além das soluções comerciais, a necessidade de gerir o acesso a modelos levou empresas a desenvolver as suas próprias *gateways* internas. A Uber[39], por exemplo, integrou este componente na sua plataforma de machine learning (Michelangelo) para resolver a fragmentação de acessos e garantir um controlo de segurança unificado. Esta infraestrutura revelou-se indispensável para coordenar vários agentes de IA a serem utilizados em simultâneo, exigindo uma rigorosa gestão centralizada de identidades e permissões.

Capítulo 3

Arquitetura e Desenho do Sistema

A arquitetura de alto nível apresentada na Figura 3.1 posiciona o **OmniAI SDK** como o principal resultado deste projeto. Em vez de constituir uma aplicação final isolada, o **OmniAI SDK** fornece um conjunto de abstrações, componentes e mecanismos reutilizáveis que permitem construir soluções de integração com múltiplos fornecedores de modelos de linguagem de forma consistente. Esta estrutura engloba abstrações (como o domínio comum de pedidos e respostas e os contratos normalizados), componentes (como os adaptadores responsáveis pela tradução entre o modelo interno e as APIs externas) e mecanismos (como o *pipeline* de interceptação para funcionalidades transversais e uma DSL de configuração que simplifica a composição do sistema).

A utilização do **OmniAI SDK** traduz-se, na prática, na criação de uma **OmniAI Gateway** adaptada às necessidades de cada cenário. A **Gateway** funciona como um intermediário de acesso unificada sobre diferentes fornecedores de modelos, abstraindo as diferenças existentes entre APIs e disponibilizando uma interface homogênea para clientes e aplicações consumidoras.

Para validar a arquitetura proposta e demonstrar a aplicabilidade do nosso *Software Development Kit* num cenário real, foi desenvolvida uma instância concreta da **OmniAI Gateway** como Prova de Conceito (*Proof of Concept* – PoC). Esta instância é construída integralmente através dos mecanismos disponibilizados pelo **OmniAI SDK**, recorrendo a configurações declarativas para definir fornecedores, modelos, políticas de autenticação, métricas e restantes componentes da infraestrutura. Durante o arranque da aplicação, estas configurações são utilizadas para compor os adaptadores, a *pipeline* de processamento e os conectores **HTTP** baseados em **Ktor** [39], expondo um conjunto de *endpoints* compatíveis com os contratos normalizados definidos pelo domínio comum.

Esta separação entre **OmniAI SDK** e **OmniAI Gateway** constitui uma das principais decisões arquiteturais do projeto. A biblioteca concentra toda a lógica reutilizável e independente do contexto de execução, enquanto cada **Gateway** representa uma instanciação específica dessa infraestrutura, podendo ser configurada para diferentes fornecedores, requisitos operacionais ou cenários de utilização sem necessidade de alterar os componentes centrais do sistema.

Embora o foco central deste projeto e do presente capítulo resida no acesso a APIs de inferência e na construção de uma Gateway em torno das mesmas, uma infraestrutura de IA completa exige componentes adicionais. Nesse sentido, foi posteriormente integrado no **SDK** um **MCP Broker**, que atua em paralelo com a Gateway de inferência, sendo responsável pela exposição e gestão de ferramentas externas via *Model Context Protocol*. Os detalhes arquiteturais específicos de implementação não se encontram nesta secção, estando concentrados no Capítulo 5.

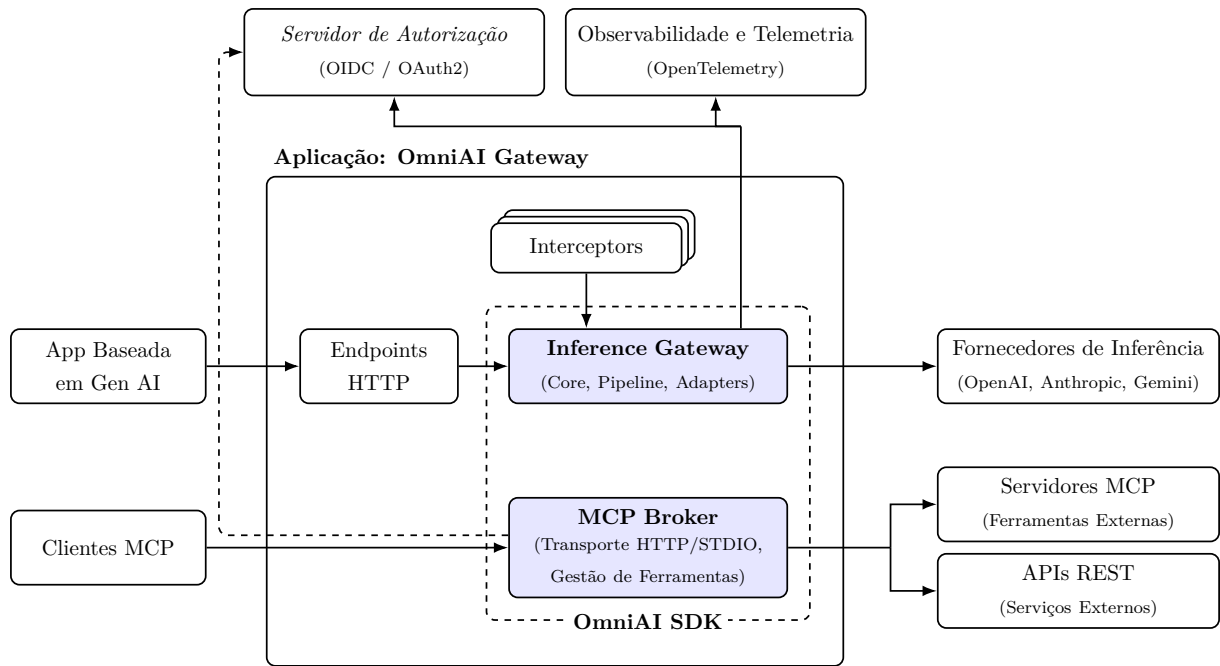


Figura 3.1: Arquitetura de alto nível: **OmniAI SDK** integrada numa Gateway, nomeadamente uma **OmniAI Gateway**.

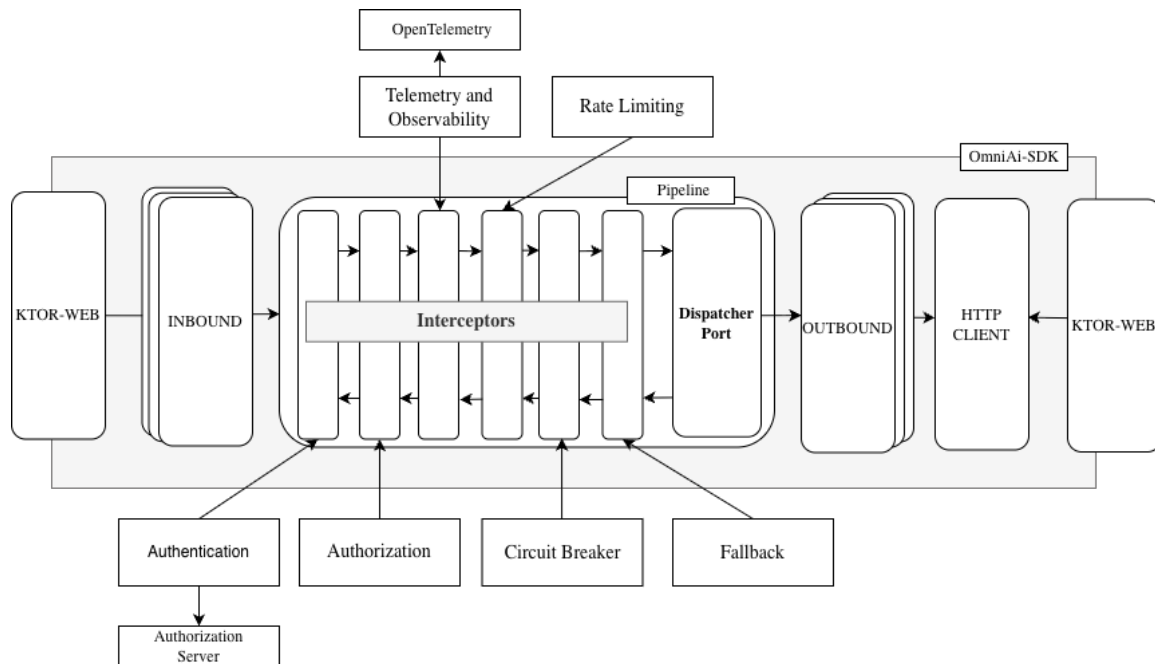


Figura 3.2: Visão da integração entre **Ktor**, *inbounds*, *pipeline*, *interceptors*, *outbounds* e **HTTP** client no **OmniAI SDK**

A Figura 3.1 mostra que a **OmniAI Gateway** não contém a lógica essencial de tradução, despacho ou resiliência. Essa lógica pertence ao **OmniAI SDK** e é apenas instanciada pela aplicação final. A Figura 3.2 aproxima a visão interna isto é um conector **HTTP** recebe o corpo do pedido, converte-o para um DTO de contrato, invoca um *inbound adapter*, e este chama um **DispatcherPort**. A partir desse ponto, a execução pode seguir por um *dispatcher* direto ou por um *dispatcher* apoiado numa *pipeline*.

No final, um *outbound adapter* traduz o pedido comum para o contrato do fornecedor e utiliza um cliente **HTTP** abstrato para executar a chamada real.

Esta organização permite que aplicações já compatíveis com APIs existentes possam ser integradas com esforço reduzido. Por exemplo, uma aplicação preparada para a API **Anthropic** pode alterar o seu `baseURL` para apontar para a **OmniAI Gateway**, mantendo o contrato esperado pelo cliente. O mesmo princípio aplica-se a clientes compatíveis com a API de inferência da **OpenAI** e API de inferência do **Gemini**.

A arquitetura foi também pensada para escalamento horizontal: o **OmniAI SDK** evita estado global obrigatório, concentra estado transitório no contexto do pedido e recorre à injeção de dependências através de interfaces, permitindo a substituição de *stores* em memória por *backends* partilhados quando a **Gateway** opera em múltiplas instâncias

3.1 Modelação do Domínio Comum

3.1.1 Visão Global do Domínio

No final do estudo prévio realizado no capítulo anterior, verificou-se uma acentuada fragmentação estrutural entre os principais fornecedores de Modelos de Linguagem de Grande Escala (LLMs) [13]. A ausência de uma padronização ou um protocolo comum leva à inconsistência entre as APIs da **OpenAI** [1], **Anthropic** [2] e **Gemini** [3], as quais têm campos distintos, diferentes ciclos de vida para o fluxo de dados e estruturas de objetos incompatíveis entre as várias APIs.

Para garantir a interoperabilidade, a arquitetura do **OmniAI SDK** assenta na criação de um modelo de domínio comum. A ideia foi criar um modelo de dados interoperável com os maiores fornecedores de inferência para representar pedidos, respostas e eventos dos mesmos, mas sem copiar diretamente a estrutura de nenhum deles.

Assim, o domínio funciona como um ponto intermédio estável que centraliza qualquer interação no **OmniAI SDK**. Isto significa que, independentemente de o cliente estruturar o seu pedido original no formato de um fornecedor específico (como por exemplo a **Anthropic**), esse contrato é imediatamente convertido para o domínio unificado e em seguida, para efetuar a chamada à API, o modelo comum é novamente traduzido para o formato nativo do fornecedor de destino. O mesmo processo inverso ocorre na receção da resposta, os dados que são devolvidos pela API externa são normalizados no domínio comum antes de serem entregues ao cliente na estrutura esperada. Este fluxo completo de transformações está representado visualmente na Figura 3.1.

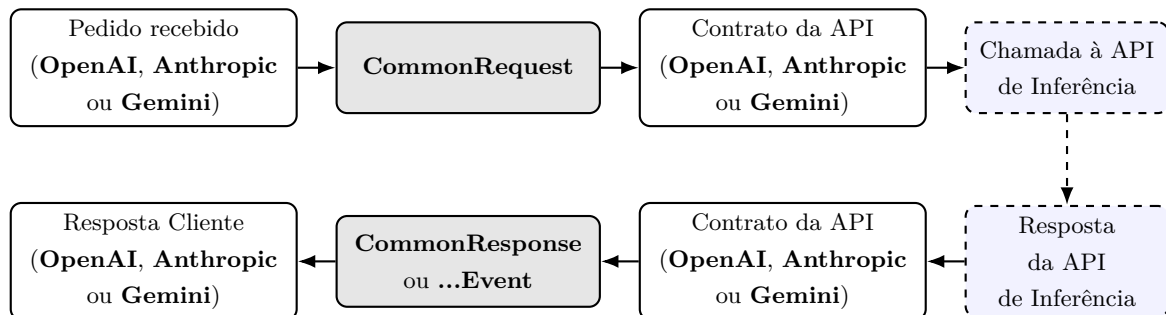


Figura 3.3: Fluxo de normalização entre contratos externos e domínio comum (**OmniAi-SDK**)

3.1.2 Estrutura do Domínio Comum

O domínio comum do **OmniAI SDK** que consiste no conjunto de classes e relações que padroniza as interações com os diferentes fornecedores divide-se em dois blocos principais: **CommonRequest** e **CommonResponse/CommonResponseEvent**.

- O **CommonRequest** define a estrutura dos pedidos enviados ao modelo.
- O **CommonResponse/CommonResponseEvent** descreve as respostas devolvidas, quer no formato incremental ou através de eventos.

Modelo Comum de Pedido

O modelo de pedido comum, designado por **CommonRequest**, é a base que normaliza qualquer pedido a LLMs, garantindo um contrato único e consistente, independentemente do fornecedor. A sua estrutura foi desenhada para manter os elementos essenciais da geração de texto – nomeadamente o fornecedor, o modelo e as mensagens – permitindo, simultaneamente, extensões controladas através de configuração e opções específicas do fornecedor. Como se observa na Figura 3.2, o pedido comum agrega diretamente as propriedades **provider**, **model** e a lista de **messages**, que representam o histórico conversacional. Cada mensagem do pedido é materializada em **CommonRequestMessage** e contém o papel da entidade comunicante na propriedade **CommonRole**, bem como o conteúdo dividido em partes através de **RequestContentPart**. Esta divisão permite combinar texto, JSON ou chamadas a ferramentas. A configuração de geração é isolada na classe **CommonGenerationConfig**, enquanto as ferramentas disponíveis são descritas pela classe **CommonTool** e controladas pela política **ToolChoice**. Por fim, as propriedades globais **SystemPrompt** e específicas **providerOptions** suportam personalizações de contexto.

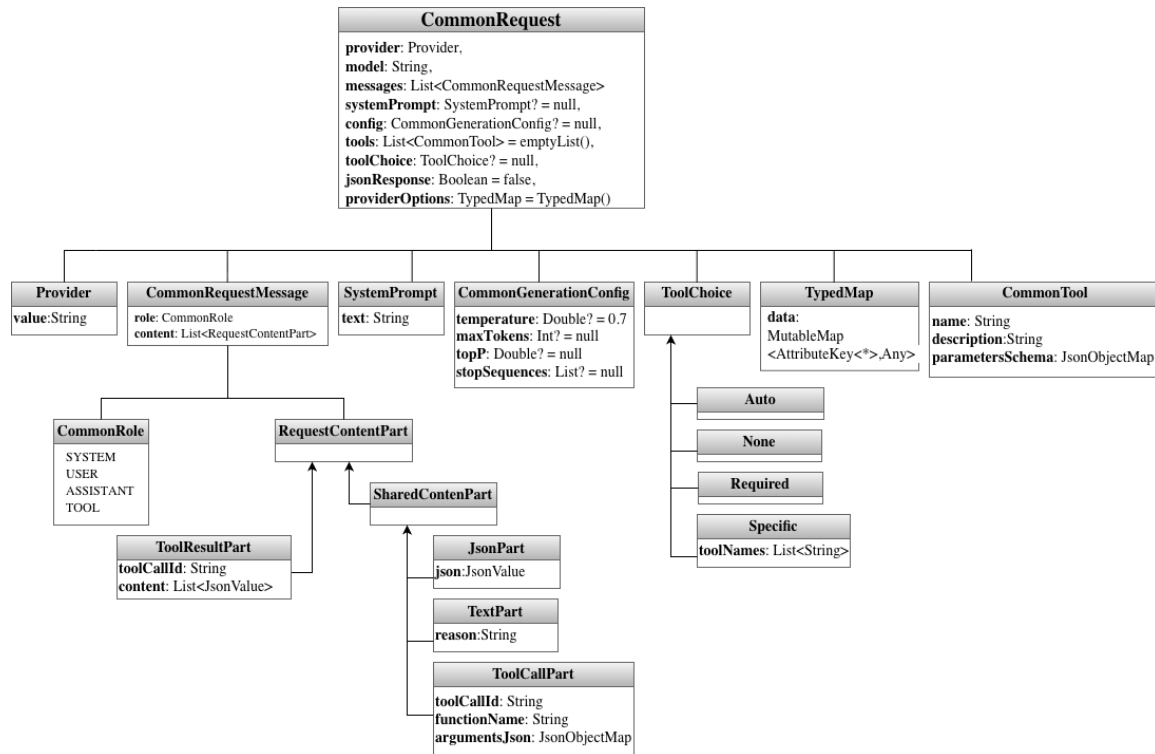


Figura 3.4: Estrutura de classes do **CommonRequest**.

Para sistematizar a estrutura apresentada na Figura 3.2, a Tabela 3.1 agrega todos os parâmetros

relevantes do domínio de *Request*, incluindo o objeto principal e os tipos diretamente associados. O objetivo é apresentar o significado funcional de cada campo no fluxo de construção do pedido.

Elemento	Campo	Tipo	Descrição
CommonRequest	provider	Provider	Fornecedor alvo da chamada.
CommonRequest	model	String	Identificador do modelo a utilizar.
CommonRequest	messages	List<Common-RequestMessage>	Histórico de mensagens enviado ao modelo.
CommonRequest	systemPrompt	SystemPrompt?	Prompt de sistema global (opcional).
CommonRequest	config	Common-GenerationConfig?	Parâmetros de geração (opcional).
CommonRequest	tools	List<CommonTool>	Ferramentas disponíveis para chamadas de função.
CommonRequest	toolChoice	ToolChoice?	Política de seleção de ferramentas.
CommonRequest	jsonResponse	Boolean	Indica se a resposta deve ser forçada a JSON.
CommonRequest	providerOptions	TypedMap	Opções específicas do fornecedor.
CommonRequest-Message	role	CommonRole	Papel da mensagem (SYSTEM, USER, ASSISTANT, TOOL).
CommonRequest-Message	content	List<Request-ContentPart>	Conteúdo segmentado da mensagem.
SystemPrompt	text	String	Conteúdo do prompt de sistema.
Common-GenerationConfig	temperature	Double?	Grau de aleatoriedade da geração.
Common-GenerationConfig	maxTokens	Int?	Limite máximo de <i>tokens</i> de saída.
Common-GenerationConfig	topP	Double?	Probabilidade cumulativa.
Common-GenerationConfig	stopSequences	List<String>?	Sequências que interrompem a geração.
CommonTool	name	String	Nome da ferramenta.
CommonTool	description	String	Descrição funcional da ferramenta.
CommonTool	parametersSchema	JsonObjectMap	Esquema JSON dos parâmetros.
ToolChoice	Auto / None / Required / Specific	–	Estratégia de uso de ferramentas (auto, nenhuma, obrigatório ou lista específica).
ToolChoiceAuto	–	–	Subtipo sem campos (seleção automática).
ToolChoiceNone	–	–	Subtipo sem campos (sem ferramentas).
ToolChoiceRequired	–	–	Subtipo sem campos (uso obrigatório).
ToolChoiceSpecific	toolNames	List<String>	Lista de ferramentas permitidas.
Provider	value	String	Identificador textual do fornecedor.
TypedMap	data	Map<String, Any?>	Mapa genérico de opções específicas do fornecedor.
RequestContentPart	TextPart	text: String	Conteúdo textual.
RequestContentPart	JsonPart	json: JsonValue	Conteúdo JSON estruturado.
RequestContentPart	ToolCallPart	toolCallId, functionName, argumentsJson	Chamada a ferramenta com argumentos.
RequestContentPart	ToolResultPart	toolCallId, content	Resultado de execução de ferramenta.

Tabela 3.1: Parâmetros do domínio de **CommonRequest**.

Modelo Comum de Resposta e Eventos

O modelo de resposta comum, sob a designação **CommonResponse**, é a base que normaliza as respostas devolvidas pelos diferentes fornecedores. A sua estrutura foi desenhada para representar o resultado final da geração, mantendo as várias opções de resposta – as escolhas representadas pela classe **CommonChoice** – juntamente com o respetivo índice e a razão de término indicada por **FinishReason**. Cada escolha contém uma mensagem de resposta, definida em **CommonResponseMessage**, que estipula o papel da entidade na propriedade **CommonRole** e o seu conteúdo segmentado em partes através de **ResponseContentPart**, permitindo assim combinar texto, JSON, chamadas a ferramentas ou recusas do modelo. A informação

de utilização isolada em `CommonUsage` quantifica o consumo de *tokens* e facilita a extração de métricas e contabilização. Por fim, a propriedade `providerOptions` preserva metadados específicos do fornecedor.

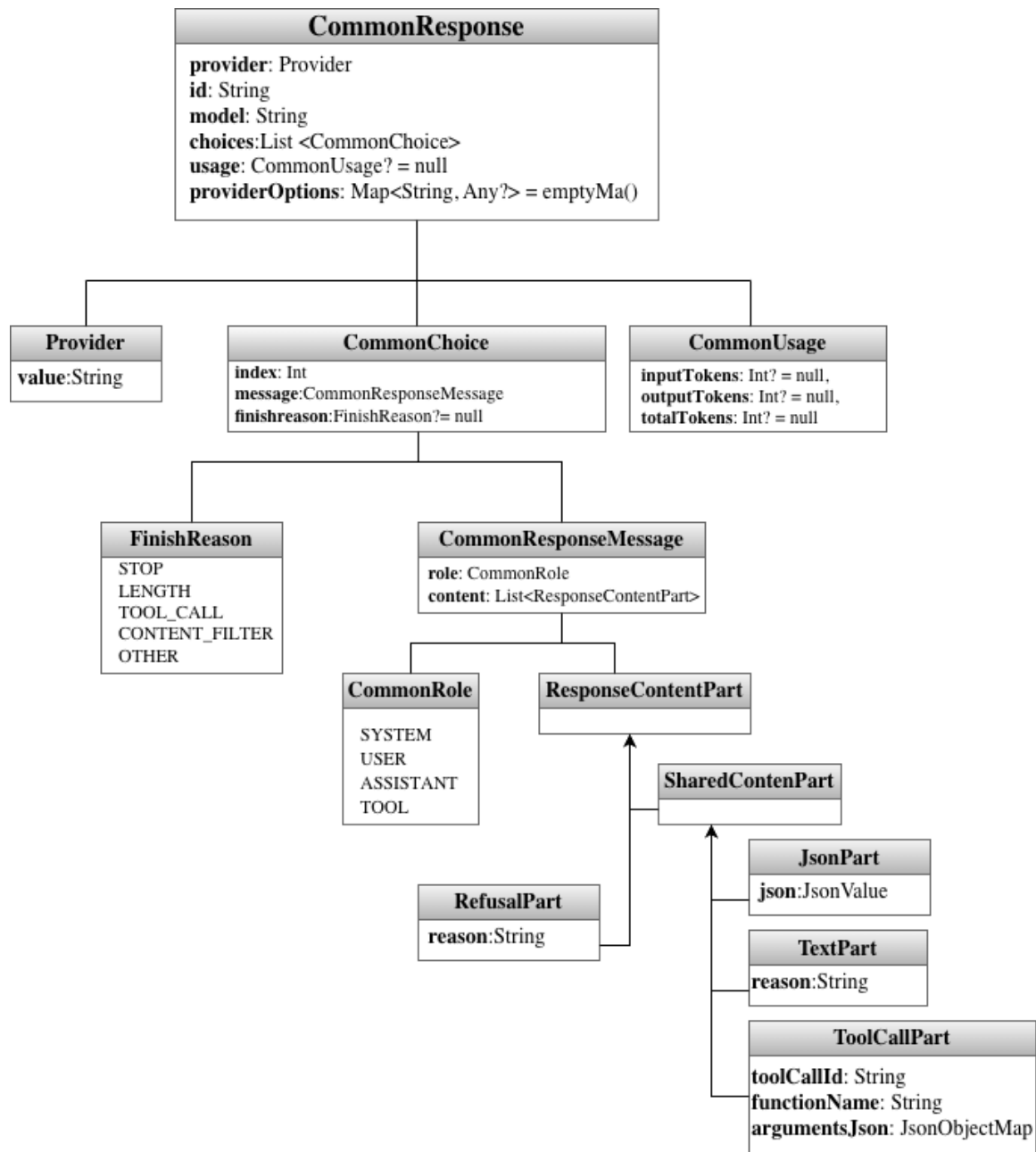


Figura 3.5: Estrutura de classes do `CommonResponse`.

A Tabela 3.2 reúne os campos do domínio de `CommonResponse`, incluindo o objeto base, as escolhas e os eventos de *streaming*.

Elemento	Campo	Tipo	Descrição
CommonResponse	<i>provider</i>	Provider	Fornecedor que produziu a resposta.
CommonResponse	id	String	Identificador do pedido/execução.
CommonResponse	model	String	Identificador do modelo utilizado.
CommonResponse	choices	List<CommonChoice>	Lista de escolhas geradas.
CommonResponse	usage	CommonUsage?	Contabilização de <i>tokens</i> (opcional).
CommonResponse	providerOptions	Map<String, Any?>	Metadados específicos do fornecedor.
CommonChoice	index	Int	Índice da escolha.
CommonChoice	message	CommonResponseMessage	Mensagem com papel e conteúdo.
CommonChoice	finishReason	FinishReason?	Razão de término da escolha (opcional).
CommonResponseMessage	role	CommonRole	Papel da mensagem.
CommonResponseMessage	content	List<ResponseContentPart>	Conteúdo segmentado da resposta.
CommonUsage	inputTokens	Int?	Tokens de entrada.
CommonUsage	outputTokens	Int?	Tokens de saída.
CommonUsage	totalTokens	Int?	Total de <i>tokens</i> .
FinishReason	–	STOP / LENGTH / TOOL_CALL / CONTENT_FILTER / OTHER	Motivo de conclusão da geração.
ResponseContentPart	TextPart	text: String	Segmento textual.
ResponseContentPart	JsonPart	json: JsonValue	Segmento JSON estruturado.
ResponseContentPart	ToolCallPart	toolCallId, functionName, argumentsJson	Chamada a ferramenta.
ResponseContentPart	RefusalPart	reason: String	Recusa do modelo com justificção.

Tabela 3.2: Parâmetros do domínio de *Response* (CommonResponse).

O `CommonResponseEvent` complementa o `CommonResponse` ao representar respostas em *streaming* como uma sequência ordenada de eventos. Esta abordagem permite construir a saída de forma incremental: primeiro sinaliza o início da resposta, depois o arranque de cada escolha, segue com deltas de texto ou de argumentos de ferramentas e, por fim, reporta o fecho da escolha, o uso de *tokens* e a conclusão (ou erro) do *stream*. Cada evento partilha um conjunto mínimo de metadados (fornecedor, identificador, modelo e sequência) que asseguram consistência e reordenação determinística.

Elemento	Campo	Tipo	Descrição
CommonResponseEvent	<i>provider</i>	Provider	Fornecedor associado ao evento.
CommonResponseEvent	id	String	Identificador do pedido/execução.
CommonResponseEvent	model	Model	Modelo utilizado no evento.
CommonResponseEvent	sequence	Long	Ordem do evento no <i>stream</i> .
CommonResponseEvent	<i>provider</i> EventType	String?	Tipo bruto do evento do fornecedor (opcional).
ResponseStarted	–	–	Início do <i>streaming</i> da resposta.
ChoiceStarted	choiceIndex	Int	Índice da escolha iniciada.
ChoiceStarted	role	CommonRole?	Papel associado à escolha (opcional).
TextDeltaEvent	choiceIndex	Int	Índice da escolha que recebe o delta.
TextDeltaEvent	text	String	Fragmento incremental de texto.
ToolCallStartedEvent	choiceIndex	Int	Índice da escolha.
ToolCallStartedEvent	toolCallIndex	Int	Índice da chamada a ferramenta.
ToolCallStartedEvent	toolCallId	String	Identificador da chamada a ferramenta.
ToolCallStartedEvent	functionName	String	Nome da função invocada.
ToolCallArgumentsDeltaEvent	choiceIndex	Int	Índice da escolha.
ToolCallArgumentsDeltaEvent	toolCallIndex	Int	Índice da chamada a ferramenta.
ToolCallArgumentsDeltaEvent	arguments Fragment	String	Fragmento incremental de argumentos.
ChoiceFinished	choiceIndex	Int	Índice da escolha terminada.
ChoiceFinished	finishReason	FinishReason?	Razão de término (opcional).
UsageReported	usage	CommonUsage	Contabilização de <i>tokens</i> reportada.
ResponseCompleted	–	–	Conclusão do <i>streaming</i> da resposta.
ResponseErrored	message	String	Mensagem de erro.
ResponseErrored	retryable	Boolean	Indica se o erro é recuperável.

Tabela 3.3: Parâmetros do domínio de *Response* (CommonResponseEvent).

3.1.3 Mapa Tipificado e Extensibilidade do Domínio

Nem todos os dados relevantes pertencem ao modelo mínimo de inferência. Algumas informações são específicas do fornecedor, como parâmetros avançados do **Gemini** e outras pertencem ao contexto operacional, como metadados de autenticação, IP do cliente, cabeçalhos **HTTP** ou atributos necessários para observabilidade e autorização. Para estes casos, o **OmniAI SDK** introduz a estrutura **TypedMap**.

O **TypedMap** funciona como um mapa de atributos tipado por chave. Na sua base, o conceito assenta na utilização de um **AttributeKey<T>**, que atua como um marcador tipificado. Em vez de se utilizar uma simples *string* como chave, o que obrigaria a conversões de tipo manuais e inseguras no momento da leitura, cada **AttributeKey** encapsula não só o nome lógico do atributo, mas também a sua informação de tipo (**KClass<T>**).

A implementação desta chave demonstra como a segurança é alcançada, garantindo que atributos com o mesmo nome mas tipos diferentes não colidam, e fornecendo uma forma idiomática de instanciar novas chaves:

```

class AttributeKey<T : Any>
    @PublishedApi
    internal constructor(
        val name: String,
        val type: KClass<T>,
    ) {
        override fun equals(other: Any?): Boolean =
            (other is AttributeKey<*>) && name == other.name && type == other.type

        override fun hashCode(): Int = name.hashCode() * 31 + type.hashCode()

        override fun toString(): String = "Key($name: ${type.simpleName})"
    }

inline fun <reified T : Any> key(name: String) = AttributeKey(name, T::class)

```

A função auxiliar `key` utiliza a reificação de tipos (*reified type parameters*) da linguagem Kotlin, extraíndo o tipo de forma transparente para o programador em tempo de compilação.

Com o `AttributeKey` estabelecido, o `TypedMap` encarrega-se do armazenamento e da gestão dos valores. Internamente, recorre a um mapa padrão, mas a sua interface pública impõe o uso das chaves tipificadas nas operações, garantindo *type-safety*:

```

class TypedMap(
    private val data: MutableMap<AttributeKey<*>, Any> = mutableMapOf(),
) {
    fun <T : Any> put(key: AttributeKey<T>, value: T) {
        data[key] = value
    }

    @Suppress("UNCHECKED_CAST")
    operator fun <T : Any> get(key: AttributeKey<T>): T? = data[key] as? T

    fun <T : Any> require(key: AttributeKey<T>): T =
        get(key) ?: error("Missing required key: $key")
}

```

Esta abordagem revela-se significativamente mais segura do que um `Map<String, Any?>` genérico, pois cada operação de leitura assegura de imediato o tipo esperado do valor, tal como foi especificado na chave. Em simultâneo, é mais flexível do que adicionar dezenas de campos opcionais diretamente às classes de domínio sempre que surge uma nova funcionalidade no sistema.

Para além da sua utilidade no domínio de inferência, o facto de a estrutura `TypedMap` ter sido concebida de forma totalmente genérica e agnóstica permitiu o seu reaproveitamento extensivo noutras partes críticas do sistema. Por ser independente de qualquer lógica de negócio específica, tornou-se o mecanismo padrão do **OmniAI SDK** para transporte seguro de metadados, sendo utilizada tanto no *pipeline* de interceção de pedidos como nos resultados de chamadas **HTTP** e gestão de sessões do utilizador.

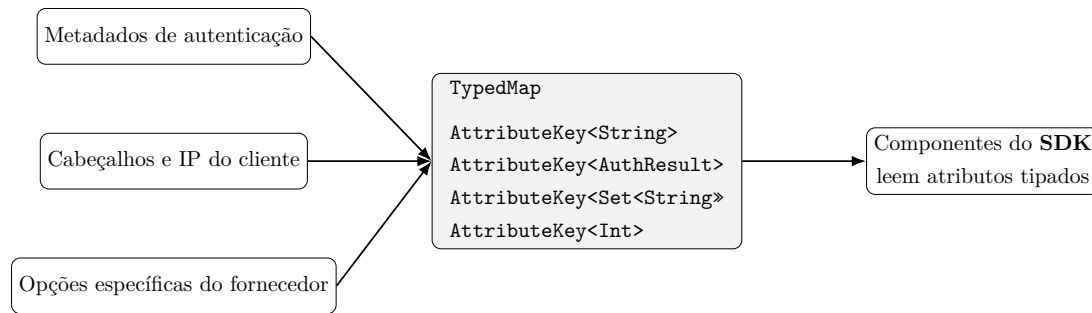


Figura 3.6: Utilização da `TypedMap` para transportar atributos extensíveis

No domínio de inferência, a estrutura `TypedMap` é utilizada na propriedade de domínio comum `providerOptions`, permitindo preservar dados que não pertencem ao subconjunto padronizado, mas que não devem ser perdidos durante o processo de tradução entre formatos. Fora da inferência estrita, a mesma estrutura permite transportar metadados entre componentes sem criar dependências diretas entre eles. Esta característica é fundamental para a evolução futura do **OmniAI SDK**, porque permite que mecanismos como autorização, observabilidade ou políticas de execução, consumam e injetem atributos adicionais sem forçar alterações constantes nas classes centrais do domínio.

3.2 Arquitetura do Sistema

3.2.1 Core

No contexto do **OmniAI SDK**, o domínio comum apresentado na secção anterior, juntamente com as interfaces fundamentais de comunicação, encontra-se centralizado no módulo **core**. Este módulo atua como o centro do sistema, contendo as entidades, os contratos genéricos e as portas (interfaces) que definem o comportamento da aplicação, sem possuir qualquer dependência para com bibliotecas externas de rede ou fornecedores específicos. A adoção do termo *core* ao longo deste documento refere-se especificamente a este módulo responsável por garantir a estabilidade e as bases centrais da arquitetura.

A Arquitetura Hexagonal, também conhecida como padrão de Portas e Adaptadores, foi escolhida neste projeto para garantir que esta lógica principal do sistema permaneça independente de detalhes técnicos e de integrações externas. Em vez de o *core* da aplicação depender diretamente de bibliotecas, protocolos ou APIs específicas, toda a comunicação com o exterior acontece através de interfaces bem definidas, enquanto os adaptadores tratam das implementações concretas [7, 40, 41].

Esta separação torna-se especialmente importante no contexto de uma *Gateway* de Inteligência Artificial, uma vez que o sistema atua como ponto central de comunicação entre clientes **HTTP**, fornecedores de inferência, mecanismos de autenticação e serviços de observabilidade. Sem um isolamento arquitetural claro, a lógica de domínio acabaria inevitavelmente dependente de *SDKs* externos, formatos proprietários e detalhes específicos de integração de cada fornecedor.

Ao adotar esta abordagem, foi possível manter o *core* do **OmniAI SDK** focado apenas nas responsabilidades centrais de normalização, abstração e coordenação. Assim, funcionalidades como chamadas de rede, serialização de dados ou integração com APIs externas ficam confinadas aos adaptadores. Isto permite que novos fornecedores de IA, novos mecanismos de autenticação e outros mecanismos sejam adicionados com alterações mínimas na lógica principal da aplicação.

Durante o desenho da solução, foram analisadas outras abordagens arquiteturais para avaliar qual delas se ajustava melhor aos requisitos do projeto. Uma das alternativas consideradas foi a arquitetura tradicional em camadas [42]. Apesar de ser uma solução mais simples numa fase inicial, este modelo tende

a perder clareza estrutural à medida que o sistema cresce. Com o tempo, torna-se comum encontrar regras de negócio misturadas com lógica de transporte, chamadas **HTTP** e tratamento de contratos externos, aumentando o acoplamento e dificultando a manutenção [43].

Neste contexto, a Arquitetura Hexagonal revelou-se a solução mais adequada para os objetivos do projeto, oferecendo três benefícios fundamentais: modularidade, ao organizar as diferentes políticas transversais exigidas por uma *Gateway* de Inteligência Artificial (como autenticação, resiliência, observabilidade e roteamento de pedidos) em componentes independentes; isolamento, ao proteger o *core* da aplicação de detalhes de implementação de tecnologias e APIs externas; e flexibilidade, facilitando a introdução de novos fornecedores e novas funcionalidades sem comprometer a estabilidade do sistema, o que mantém a estrutura organizada e simples de evoluir [18–20].

3.2.2 Contratos

Os módulos `contracts:openai`, `contracts:anthropic` e `contracts:gemin`i modelam os formatos de pedido, resposta e eventos de cada fornecedor. Estes contratos são DTOs no sentido em que representam dados transportados na fronteira do sistema, mas não são apenas DTOs genéricos. São chamados *contracts* porque codificam uma compatibilidade externa concreta: nomes de campos, estruturas JSON, eventos de *streaming*, campos opcionais, formatos de erro e convenções específicas de cada API.

Esta distinção é importante. Um DTO interno poderia ser livremente alterado sem impacto externo um contrato compatível com **OpenAI**, **Anthropic** ou **Gemini** precisa de respeitar o formato que clientes e fornecedores esperam. Ao isolar estes contratos em módulos próprios, o **OmniAI SDK** evita que detalhes como `tool_calls`, `content_block_delta` ou `functionDeclarations` se espalhem pelo domínio comum.

3.2.3 Adaptadores de Entrada

Os adaptadores de entrada recebem pedidos no formato do cliente e traduzem-nos para `CommonRequest`. Cada *inbound* implementa a porta `InboundPort` e recebe um `DispatcherPort`, não uma implementação concreta de *pipeline* ou de fornecedor. Além disso, cada *inbound* usa uma implementação de `InboundTranslator`, responsável por converter o objeto de entrada para domínio e por converter a resposta comum de volta para o contrato do cliente.

Esta decisão evita que o *inbound* exponha diretamente uma Web API. O **OmniAI SDK** fornece *adapters* que trabalham sobre DTO's, e a exposição **HTTP** fica noutra *adapter*, como o `gateway-ktor-server` (detalhado na Secção 4.5). Assim, um servidor **Ktor**, uma função *serverless*, um *handler* Express ou outro mecanismo de entrada podem chamar o mesmo *inbound* desde que consigam construir o DTO esperado. Esta separação reduz a dependência do domínio e dos *inbounds* face a bibliotecas **HTTP** externas.

3.2.4 Adaptadores de Saída

Os adaptadores de Saída fazem o caminho inverso: recebem um `CommonRequest`, traduzem-no para o contrato do fornecedor real e executam a chamada externa. Cada *outbound* implementa `OutboundPort`, identifica `provider`, `model` e `key`, e delega a tradução num `OutboundTranslator`. A implementação atual inclui *outbounds* para **OpenAI**, **Anthropic** e **Gemini**.

Ao contrário dos *inbounds*, os *outbounds* precisam de comunicar com serviços externos. Por isso, recebem uma implementação de `HttpClient`, a porta de transporte **HTTP** descrita na Secção 3.2.5, que abstrai por completo o mecanismo de rede utilizado. Esta inversão de dependência é o que permite testar *adapters* sem depender de chamadas reais e evoluir o ambiente de execução sem reescrever o domínio.

3.2.5 Camada de Comunicação HTTP

A interface `HttpTransportClient`, definida no módulo `core`, representa a porta de saída responsável por toda a comunicação **HTTP** do nosso **SDK** com os fornecedores externos. Esta abstração isola completamente os *outbound adapters* de qualquer biblioteca ou mecanismo concreto de rede, seguindo o princípio de inversão de dependência que caracteriza a Arquitetura Hexagonal.

A interface expõe três operações distintas. A operação `execute` é utilizada para pedidos síncronos: recebe uma configuração tipificada de pedido (`RequestConfig`) e um serializador, executa a chamada **HTTP** e devolve um `HttpCallResult`. A operação `listen` destina-se a respostas em *streaming* baseadas em *Server-Sent Events* (**SSE**) com um único tipo de evento, devolvendo um `Flow` de resultados. A operação `listenMany` generaliza o modelo de **SSE** para cenários com múltiplos tipos de eventos, selecionando o serializador adequado com base no campo `event` de cada mensagem.

O resultado de qualquer chamada **HTTP** é representado pela classe selada `HttpCallResult`, parametrizada pelo tipo da resposta esperada. Esta classe cobre os seguintes casos: `Success`, que encapsula os dados desserializados; `ApiError`, que retém o código de estado **HTTP** e o corpo da resposta de erro; `NetworkError`, originado por falhas de conectividade; `SerializationError`, resultante de respostas com formato inesperado; e `UnknownError`, para situações não classificadas. Todos os casos carregam um `TypedMap` de metadados, que permite propagar cabeçalhos de resposta e outras informações contextuais para os adaptadores e interceptores de forma estruturada.

A implementação concreta desta interface é descrita na Secção 4.6.

3.2.6 Dispatcher

O `DispatcherPort` é a porta chamada pelos *inbounds* para executar um `CommonRequest`. A sua responsabilidade não é, por si só, conter inteligência de roteamento; é despachar o pedido para uma estratégia de execução. No **OmniAI SDK** é fornecida uma implementação simples, criada por `routingDispatcherFactory`, que procura o *outbound* cujo `provider` e `model` correspondem aos campos do pedido e envia a chamada para esse *adapter*.

Como se trata de uma interface, a arquitetura permite outras implementações com objetivos diferentes. Um consumidor do **SDK** pode fornecer um *dispatcher* direto, um *dispatcher* com seleção por custo, um *dispatcher* que consulta políticas externas ou um *dispatcher* que aplica regras mais complexas. A *pipeline* fornecida pelo **SDK** é ela própria encaixada através de um *adapter*, `PipelineBackedDispatcher`, preservando a ideia de que os *inbounds* dependem apenas de `DispatcherPort`.

3.3 Organização do *SDK*

O **SDK** está organizado como um projeto **Gradle** multi-módulo. Os módulos principais incluem `core`, `contracts`, `inbound`, `outbound`, `interceptors`, `mcp-broker`, `dispatcher`, `gateway-client`, `gateway-ktor-server` e `contracts:ktor-http`. Esta divisão permite evoluir cada área com responsabilidades claras: domínio e portas em `core`, contratos externos em `contracts`, *adapters* de entrada e saída em módulos próprios, preocupações transversais em `interceptors`, montagem declarativa em `gateway-client` e exposição **HTTP/Ktor** em `gateway-ktor-server`.

O módulo `contracts:ktor-http` fornece a abstração de transporte `HttpTransportClient` e a implementação `KtorHttpTransportClient`. Esta camada encapsula chamadas **HTTP**, **SSE**, serialização, metadados de resposta, erros de rede e erros de *parsing*. Ao colocá-la atrás de uma interface, os *outbounds* não ficam presos a uma implementação específica de cliente **HTTP**.

O módulo `gateway-client` disponibiliza uma DSL para montar uma **Gateway**. A aplicação consumidora pode declarar *outbounds*, *interceptors*, autenticação, autorização e modo de execução. A DSL permite

escolher entre `useNativePipeline`, que monta a *pipeline* e o *dispatcher* padrão, e `useCustomDispatcher`, que permite substituir totalmente a estratégia de execução. Esta flexibilidade é essencial para um **SDK**: o POC usa a montagem nativa, mas outro consumidor pode querer um *dispatcher* próprio.

A instância da **OmniAI Gateway** desenvolvida no âmbito deste projeto foi concebida exclusivamente como uma Prova de Conceito (*Proof of Concept* - PoC), com o objetivo de validar o processo de instanciação e configuração disponibilizado pelo **SDK**. Durante o desenvolvimento, a solução foi estruturada recorrendo à funcionalidade de *composite build* do **Gradle**, através da diretiva `includeBuild("../OmniAi-SDK")` [44].

Esta decisão não pretende representar o modelo final de utilização da plataforma em ambiente de produção. A sua adoção teve um propósito essencialmente prático, permitindo desenvolver e compilar simultaneamente o **SDK** e a **Gateway** instanciada sem depender de ciclos constantes de publicação de versões intermédias da biblioteca. Desta forma, qualquer alteração efetuada no núcleo do **SDK** podia ser imediatamente refletida e validada no *Proof of Concept* criado, reduzindo significativamente o tempo necessário para testar novas funcionalidades, ajustes arquiteturais ou alterações ao *pipeline* interno da **Gateway**.

Sem esta abordagem, o processo de desenvolvimento tornar-se-ia substancialmente mais lento, uma vez que cada alteração implicaria publicar uma nova versão da biblioteca, atualizar dependências no projeto consumidor e recompilar novamente toda a aplicação. A utilização de *composite builds* permitiu assim criar um ciclo de desenvolvimento mais rápido e iterativo, particularmente importante numa fase de exploração arquitetural e validação técnica do **SDK**.

Num cenário de adoção real, a expectativa arquitetural é diferente. O **SDK** deverá ser consumido como uma dependência externa devidamente versionada, distribuída através de plataformas de gestão de dependências, como **Maven** no ecossistema **JVM** ou **NPM** em ambientes **Node.js** e **TypeScript**. Desta forma, diferentes equipas poderão instanciar e integrar a sua própria **Gateway** de forma independente, mantendo um processo de versionamento e distribuição consistente com as práticas tradicionais de engenharia de software.

Capítulo 4

Implementação e Evolução Técnica

4.1 Tecnologias e Metodologias Utilizadas

O **OmniAI SDK** foi desenvolvido em Kotlin Multiplatform [6] com *targets* JVM e JavaScript/Node.js. A escolha de KMP permite partilhar o domínio, os contratos, as portas, os *adapters* principais e parte da lógica geral entre plataformas, mantendo implementações específicas quando o ambiente o exige [23]. A serialização dos contratos é baseada em Kotlinx Serialization [45], o que facilita a definição explícita dos modelos JSON que foram apresentados anteriormente.

O *core* do **OmniAI SDK** é estritamente agnóstico em relação a *frameworks* web e mecanismos de transporte HTTP. Contudo, para fornecer uma solução imediatamente utilizável, o projeto inclui módulos de integração baseados na *framework* Ktor [39]. Estes módulos opcionais oferecem implementações de referência tanto para a execução de pedidos aos fornecedores (*client-side*) como para a exposição das rotas da **Gateway** (*server-side*). A **OmniAI Gateway** instanciada como Prova de Conceito (PoC) tira partido destas implementações prontas a usar. No entanto, a adoção do **OmniAI SDK** não obriga ao uso do Ktor, a arquitetura garante que qualquer utilizador possa descartar estes módulos e implementar os adaptadores de rede utilizando a tecnologia da sua preferência, como o Spring [46] por exemplo, entre outros.

As ferramentas de desenvolvimento incluíram Git, IntelliJ IDEA, o cliente **HTTP** do IDE para pedidos **REST**, **Docker Compose** para **Logto** [47], **PostgreSQL** [48], **OpenTelemetry Collector** [49], **Prometheus** [50] e **Grafana** [51], além de clientes reais como **Claude Code** e SDKs oficiais dos fornecedores. Também foram usados assistentes de apoio, como Gemini no *browser* e agentes de programação como o **Antigravity** e **Codex**, para pesquisa, comparação de documentação e aceleração de tarefas, mantendo a validação no código, nos testes e nos cenários executados. Foi criada uma *pipeline* de **Continuous Integration** (CI) [52] que executa `./gradlew clean build` e `./gradlew check`, que realizam build ao projeto e executam testes unitários, como passo evolutivo, está prevista a criação de uma *pipeline* de **Continuous Deployment** (CD) para publicar as versões do nosso *Software Development Kit*, o **OmniAI SDK** em Maven e NPM.

4.2 A Primeira Iteração da *Gateway*

A primeira iteração do projeto consistiu numa **Gateway HTTP** construída diretamente, responsável por receber pedidos de clientes e encaminhá-los para fornecedores de inferência. Esta fase foi útil para validar rapidamente a viabilidade da solução, testar pedidos reais e compreender diferenças práticas entre as diferentes API's dos fornecedores, neste caso da **OpenAI** [1], **Anthropic** [2] e **Gemini** [3].

À medida que o sistema cresceu, essa abordagem revelou acoplamento excessivo. A tradução entre contratos, a configuração dos fornecedores, a autenticação, o tratamento de erros e o encaminhamento de pedidos começaram a concentrar-se na própria aplicação. Cada novo fornecedor ou preocupação exigia alterações no mesmo conjunto de componentes, tornando o código difícil de evoluir e de se testar.

Para resolver os problemas de acoplamento identificados, a arquitetura foi dividida em duas componentes principais: uma biblioteca central (**OmniAI SDK**) e uma aplicação final consumidora (**OmniAI Gateway**).

O **OmniAI SDK** passou a concentrar toda a lógica e abstrações reutilizáveis do domínio, incluindo contratos, portas e adaptadores, o *dispatcher*, a *pipeline* de execução, os *interceptors*, as *DSLs* de configuração e abstrações HTTP multiplataforma. Ao concentrar estas responsabilidades, construiu-se uma base que permite instanciar e parametrizar *gateways* à medida de diferentes cenários.

Por sua vez, a aplicação **OmniAI Gateway** representa apenas uma das múltiplas instanciações possíveis do nosso SDK. A sua principal função é carregar as configurações do sistema, inicializar os componentes, associar os conectores do **Ktor** e expor os *endpoints* da rede. A aplicação serve então como prova de conceito para comprovar que a nossa arquitetura suporta soluções funcionais e totalmente independentes dos fornecedores de IA.

Esta separação teve um impacto direto na preparação do sistema para escalamento horizontal. O **OmniAI SDK** foi concebida para não guardar dados persistentes locais, mantendo a informação dos pedidos exclusivamente em estruturas transitórias (como o **GatewayContext** e o **TypedMap**). Qualquer mecanismo que exija estado partilhado como sistemas de *cache*, controlo de acessos (*rate limiting*), *circuit breakers* ou métricas é delegado através de portas de integração definidas.

Com esta arquitetura, o arranque inicial do projeto pode ser feito de forma muito simples, com os mecanismos auxiliares a correrem diretamente em memória. Contudo, perante a necessidade de distribuir o processamento por vários servidores, a arquitetura permite substituir estas implementações locais por serviços externos partilhados (como o *Redis* [53] para a *cache*), sem alterar uma única linha dos contratos de negócio originais.

Assim esta decisão permitiu que a aplicação evolua de um formato monolítico local para um modelo distribuído e escalável.

4.3 Decisões de Desenho e Alternativas Consideradas

A análise de *trade-offs* partiu de três critérios principais: preservar a estabilidade da versão demonstrável, manter o **OmniAI SDK** reutilizável e extensível independentemente da aplicação final e evitar que funcionalidades experimentais introduzissem acoplamento excessivo no domínio comum.

As decisões seguintes não representam funcionalidades abandonadas, mas sim escolhas conscientes de âmbito para garantir que a primeira versão permanecesse estável, testável e arquiteturalmente coerente:

- **Geração de *bridges* com KSP (Kotlin Symbol Processing):** Foi considerada a utilização do KSP para gerar automaticamente o código de adaptação entre a lógica desenvolvida em **Kotlin** e a interface disponibilizada em **TypeScript/Node.js**. Apesar de esta abordagem reduzir a implementação manual, introduziria uma maior complexidade ao nível da configuração do processo de compilação, da compatibilidade entre plataformas e do tratamento das diferenças entre os sistemas de tipos das duas linguagens.

Como alternativa, optou-se pela implementação manual destas *bridges*. Esta solução permite controlar explicitamente a conversão entre os tipos definidos na lógica comum em **Kotlin** e as estruturas equivalentes no ecossistema **JavaScript**, como *Array* e *Map*. Posteriormente, estas interfaces foram expostas para utilização em **JavaScript** através da anotação **@JsExport**, disponibilizada pelo

Kotlin Multiplatform.

- **Injeção Automática de Ferramentas via SSE:** Também foi avaliada a possibilidade de a **Gateway** intercetar automaticamente *tool calls* durante *streams*, executar ferramentas e reinjetar os resultados no fluxo **SSE** de forma transparente para o cliente. Apesar de tecnicamente viável, esta funcionalidade exigiria mecanismos adicionais de gestão de estado conversacional, suspensão e retoma de streams, coordenação concorrente e normalização de diferenças entre fornecedores que emitem eventos distintos. A capacidade ficou identificada como evolução futura, mantendo a primeira versão focada nos elementos considerados essenciais: contratos comuns, *adapters*, *pipeline* de *interceptors*, segurança, observabilidade e resiliência.

4.4 Evolução Arquitetural: De *Inference Service* a *Dispatcher*

A primeira iteração da arquitetura concentrava comunicação, tradução de contratos e roteamento num componente designado por **InferenceService**. Embora esta abordagem tenha sido eficaz durante a fase inicial de desenvolvimento e prototipagem, rapidamente começou a aproximar-se de um *God Object* [54]: cada novo fornecedor, métrica, regra de segurança ou política de erro aumentava progressivamente a responsabilidade do mesmo serviço.

O problema principal não era apenas o crescimento do componente, mas sobretudo a direção das dependências. Ao conhecer diretamente contratos **HTTP**, clientes externos, regras de seleção de fornecedor e políticas transversais, o serviço tornava-se o ponto central de acoplamento da arquitetura. Qualquer alteração numa dessas áreas propagaria impacto para o core do sistema, dificultando testes unitários, aumentando o custo de introdução de novos *adapters* e reduzindo significativamente o valor do **SDK** enquanto fundação reutilizável para diferentes *gateways* e cenários de execução.

A refatorização substituiu esse modelo por um contrato pequeno e explícito: o **DispatcherPort**. A responsabilidade do *dispatcher* passou a ser exclusivamente receber um **CommonRequest** e delegar a execução para uma estratégia de despacho. A implementação padrão fornecida pelo **SDK**, criada através de **routingDispatcherFactory**, realiza um roteamento simples baseado no fornecedor e no modelo presentes no pedido. Contudo, por se tratar apenas de uma interface, o **SDK** não impõe esta estratégia: qualquer consumidor pode substituir o *dispatcher* por outra implementação sem necessidade de alterar *inbounds*, contratos ou *adapters* existentes.

Esta alteração teve impacto direto na modularidade da arquitetura. Os *inbounds* passaram a depender apenas da porta **DispatcherPort**; os *outbounds* ficaram responsáveis exclusivamente pela tradução de contratos e comunicação externa; e as preocupações transversais foram deslocadas para a *pipeline* de *interceptors*. Como consequência, a **OmniAiGateway** deixou de representar uma implementação rigidamente acoplada à infraestrutura interna e passou a funcionar como uma composição configurável construída sobre o **OmniAI SDK**.

Na prática, esta separação permitiu preservar o **SDK** como uma fundação independente do *Proof of Concept*, reutilizável na construção de outras *gateways*, runtimes ou integrações futuras.

4.5 Implementação da Camada HTTP

A camada de exposição **HTTP** de entrada da **Gateway** (responsável por receber os pedidos dos clientes) é implementada no módulo **gateway-ktor-server**. Este módulo estabelece a ponte entre o servidor web (**Ktor**) e os *inbound adapters* do domínio, garantindo que o núcleo do sistema permanece agnóstico em relação à tecnologia de transporte de entrada.

A implementação baseia-se num padrão de conectores (**InboundConnector**) que regista rotas específicas para cada contrato suportado. O módulo expõe três funções de extensão sobre a interface de encaminhamento (**Route**) do **Ktor**:

Conector Ktor	Caminho (Path) Predefinido	DTOs de Contrato (Pedido / Resposta)
<code>openAiConnector()</code>	<code>/v1/chat/completions</code>	<code>OpenAiChatCompletionsRequest</code> / <code>OpenAiChatCompletionsResponse</code>
<code>anthropicConnector()</code>	<code>/v1/messages</code>	<code>AnthropicMessagesRequest</code> / <code>AnthropicMessageResponse</code>
<code>geminiConnector()</code>	<code>/v1beta/models/{model}:generateContent</code>	<code>GeminiGenerateContentRequest</code> / <code>GeminiGenerateContentResponse</code>

Tabela 4.1: Conectores **HTTP** disponibilizados pela **Gateway Ktor**

A ligação entre a rota **HTTP** e a porta de entrada opera num ciclo de duas fases. Numa primeira fase, as rotas são registadas e aguardam a injeção da dependência. Quando o servidor é iniciado, o *adapter* concreto (por exemplo, `OpenAiInboundAdapter`) é injetado no conector através da função `connect()`.

O fluxo de processamento de um pedido **HTTP** obedece sempre à mesma sequência:

1. **Desserialização:** O corpo do pedido **HTTP** é convertido para o DTO do contrato correspondente (e.g., `OpenAiChatCompletionsRequest`), devolvendo um erro 400 **Bad Request** em caso de falha sintática.
2. **Extração de Metadados:** Atributos essenciais do protocolo **HTTP** são extraídos e propagados para a `TypedMap` do contexto interno. A configuração `defaultKtorRequestMetadataBinder` processa o cabeçalho **Authorization** (removendo o prefixo **Bearer**), resolve o endereço IP do cliente (verificando **X-Forwarded-For** e **X-Real-IP**) e mapeia parâmetros de rota, como o modelo dinâmico na API do **Gemini**.
3. **Execução e *Streaming*:** O conector avalia se o cliente solicitou *streaming* (através de *flags* no corpo do pedido ou parâmetros *query*). Consoante o caso, delega a execução para o método `generate()` (síncrono) ou `generateStream()` da respetiva porta.
4. **Resposta:** Os resultados unários são diretamente serializados para JSON. Em contraste, as respostas em *streaming* originam um **Flow** reativo, onde cada evento emitido é individualmente serializado em JSON e transmitido de forma progressiva ao cliente através de *Server-Sent Events* (**SSE**).

4.6 Implementação do **HttpTransportClient** em **Ktor**

A abstração de chamadas **HTTP** de saída é garantida pela interface **HttpTransportClient**, descrita na Secção 3.2.5. A implementação concreta fornecida pelo *SDK*, `KtorHttpTransportClient`, assenta no cliente **HTTP Ktor** e reside no módulo `contracts:ktor-http`.

Esta implementação suporta retentativas configuráveis com atraso temporal entre tentativas (*delay*), tratamento explícito de exceções de rede (traduzidas para **NetworkError**) e falhas de *parsing* de JSON (traduzidas para **SerializationError**). Suporta ainda a integração transparente com o *plugin* nativo de **SSE** do **Ktor** para o consumo de respostas progressivas.

A configuração do cliente subjacente é centralizada em **PlatformHttpClient**, que padroniza tempos limite (*timeouts*) de ligação e *socket*, bem como o processo de *Content Negotiation*. Um aspeto fulcral

desta organização é o facto de a configuração base ser resolvida por implementações específicas de plataforma (**JVM** ou **JavaScript**). Isto permite ao *SDK* funcionar corretamente como biblioteca **Kotlin Multiplatform**.

Esta arquitetura permite adaptar o comportamento da camada **HTTP** ao contexto de execução sem alterar os *adapters* dos fornecedores. Durante os testes, por exemplo, os *outbounds* podem comunicar com implementações fictícias em memória, evitando dependências de rede externa. Já em ambientes produtivos, a implementação base pode ser substituída por variantes com características específicas definidas pelo consumidor do **OmniAI SDK**, mantendo inalterada a lógica de abstração e tradução dos contratos dos diferentes fornecedores de Inteligência Artificial.

4.7 Implementações Inbound e Outbound

O lógico central da **OmniAI Gateway** materializa-se nos *adapters* de entrada (*inbounds*) e saída (*outbounds*). Cada *adapter* é rigorosamente desenhado em torno do padrão Porta-Adaptador e apoiado num *Translator* dedicado. Esta separação permite que a realização do pedido e o mapeamento físico de atributos JSON vivam em componentes distintos, facilitando testes isolados. A Figura 4.1 ilustra a arquitetura base de tradução.

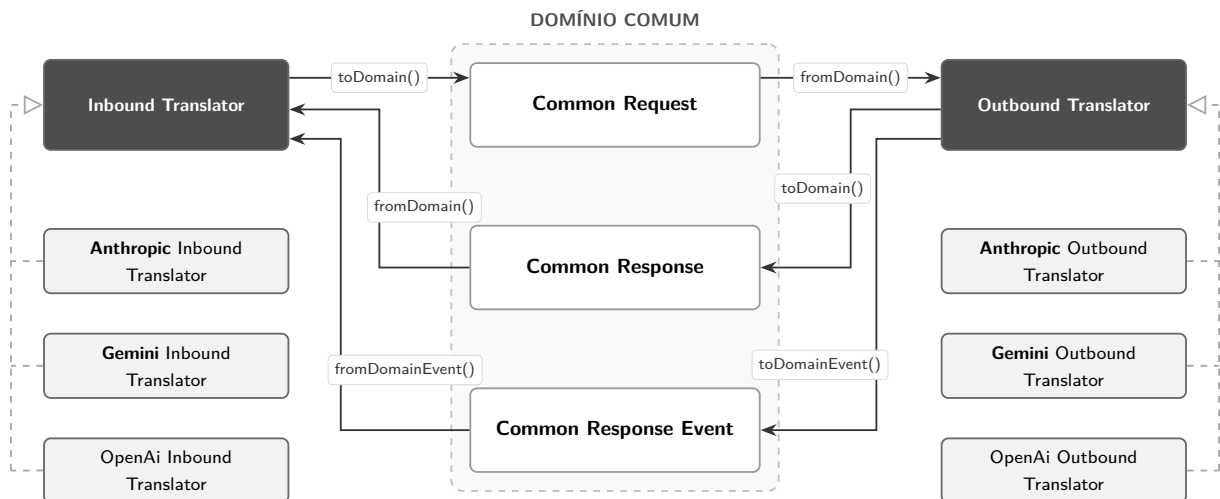


Figura 4.1: Arquitetura de Tradução e Fluxo de Dados entre Contratos e Domínio Comum

4.8 Tradutores de Entrada (Inbound)

Os *Inbound Translators* são responsáveis por converter os contratos específicos de cada fornecedor de inferência para o modelo de domínio comum do **OmniAI SDK** e por realizar o processo inverso na geração de respostas. Além da tradução de pedidos e respostas completas, estes componentes também adaptam eventos incrementais de *streaming* para o formato esperado por cada API.

4.8.1 OpenAI Inbound Translator

A implementação `OpenAiInboundTranslator` adapta os contratos compatíveis com a API da **OpenAI** para o domínio comum.

Contrato Inbound (OpenAI)	Mapeamento para CommonRequest
<code>temperature, maxTokens, topP</code>	Integrados na <code>CommonGenerationConfig</code> . O campo <code>model</code> é mapeado diretamente para o nível superior do <code>CommonRequest</code> .
<code>stop</code>	É convertido para uma <code>List<String></code> , distinguindo automaticamente entre valores únicos (<code>Single</code>) e múltiplos (<code>Multiple</code>).
<code>messages[].role</code>	Convertido para os papéis do domínio comum: <code>SYSTEM</code> , <code>ASSISTANT</code> , <code>TOOL</code> e <code>USER</code> . Valores não reconhecidos são tratados como <code>USER</code> .
<code>messages[].content & toolCalls</code>	O conteúdo textual não vazio origina instâncias de <code>TextPart</code> . As chamadas de ferramentas são convertidas em <code>ToolCallPart(toolCallId, functionName)</code> , com os argumentos encapsulados num mapa sob a chave <code>"raw"</code> ; se o identificador vier omissa, é gerado no formato <code>"openai-tool-call-\$index"</code> . Quando o papel é <code>TOOL</code> e existem <code>toolCallId</code> e conteúdo, é gerado um <code>ToolResultPart</code> , convertendo o conteúdo textual para valor JSON.
<code>tools[].function</code>	Convertido para <code>CommonTool(name, description, parametersSchema)</code> , extraindo as propriedades definidas no schema da ferramenta. Se a descrição estiver ausente, é usado o nome da função; se os parâmetros estiverem ausentes, o schema fica vazio.
<code>toolChoice</code>	Converte os valores <code>auto</code> , <code>none</code> e <code>required</code> para os respetivos tipos do domínio comum (<code>ToolChoice.Auto</code> , <code>ToolChoice.None</code> e <code>ToolChoice.Required</code>), podendo também representar a seleção explícita de uma função específica.
<code>responseFormat</code>	Verifica se o tipo corresponde a <code>"json_object"</code> ou <code>"json_schema"</code> , ativando a opção <code>jsonResponse</code> .
<code>stream, frequencyPenalty, presencePenalty, n, seed, user, logitBias, logProbs, topLogProbs, responseFormat</code>	Armazenados na <code>TypedMap(providerOptions)</code> do <code>CommonRequest</code> , preservando opções específicas do fornecedor que não possuem representação no domínio comum.

Tabela 4.2: Mapeamento Inbound: **OpenAI** Request para o Domínio Comum

No sentido inverso, a tradução da resposta converte os objetos do domínio comum para a estrutura nativa da API da **OpenAI**.

Domínio Comum	Mapeamento para Contrato Resposta (OpenAI)
<code>id & created</code>	Utiliza o <code>id</code> nativo ou gera um identificador no formato <code>chatcmpl-UUID</code> . O instante temporal é recuperado das <code>providerOptions</code> ou gerado pelo <code>Clock.System</code> ; o <code>systemFingerprint</code> , quando existe, também é recuperado das <code>providerOptions</code> .
<code>choices[].message.role</code>	Convertido de <code>CommonRole</code> para os papéis nativos da OpenAI : <code>system</code> , <code>user</code> , <code>assistant</code> ou <code>tool</code> .
<code>choices[].message.content</code>	O conteúdo é obtido através de <code>firstRenderableContentOrNull()</code> , juntando os <code>TextPart</code> com quebras de linha. Se não existir texto, recorre ao primeiro <code>RefusalPart</code> ou à representação textual do primeiro <code>JsonPart</code> .
<code>choices[].message.toolCalls</code>	Um <code>ToolCallPart</code> é convertido para um <code>OpenAiToolCallOutput</code> , formatando os <code>argumentsJson</code> numa única representação textual.
<code>choices[].finishReason</code>	Converte <code>STOP</code> para <code>stop</code> , <code>LENGTH</code> para <code>length</code> , <code>TOOL_CALL</code> para <code>tool_calls</code> e <code>CONTENT_FILTER</code> para <code>content_filter</code> . Valores <code>OTHER</code> ou nulos não são emitidos.
<code>usage</code>	Mapeia <code>inputTokens</code> , <code>outputTokens</code> e <code>totalTokens</code> para <code>promptTokens</code> , <code>completionTokens</code> e <code>totalTokens</code> , respetivamente.

Tabela 4.3: Mapeamento Inbound: Domínio para **OpenAI** Response Unária

Para respostas em *streaming*, os eventos internos do domínio são convertidos para os *chunks* **SSE** utilizados pela **OpenAI**.

Common Response Event	Mapeamento para Stream Chunk (OpenAI)
ResponseStarted	Emite Chunk inicial. choices ficam vazias.
ChoiceStarted	Emite Chunk com índice da <i>choice</i> . O <i>delta</i> contém apenas a definição do <i>role</i> ; se o papel vier ausente, assume <i>assistant</i> .
TextDeltaEvent	Emite Chunk com <i>delta.content</i> preenchido pelo texto.
ToolCallStartedEvent	Emite Chunk inicializando o <i>delta</i> de <i>toolCalls</i> : define o <i>id</i> , a <i>type="function"</i> e o nome da função (<i>arguments=</i>).
ToolCallArgumentsDeltaEvent	Emite Chunk contendo fragmentos incrementais em <i>function.arguments</i> . O nome da função é omitido neste pacote.
ChoiceFinished	Emite Chunk apenas com a tradução do <i>finishReason</i> ; o <i>delta</i> não é preenchido.
UsageReported	Emite Chunk especial sem <i>choices</i> , transportando o objeto global <i>usage</i> .
ResponseCompleted	Emite Chunk sem <i>choices</i> , sinalizando a conclusão bem-sucedida do <i>stream</i> .
ResponseErrored	Emite Chunk injetando a mensagem de erro como texto no <i>delta.content</i> .

Tabela 4.4: Mapeamento Inbound: Domínio para **OpenAI** Streaming (fromDomainEvent)

4.8.2 Anthropic Inbound Translator

O `AnthropicInboundTranslator` adapta os contratos compatíveis com a *Messages API* da **Anthropic** para o domínio comum.

Contrato Inbound (Anthropic)	Mapeamento para CommonRequest
<code>temperature, maxTokens, topP, stopSequences</code>	Agrupados na <code>CommonGenerationConfig</code> . O campo <code>model</code> , por sua vez, fica mapeado logo na raiz do <code>CommonRequest</code> .
<code>system</code>	O conteúdo de sistema é convertido para <code>SystemPrompt</code> . Quando chega como <code>RawText</code> , o texto é usado diretamente; quando chega como lista de blocos, apenas os blocos <code>Text</code> são unidos com <code>joinToString()</code> . Se o resultado estiver em branco, é ignorado.
<code>messages[].role</code>	Convertido para os enums de domínio: <code>SYSTEM</code> , <code>ASSISTANT</code> , <code>TOOL</code> ou <code>USER</code> . Valores não reconhecidos são tratados como <code>USER</code> .
<code>messages[].content</code>	O <code>Text</code> passa a <code>TextPart</code> . O <code>ToolUse</code> dá origem a um <code>ToolCallPart(id, name, input)</code> , gerando um ID automático (" <code>anthropic-tool-use-\$index</code> ") se for necessário. O <code>ToolResult</code> é encapsulado num <code>ToolResultPart</code> . Nota: Blocos de <code>Thinking</code> (raciocínio interno) que venham no input são ignorados e retornam <code>null</code> .
<code>tools</code>	A lista é mapeada para instâncias de <code>CommonTool</code> . O <code>inputSchema</code> original é convertido para <code>JsonObject</code> e guardado no campo <code>parametersSchema</code> ; se não for um objeto JSON, o schema fica vazio.
<code>toolChoice</code>	Traduzido diretamente: <code>auto</code> para <code>Auto</code> , <code>none</code> para <code>None</code> , <code>any</code> para <code>Required</code> , e os casos de <code>tool</code> passam a <code>Specific(name)</code> .
<i>(Constantes da Integração)</i>	O <code>provider</code> fica definido estaticamente como <code>Provider.ANTHROPIC</code> e o <code>jsonResponse</code> assume sempre o valor <code>false</code> .
<code>stream, topK, stopToken, thinking, metadata</code>	Por serem opções específicas e mais avançadas, são guardadas no <code>TypedMap(providerOptions)</code> dentro do <code>CommonRequest</code> .

Tabela 4.5: Mapeamento Inbound: Request da **Anthropic** para o Domínio

No sentido inverso, a resposta do domínio comum é convertida para o formato `AnthropicMessageResponse`.

Domínio Comum	Mapeamento para Contrato Resposta (Anthropic)
<code>id & model</code>	Utiliza diretamente os valores de <code>id</code> e <code>model</code> da <code>CommonResponse</code> . A resposta nativa é construída a partir da primeira <code>choice</code> ; se não existir, o conteúdo fica vazio e o papel assume <code>assistant</code> .
<code>choices[].message.role</code>	Convertido de <code>CommonRole</code> para o papel nativo. <code>USER</code> passa a <code>user</code> ; <code>SYSTEM</code> , <code>ASSISTANT</code> e <code>TOOL</code> passam a <code>assistant</code> .
<code>choices[].message.content</code>	O conteúdo é convertido preservando os elementos nativos: <code>TextPart</code> gera <code>Text</code> , <code>ToolCallPart</code> origina <code>ToolUse</code> preservando a hierarquia do JSON interno, <code>JsonPart</code> é serializado como texto JSON, e os <code>RefusalPart</code> são omitidos do mapeamento.
<code>choices[].finishReason</code>	Convertido para <code>end_turn</code> , <code>max_tokens</code> , <code>tool_use</code> e <code>stop_sequence</code> . Valores <code>OTHER</code> ou nulos não são emitidos.
<code>usage</code>	Mapeado para o correspondente <code>AnthropicUsage</code> (<code>inputTokens</code> e <code>outputTokens</code>).

Tabela 4.6: Mapeamento Inbound: Domínio para **Anthropic** Message Response Unária

Em modo *streaming*, os eventos do domínio são traduzidos para os eventos **SSE** definidos pela **Anthropic**.

Common Response Event	Mapeamento para Evento SSE (Anthropic)
<code>ResponseStarted</code>	Retorna um evento <code>MessageStart</code> com o <code>id</code> e o <code>model</code> do evento de domínio, assumindo o papel de <code>role="assistant"</code> .
<code>ChoiceStarted</code>	Emite um <code>ContentBlockStart</code> com o índice da <code>choice</code> correspondente, indicando o início de um bloco de <code>Text</code> .
<code>TextDeltaEvent</code>	O fragmento de texto (<code>text</code>) é repassado de forma direta para o evento SSE <code>ContentBlockDelta(TextDelta)</code> .
<code>ToolCallStartedEvent</code>	Inicia o bloco com um <code>ContentBlockStart(ToolUse)</code> , mantendo o <code>id</code> e o <code>name</code> originais, e inicializando a chamada com um <code>JsonObject</code> vazio.
<code>ToolCallArgumentsDeltaEvent</code>	Encaminha os fragmentos parciais dos argumentos recebidos para um evento SSE do tipo <code>ContentBlockDelta(InputJsonDelta)</code> .
<code>ChoiceFinished</code>	Gera um evento <code>MessageDelta</code> que inclui o motivo de paragem (<code>stopReason</code>) associado a esta <code>choice</code> .
<code>UsageReported</code>	Gera um evento <code>MessageDelta</code> contendo a contagem atualizada de <code>tokens</code> processados.
<code>ResponseCompleted</code>	Emite o evento <code>MessageStop</code> , sinalizando o fecho definitivo da <i>stream</i> .
<code>ResponseErrored</code>	Interrompe a <i>stream</i> com um evento de <code>Error</code> . Faz-se a distinção entre falhas temporárias (<code>overloaded_error</code>) e erros gerais da API (<code>api_error</code>).

Tabela 4.7: Mapeamento Inbound: Eventos **SSE Anthropic** (`fromDomainEvent`)

4.8.3 Gemini Inbound Translator

O `GeminiInboundTranslator` adapta os contratos compatíveis com a API **Gemini** para o domínio comum.

Contrato Inbound (Gemini)	Mapeamento para CommonRequest
<code>generationConfig</code>	As propriedades base (<code>temperature</code> , <code>topP</code> , <code>stopSequences</code>) são extraídas diretamente para a <code>CommonGenerationConfig</code> .
<code>model</code>	O identificador do modelo é lido do contrato recebido. Caso não seja fornecido, assume-se a constante "NO MODEL" como <i>fallback</i> .
<code>systemInstruction</code>	Convertido para um <code>SystemPrompt</code> , agrupando as <code>parts</code> de texto não vazias num único bloco contínuo, separadas por quebras de linha. Se o resultado ficar vazio, o <code>SystemPrompt</code> não é criado.
<code>contents[].role</code>	Adapta os papéis nativos para o domínio: <code>system</code> passa a <code>SYSTEM</code> , <code>model</code> e <code>assistant</code> passam a <code>ASSISTANT</code> , <code>tool</code> e <code>function</code> passam a <code>TOOL</code> , enquanto <code>user</code> se mantém como <code>USER</code> . Valores não reconhecidos são tratados como <code>USER</code> .
<code>contents[].parts</code>	O conteúdo é transformado de acordo com o seu tipo: <code>text</code> gera um <code>TextPart</code> , <code>functionCall</code> origina um <code>ToolCallPart</code> com identificador local "gemini-tool-call-\$index", e <code>functionResponse</code> converte-se em <code>ToolResultPart</code> .
<code>tools[].functionDeclarations</code>	A hierarquia aninhada (<code>tools</code> -> <code>functionDeclarations</code>) é simplificada (<i>flattened</i>) para criar uma lista única de ferramentas (<code>List<CommonTool></code>).
<code>toolConfig</code> <code>.functionCallingConfig</code>	Se existirem <code>allowedFunctionNames</code> , o tradutor cria uma escolha específica. Caso contrário, adapta o modo de execução: <code>auto</code> passa a <code>Auto</code> , <code>none</code> a <code>None</code> , e <code>any</code> ou <code>required</code> a <code>Required</code> .
<code>generationConfig</code> <code>.responseMimeType</code>	A <i>flag</i> <code>jsonResponse</code> assume o valor <code>true</code> caso o tipo MIME seja <code>application/json</code> ou se existir um <code>responseJsonSchema</code> explicitamente definido.
<code>topK</code> , <code>thinkingConfig</code> , <code>responseMimeType</code> , <code>responseJsonSchema</code>	Por se tratarem de configurações específicas do Gemini , são guardadas no <code>TypedMap</code> (<code>providerOptions</code>) do <code>CommonRequest</code> .

Tabela 4.8: Mapeamento Inbound: Request da **Gemini** para o Domínio

No sentido inverso, a resposta do domínio comum é convertida para o formato `GeminiGenerateContentResponse`.

Domínio Comum	Mapeamento para Contrato Resposta (Gemini)
<code>id & model</code>	São atribuídos de forma direta aos campos <code>responseId</code> e <code>modelVersion</code> do Gemini .
<code>choices</code>	Dão origem aos respetivos objetos <code>GeminiCandidate</code> , preservando o índice da resposta. A conversão de papéis é feita no sentido inverso: <code>SYSTEM</code> e <code>ASSISTANT</code> passam a "model", <code>TOOL</code> a "function", e <code>USER</code> mantém-se como "user".
<code>choices[].message</code> <code>.content</code>	De forma simétrica, o conteúdo é traduzido consoante a sua natureza: um <code>TextPart</code> gera o texto base de um <code>GeminiResponsePart</code> ; um <code>ToolCallPart</code> preenche o nó correspondente à chamada de função; um <code>JsonPart</code> é convertido numa <code>String</code> com o conteúdo JSON; e um <code>RefusalPart</code> é emitido como texto com o motivo da recusa.
<code>choices[].finishReason</code>	Mapeado para os motivos de paragem nativos do Gemini : <code>STOP</code> , <code>MAX_TOKENS</code> ou <code>SAFETY</code> . No caso de <code>TOOL_CALL</code> , a implementação emite <code>STOP</code> ; valores <code>OTHER</code> ou nulos não são emitidos.
<code>usage</code>	Os contadores genéricos do domínio são convertidos e agrupados na estrutura <code>GeminiUsageMetadata</code> , alimentando os campos <code>promptTokenCount</code> , <code>candidatesTokenCount</code> e <code>totalTokenCount</code> .

Tabela 4.9: Mapeamento Inbound: Domínio para **Gemini** Generate Content Response Unária

Durante o fluxo de *streaming* via Server-Sent Events (SSE), a API **Gemini** diferencia-se de fornecedores como o **OpenAI** ou a **Anthropic** por não adotar objetos específicos para deltas parciais. Em vez disso, reutiliza a mesma estrutura da resposta unária (`GeminiGenerateContentResponse`) para en-

capsular cada evento da *stream*. Como consequência, o tradutor mapeia os deltas granulares de domínio (`CommonResponseEvent`) reconstruindo a hierarquia do DTO nativo a cada evento emitido.

Common Response Event	Mapeamento para Evento SSE (Gemini Chunk)
<code>ResponseStarted</code>	Emite uma instância básica contendo apenas os metadados iniciais, como o <code>modelVersion</code> e o <code>responseId</code> .
<code>ChoiceStarted</code>	Inicializa as instâncias de <code>GeminiCandidate</code> , definindo apenas os atributos essenciais, como o índice (<code>index</code>) e o papel (<code>role</code>).
<code>TextDeltaEvent</code>	Encaminha o fragmento de texto (<i>delta</i>) reconstruindo a estrutura aninhada da resposta: <code>candidates[] -> content -> parts[] -> GeminiResponsePart(text)</code> .
<code>ToolCallStartedEvent</code>	Gera um candidato que assinala o início da chamada de função através de um <code>GeminiResponsePart(functionCall)</code> , configurando o seu nome e inicializando os argumentos com um <code>JsonObject</code> vazio.
<code>ToolCallArgumentsDeltaEvent</code>	Transmite os fragmentos incrementais dos argumentos utilizando uma chave provisória (" <code>partialJson</code> "), que transporta a <i>string</i> bruta para facilitar a posterior reconstrução do <i>payload</i> .
<code>ChoiceFinished</code>	Emite apenas o motivo de paragem (<code>finishReason</code>) correspondente ao término desta resposta parcial (<i>choice</i>); no caso de <code>TOOL_CALL</code> , o valor nativo emitido é <code>STOP</code> .
<code>UsageReported</code>	Emite os contadores de <i>tokens</i> consumidos, encapsulados num objeto <code>GeminiUsageMetadata</code> .
<code>ResponseCompleted</code>	Sinaliza o fim da <i>stream</i> , emitindo um evento com uma estrutura idêntica à do <code>ResponseStarted</code> .
<code>ResponseErrored</code>	Emite uma resposta com <code>GeminiPromptFeedback</code> . Quando o erro é marcado como recuperável, o <code>blockReason</code> recebe o prefixo <code>TRANSIENT_ERROR</code> ; nos restantes casos, recebe o prefixo <code>ERROR</code> .

Tabela 4.10: Mapeamento Inbound: Eventos **SSE Gemini** (`fromDomainEvent`)

4.9 Tradutores de Saída (Outbound)

Os *Outbound Translators* ligam o modelo de domínio comum do **OmniAI SDK** às APIs externas usadas pelo *gateway*. No envio, constroem o pedido nativo de cada fornecedor a partir de um `CommonRequest`. Na receção, fazem o percurso inverso: convertem respostas completas e eventos de *streaming SSE* para `CommonResponse` e `CommonResponseEvent`. As opções que pertencem ao domínio comum ficam explícitas no contrato comum, enquanto detalhes específicos de cada fornecedor são preservados através de `providerOptions`.

4.9.1 OpenAI Outbound Translator

A implementação `OpenAiOutboundTranslator` adapta o `CommonRequest` ao formato de *chat completions* da **OpenAI**. Os campos comuns são mapeados diretamente sempre que possível, e as opções próprias da **OpenAI** são lidas da `TypedMap` de *provider*.

Domínio Comum	Mapeamento para Contrato Saída (OpenAI Request)
<code>model</code>	Mapeado de forma direta.
<code>messages[].content</code>	Os blocos <code>TextPart</code> são unidos com quebras de linha. Os <code>ToolCallPart</code> são convertidos em <code>OpenAiToolCall(id, function(name, arguments))</code> , usando a representação textual dos argumentos. Quando a mensagem tem papel <code>TOOL</code> , o primeiro <code>ToolResultPart</code> preenche o <code>content</code> e o respetivo <code>toolCallId</code> .
<code>config</code>	<code>temperature</code> , <code>maxTokens</code> e <code>topP</code> são transferidos para o pedido nativo. O <code>maxTokens</code> é limitado com <code>coerceAtMost(4000)</code> e assume 1000 por defeito. As <code>stopSequences</code> são convertidas para <code>OpenAiStop</code> .
<code>providerOptions</code>	São lidas, quando existem, as opções específicas <code>frequencyPenalty</code> , <code>presencePenalty</code> , <code>n</code> , <code>stream</code> , <code>seed</code> , <code>user</code> , <code>logitBias</code> , <code>logProbs</code> e <code>topLogProbs</code> .
<code>tools</code>	Cada <code>CommonTool</code> é convertido para <code>OpenAiFunctionDefinition(name, description, parameters)</code> dentro de um <code>OpenAiTool</code> .
<code>toolChoice</code>	Traduz <code>Auto</code> , <code>None</code> e <code>Required</code> para os modos nativos <code>auto</code> , <code>none</code> e <code>required</code> . Nos casos específicos, força a função através de <code>OpenAiToolChoice.Function</code> .
<code>jsonResponse</code>	Quando <code>jsonResponse</code> está ativo, define <code>OpenAiResponseFormat(type = "json_object")</code> .

Tabela 4.11: Mapeamento Outbound: CommonRequest para **OpenAI** Request

Na receção de uma resposta unária, o tradutor converte a resposta da **OpenAI** para o modelo comum e preserva metadados úteis do fornecedor.

Contrato Resposta (OpenAI)	Mapeamento para o Domínio
<code>id & model</code>	Mapeados diretamente para <code>CommonResponse</code> . Os campos <code>created</code> e <code>systemFingerprint</code> , quando presentes, são guardados em <code>providerOptions</code> .
<code>choices[].message.role</code>	Convertido para <code>CommonRole</code> . Se o papel vier omissa, é assumido <code>assistant</code> .
<code>choices[].message.content</code>	O conteúdo textual é convertido para <code>TextPart</code> . Se o texto aparentar conter um objeto ou lista JSON e o <code>parse</code> for válido, é convertido para <code>JsonPart</code> . As <code>toolCalls</code> da mensagem ou do <code>delta</code> são convertidas para <code>ToolCallPart</code> .
<code>choices[].finishReason</code>	Converte <code>stop</code> , <code>length</code> , <code>tool_calls</code> e <code>content_filter</code> para os motivos comuns correspondentes; valores desconhecidos ficam como <code>OTHER</code> .
<code>usage</code>	Mapeia <code>promptTokens</code> , <code>completionTokens</code> e <code>totalTokens</code> para <code>inputTokens</code> , <code>outputTokens</code> e <code>totalTokens</code> .

Tabela 4.12: Mapeamento Outbound: **OpenAI** Response Unária para Domínio

Para respostas em *streaming* via **SSE**, o tradutor mantém o `id` e o `model` entre eventos, porque nem todos os *chunks* da **OpenAI** repetem esses metadados.

Evento SSE (OpenAI)	Mapeamento para Common Response Event
Estado do fluxo (runningFold)	Mantém um <code>OpenAiEventContext</code> com o último <code>id</code> e <code>model</code> conhecidos, reutilizando-os nos eventos seguintes quando o <code>chunk</code> não os inclui.
<code>choices.first().delta</code>	Um <code>delta.role</code> origina <code>ChoiceStarted</code> . Um <code>delta.content</code> origina <code>TextDeltaEvent</code> .
<code>delta.toolCalls</code>	Se os argumentos parciais têm conteúdo e o nome da função está ausente, o evento é traduzido para <code>ToolCallArgumentsDeltaEvent</code> . Caso contrário, inicia uma chamada de ferramenta com <code>ToolCallStartedEvent</code> .
<code>finishReason</code>	Quando a <code>choice</code> inclui motivo de conclusão, é emitido <code>ChoiceFinished</code> com o <code>finishReason</code> convertido para o domínio comum.
<code>usage</code>	Quando chega num <code>chunk</code> sem <code>choices</code> , é convertido para <code>UsageReported</code> .
Done ou Chunk final	O fim explícito do fluxo, ou um objeto que já não corresponde a <code>chat.completion.chunk</code> , é traduzido para <code>ResponseCompleted</code> .
Error	Gera <code>ResponseErrored</code> . O erro só é marcado como <code>retryable</code> quando o tipo é <code>server_error</code> , <code>rate_limit_error</code> ou <code>temporarily_unavailable</code> .

Tabela 4.13: Mapeamento Outbound: **OpenAI** Streaming para Domínio (`toDomainEvent`)

4.9.2 Anthropic Outbound Translator

O `AnthropicOutboundTranslator` adapta o domínio comum à *Messages API* da **Anthropic**. O `systemPrompt` é enviado no campo próprio `system`, enquanto mensagens, ferramentas e opções específicas são organizadas segundo o contrato nativo.

Domínio Comum	Mapeamento para Contrato Saída (Anthropic Request)
<code>system</code>	O <code>systemPrompt</code> é convertido para <code>RawText</code> quando contém texto não vazio.
<code>messages[].role</code>	<code>USER</code> é convertido para <code>user</code> ; <code>SYSTEM</code> , <code>ASSISTANT</code> e <code>TOOL</code> são enviados como <code>assistant</code> .
<code>messages[].content</code>	Uma mensagem com um único <code>TextPart</code> é enviada como <code>RawText</code> . Nos restantes casos, é criado um <code>ListContentBlock</code> : <code>TextPart</code> gera <code>Text</code> , <code>ToolCallPart</code> gera <code>ToolUse</code> , <code>ToolResultPart</code> gera <code>ToolResult</code> e <code>JsonPart</code> é enviado como texto JSON.
<code>config</code>	Mapeia <code>temperature</code> , <code>topP</code> e <code>stopSequences</code> . O <code>maxTokens</code> usa o valor configurado ou 1024 por defeito.
<code>providerOptions</code>	Inclui opções próprias da Anthropic , como <code>stream</code> , <code>topK</code> , <code>stopToken</code> , <code>thinking</code> e <code>metadata</code> .
<code>tools</code>	Cada <code>CommonTool</code> é convertido para <code>AnthropicToolDefinition(name, description, inputSchema)</code> .
<code>toolChoice</code>	Converte <code>Auto</code> , <code>None</code> e <code>Required</code> para <code>auto</code> , <code>none</code> e <code>any</code> . Uma escolha específica é enviada como <code>type = "tool"</code> com o nome da função.

Tabela 4.14: Mapeamento Outbound: `CommonRequest` para **Anthropic** Request

Na receção de uma resposta unária da **Anthropic** (`AnthropicMessageResponse`), o tradutor cria uma única `CommonChoice` e converte cada bloco de conteúdo para a representação comum.

Contrato Resposta (Anthropic)	Mapeamento para o Domínio
id & model	Mapeados diretamente para <code>CommonResponse</code> ; a resposta é representada numa <code>CommonChoice</code> com índice 0.
content[].type == text	Convertido para <code>TextPart</code> .
content[].type == tool_use	Convertido para <code>ToolCallPart</code> , preservando o id, o nome da função e as propriedades do input.
content[].type == thinking	Como este tradutor não tem um bloco próprio para <code>thinking</code> no domínio comum, o conteúdo é preservado como <code>RefusalPart(reason = thinking)</code> .
stopReason	Converte <code>end_turn</code> , <code>max_tokens</code> , <code>tool_use</code> e <code>stop_sequence</code> para os motivos comuns correspondentes.
usage	Mapeia <code>inputTokens</code> e <code>outputTokens</code> ; o total é calculado a partir desses dois valores quando disponíveis.

Tabela 4.15: Mapeamento Outbound: **Anthropic** Response Unária para Domínio

Em *streaming SSE*, a **Anthropic** já separa o fluxo em eventos tipados. O tradutor converte cada tipo de evento para o evento comum correspondente e mantém em contexto o `id` e o `model` recebidos no início da mensagem.

Evento SSE (Anthropic)	Mapeamento para Common Response Event
Estado do fluxo (<code>runningFold</code>)	Mantém um <code>AnthropicEventContext</code> com o <code>id</code> e o <code>model</code> da mensagem, reutilizando-os nos eventos seguintes do mesmo fluxo.
<code>MessageStart</code>	Convertido para <code>ResponseStarted</code> , usando o <code>id</code> e o <code>model</code> da mensagem nativa.
<code>ContentBlockStart</code>	Um bloco <code>Text</code> origina <code>ChoiceStarted</code> . Um bloco <code>ToolUse</code> origina <code>ToolCallStartedEvent</code> . Um bloco <code>Thinking</code> também inicia uma <i>choice</i> de assistente.
<code>ContentBlockDelta</code>	<code>TextDelta</code> , <code>SignatureDelta</code> e <code>ThinkingDelta</code> são convertidos para <code>TextDeltaEvent</code> . <code>InputJsonDelta</code> é convertido para <code>ToolCallArgumentsDeltaEvent</code> .
<code>ContentBlockStop</code>	Convertido para <code>ChoiceFinished</code> no índice do bloco recebido.
<code>MessageDelta</code>	Quando inclui <code>usage</code> , gera <code>UsageReported</code> ; caso contrário, gera <code>ChoiceFinished</code> com o <code>stopReason</code> convertido.
<code>MessageStop</code>	Convertido para <code>ResponseCompleted</code> .
<code>Ping</code>	Mantém o fluxo ativo e é representado como <code>ResponseStarted</code> .
<code>Error</code>	Gera <code>ResponseErrored</code> . O campo <code>retryable</code> é marcado quando o tipo de erro contém <code>overloaded</code> .

Tabela 4.16: Mapeamento Outbound: **Anthropic** Streaming para Domínio (`toDomainEvent`)

4.9.3 Gemini Outbound Translator

O `GeminiOutboundTranslator` adapta o domínio comum ao formato `GenerateContent` da API **Gemini**. A implementação trata ainda diferenças próprias da plataforma, como a limpeza dos JSON *schemas* de ferramentas e a forma como os eventos progressivos reutilizam a estrutura de resposta completa.

Domínio Comum	Mapeamento para Contrato Saída (Gemini Request)
<code>systemInstruction</code>	O <code>systemPrompt</code> é convertido para <code>GeminiSystemInstruction</code> , contendo uma lista de <code>GeminiPart</code> com o texto do <i>prompt</i> sistêmico.
<code>messages[].content</code>	<code>TextPart</code> é enviado como texto. <code>ToolCallPart</code> gera <code>functionCall</code> e inclui <code>thoughtSignature = "skip_thought_signature_validator"</code> . <code>ToolResultPart</code> gera <code>functionResponse</code> . <code>JsonPart</code> é enviado como texto JSON.
<code>config</code>	<code>stopSequences</code> , <code>temperature</code> e <code>topP</code> são colocados em <code>GeminiGenerationConfig</code> . As opções <code>topK</code> , <code>thinkingConfig</code> , <code>responseMimeType</code> e <code>responseJsonSchema</code> são lidas de <code>providerOptions</code> ; quando <code>jsonResponse</code> está ativo, o <code>responseMimeType</code> passa a <code>application/json</code> .
<code>tools & cleanGeminiParameters</code>	As ferramentas são agrupadas num <code>GeminiTool</code> com <code>functionDeclarations</code> . O <code>parametersSchema</code> é limpo recursivamente para remover chaves não suportadas (<code>\$schema</code> , <code>additionalProperties</code> , <code>propertyNames</code> , <code>title</code> , <code>default</code> , <code>\$id</code> , <code>exclusiveMinimum</code>); valores <code>const</code> são convertidos para <code>enum</code> .
<code>toolChoice</code>	Converte <code>Auto</code> , <code>None</code> e <code>Required</code> para os modos <code>AUTO</code> , <code>NONE</code> e <code>ANY</code> . Em escolhas específicas, usa <code>ANY</code> com <code>allowedFunctionNames</code> .

Tabela 4.17: Mapeamento Outbound: CommonRequest para Gemini Request

Na resposta unária `GeminiGenerateContentResponse`, o tradutor percorre os candidatos devolvidos e converte as partes reconhecidas para o domínio comum.

Contrato Resposta (Gemini)	Mapeamento para o Domínio
<code>id & model</code>	Usa <code>responseId</code> quando existe; se estiver ausente, gera um identificador local por UUID. O <code>modelVersion</code> é usado como modelo e, se vier ausente, fica vazio.
<code>candidates[].parts</code>	Partes de texto geram <code>TextPart</code> . Partes <code>functionCall</code> geram <code>ToolCallPart</code> , com <code>toolCallId</code> local <code>gemini-tool-call-0</code> e argumentos convertidos a partir de <code>args</code> .
<code>candidates[].finishReason</code>	Se a resposta contém chamadas de ferramenta, o motivo comum fica como <code>TOOL_CALL</code> . Caso contrário, <code>STOP</code> , <code>MAX_TOKENS</code> e <code>SAFETY</code> são convertidos para <code>STOP</code> , <code>LENGTH</code> e <code>CONTENT_FILTER</code> .
<code>promptFeedback</code>	Quando existe, o motivo de bloqueio é preservado em <code>providerOptions</code> sob a chave <code>promptFeedback</code> .
<code>usageMetadata</code>	Mapeia <code>promptTokenCount</code> , <code>candidatesTokenCount</code> e <code>totalTokenCount</code> para a estrutura comum de utilização.

Tabela 4.18: Mapeamento Outbound: Gemini Response Unária para Domínio

Nos eventos progressivos da API **Gemini**, cada *chunk* chega com a mesma estrutura de `GeminiGenerateContentResponse`. O tradutor analisa o conteúdo disponível em cada emissão e produz o evento comum correspondente.

Evento SSE (Gemini)	Mapeamento para Common Response Event
Estado do fluxo (runningFold)	Mantém um <code>GeminiEventContext</code> com o último <code>id</code> e <code>model</code> conhecidos, para preencher eventos seguintes quando o <code>chunk</code> não repete esses dados.
<code>promptFeedback</code>	Quando inclui <code>blockReason</code> , gera <code>ResponseErrored</code> . O erro é marcado como <code>retryable</code> quando o motivo contém <code>TRANSIENT</code> .
<code>candidates</code> vazio	Se existir <code>usageMetadata</code> , gera <code>UsageReported</code> ; caso contrário, assinala o início do fluxo com <code>ResponseStarted</code> .
<code>finishReason</code>	Quando presente num candidato, gera <code>ChoiceFinished</code> com o motivo convertido para o domínio comum.
<code>functionCall</code>	Se o nome da função está presente, inicia a chamada com <code>ToolCallStartedEvent</code> . Como a Gemini não fornece identificador de ferramenta neste ponto, é criado um <code>toolCallId</code> local no formato <code>gemini-tool-call-{Random}</code> . Se o nome está ausente mas existem argumentos, emite <code>ToolCallArgumentsDeltaEvent</code> ; a chave <code>partialJson</code> é usada quando existe, caso contrário é usada a representação textual dos argumentos.
<code>text</code>	Um fragmento textual em <code>parts</code> é convertido para <code>TextDeltaEvent</code> .
<code>content.role</code>	Quando o <code>chunk</code> apenas indica o papel da resposta, é emitido <code>ChoiceStarted</code> .
<code>Done</code>	Convertido para <code>ResponseCompleted</code> .
<code>Error</code>	Gera <code>ResponseErrored</code> . O campo <code>retryable</code> é marcado para estados <code>UNAVAILABLE</code> , <code>RESOURCE_EXHAUSTED</code> , <code>DEADLINE_EXCEEDED</code> , <code>ABORTED</code> e <code>INTERNAL</code> .

Tabela 4.19: Mapeamento Outbound: **Gemini** Streaming para Domínio (`toDomainEvent`)

4.10 Evolução para a Pipeline de Interceptors

A *pipeline* foi implementada como um *adapter* de *dispatcher*. Isto é, os *inbounds* não recebem uma *pipeline*; recebem sempre um `DispatcherPort`. Quando a aplicação escolhe `useNativePipeline`, o **SDK** monta uma `GatewayPipeline` com *interceptors* e um *dispatcher* terminal, e depois expõe essa cadeia através de `PipelineBackedDispatcher`. Se a aplicação não quiser a *pipeline*, pode fornecer diretamente um *dispatcher* próprio através de `useCustomDispatcher`.

Esta decisão evita que a *pipeline* se torne uma dependência obrigatória da arquitetura. Um consumidor que pretenda apenas tradução de contratos e despacho direto pode usar um *dispatcher* simples. Um consumidor que precise de segurança, observabilidade e resiliência pode ativar a *pipeline* incluída. Um consumidor com necessidades próprias pode implementar outro `DispatcherPort`, por exemplo com seleção por custo, políticas externas, filas internas ou integração com outro orquestrador.

Modo de montagem	Componente visto pelos inbounds	Justificação
Dispatcher direto	Uma implementação de <code>DispatcherPort</code> .	Útil quando se pretende uma Gateway mínima, com tradução e envio para <i>outbounds</i> sem lógica transversal.
Pipeline nativa	<code>PipelineBackedDispatcher</code> , também exposto como <code>DispatcherPort</code> .	Permite aplicar autenticação, autorização, métricas, <i>rate limiting</i> , <i>circuit breaker</i> , <i>fallback</i> e <i>logging</i> sem alterar <i>adapters</i> .
Dispatcher customizado	Qualquer implementação externa de <code>DispatcherPort</code> .	Mantém o SDK aberto a estratégias futuras, incluindo roteamento por custo, latência, plano de cliente ou políticas de negócio.

Tabela 4.20: Formas de expor a execução aos *inbounds* sem os acoplar à *pipeline*

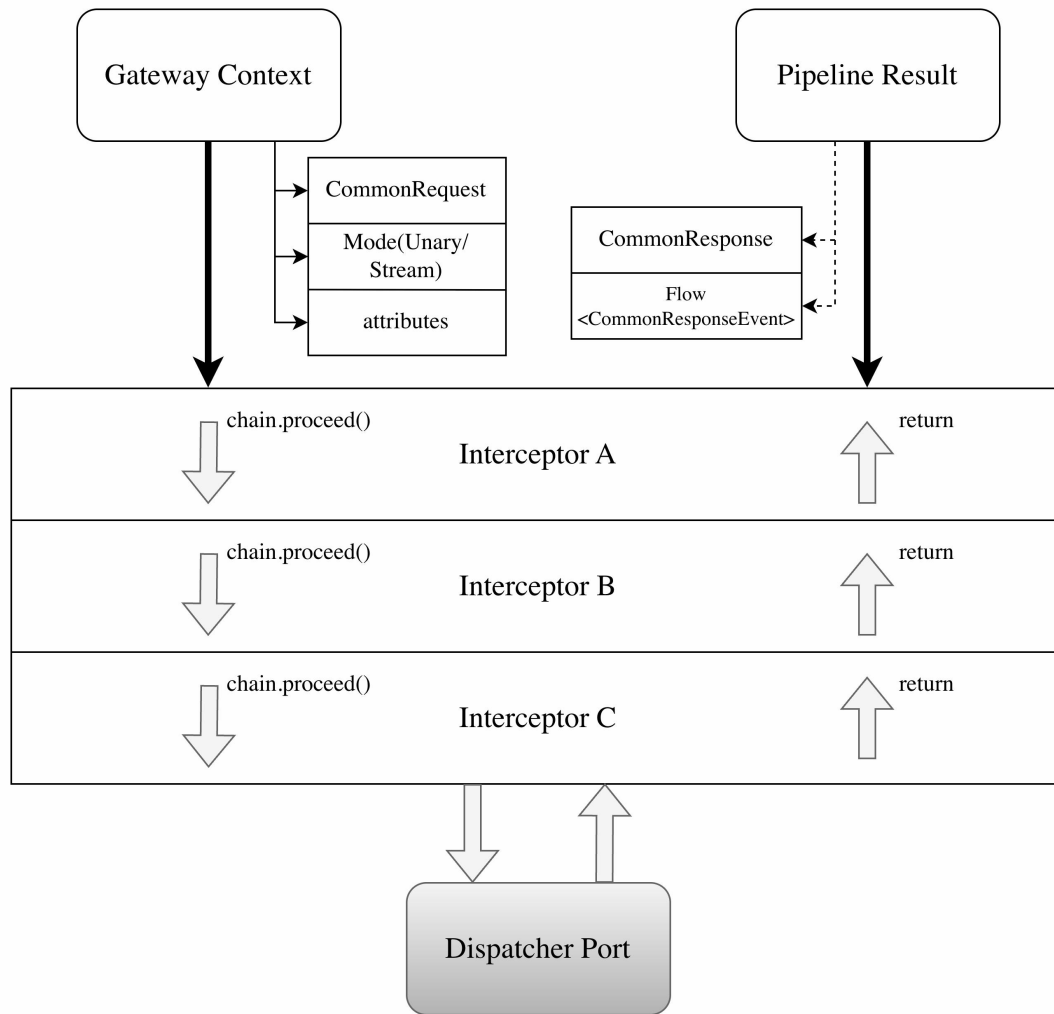


Figura 4.2: Visão conceitual da *pipeline* de *interceptors*

Esta forma mantém a arquitetura simples. Para um caso mínimo, a **Gateway** pode operar com um *dispatcher* direto. Para um caso operacional, a *pipeline* permite adicionar autenticação, autorização, métricas, limites, *circuit breaker*, *fallback*, *logging* e *tracing* sem alterar *adapters*. O modelo é semelhante ao dos filtros de **Servlets** [9]: cada filtro recebe um contexto, decide se continua a cadeia e pode executar lógica antes e depois do próximo elemento. A diferença é que, no **OmniAI SDK**, o contexto não representa apenas um pedido **HTTP**; representa uma chamada de inferência já normalizada, incluindo fornecedor, modelo, modo de execução, atributos tipados e eventual fluxo de *streaming*.

A *pipeline* é composta por `GatewayPipeline`, `GatewayPipelineChain`, `Interceptor` e `GatewayContext`. O `GatewayContext` transporta o pedido comum, atributos tipados e modo de execução, podendo representar chamadas síncronas ou *streaming*. A cadeia executa os *interceptors* por ordem e, no fim, invoca o *dispatcher* terminal.

Esta abordagem também prepara o escalamento horizontal. A *pipeline* não exige estado global obrigatório; cada pedido transporta o seu contexto. Onde existe estado partilhável, como *rate limiting* ou *circuit breaker*, o **SDK** define *stores* substituíveis. A implementação em memória é adequada para desenvolvimento e instância única, mas pode ser trocada por **Redis**, base de dados ou outro *backend* coordenado em implantações distribuídas. Assim, várias instâncias da **OmniAI Gateway** podem partilhar contadores, estados de circuito e metadados operacionais sem alterar o contrato dos *inbounds* nem a forma como os clientes chamam a **Gateway**.

4.11 Arquitetura Interna dos Interceptors

O módulo `interceptors` inclui implementações pré-construídas para preocupações operacionais e de segurança. Todos seguem o mesmo contrato: recebem um `GatewayContext`, consultam ou acrescentam atributos à `TypedMap`, decidem se chamam `chain.proceed(context)` e podem observar ou transformar o `PipelineResult`. Esta estrutura permite colocar lógica transversal antes e depois da inferência sem acoplar essa lógica aos *adapters* de entrada, aos *adapters* de saída ou ao *dispatcher*.

A ordem de execução é configurável. No *target JVM*, o **SDK** suporta a anotação `Interceptor Priority`, usada para executar primeiro *interceptors* com prioridade superior. No *target JavaScript*, onde a reflexão sobre anotações é limitada, a ordem segue a sequência definida pela DSL. Assim, autenticação e autorização podem ocorrer no início da cadeia, enquanto métricas, *tracing* e *logging* podem envolver a execução completa.

A motivação para estes *interceptors* não foi apenas organizar código, mas tornar explícitas políticas que, numa **Gateway** de IA, afetam segurança, custo e disponibilidade. A Tabela 4.21 resume essa relação.

Interceptor	Problema tratado	Motivo para estar na Gateway
Telemetria	Falta de visibilidade sobre latência, erros, <i>tokens</i> e fornecedores.	A Gateway observa todas as chamadas e consegue medir comportamento transversal sem alterar clientes ou <i>adapters</i> .
Autenticação	Necessidade de validar quem apresenta o token antes de consumir modelos pagos.	A Gateway é o recurso protegido e deve validar <i>tokens</i> antes de encaminhar tráfego para fornecedores.
Autorização	Um token válido não implica permissão para todos os modelos ou operações.	A decisão por cliente, ação, fornecedor e modelo deve ser centralizada antes da inferência.
Rate limiting	Risco de abuso, picos de carga e custos inesperados.	A Gateway é o ponto natural para aplicar quotas por cliente, plano, fornecedor ou modelo.
Circuit breaker	Dependências externas podem ficar lentas ou indisponíveis.	A Gateway evita insistir em fornecedores em falha e protege o restante sistema.
Fallback	Uma falha de fornecedor não deve, quando existe alternativa, falhar imediatamente a experiência do cliente.	A Gateway conhece os <i>outbounds</i> disponíveis e pode tentar alternativas mantendo o contrato do cliente.

Tabela 4.21: Justificação dos *interceptors* fornecidos pelo **SDK**

4.11.1 Interceptor de Telemetria e Observabilidade

O `MetricsInterceptor` recolhe dados operacionais essenciais num sistema que intermedeia chamadas a LLMs: latência, sucesso, erro, fornecedor, modelo, cliente e consumo de *tokens*. Estes dados são importantes porque uma **Gateway** centraliza tráfego de vários clientes e fornecedores; sem telemetria, não é possível perceber custos, degradação, falhas por modelo ou impacto de políticas como *fallback*.

A implementação distingue chamadas unárias de *streaming*. Em respostas progressivas, o *interceptor* não bloqueia a execução; usa operadores de fluxo para observar eventos e emitir métricas no encerramento do *stream*. Assim mede a duração total da interação, e não apenas o tempo até ao primeiro fragmento.

A arquitetura segue inversão de dependência através da porta `MetricsPort`. O **OmniAI SDK** define primitivas de métricas comuns, como `counter`, `histogram` e `upDownCounter`, enquanto o *adapter* concreto no *target JVM* exporta para **OpenTelemetry**. O uso do **OpenTelemetry Protocol (OTLP)** foi escolhido porque o **OTLP** define a codificação, transporte e entrega de dados de telemetria entre fontes, coletores e *backends* [55]. Esta escolha evita o acoplamento do **OmniAI SDK** a ferramentas específicas

como **Prometheus**, **Grafana**, **Datadog** ou outro *backend* específico: o **SDK** emite telemetria num formato aberto, o Collector recebe e processa esses dados, e a infraestrutura decide para onde exportar.

Esta decisão é especialmente importante numa **Gateway** de IA. O mesmo **OmniAI SDK** pode ser usado por uma aplicação simples em desenvolvimento, por um POC local com **Prometheus/Grafana** ou por uma instalação futura com outro *backend* de observabilidade. A **OpenTelemetry** apresenta-se precisamente como especificação aberta e agnóstica de fornecedor [49]; por isso, a instrumentação fica no **OmniAI SDK**, mas a escolha operacional do *backend* permanece fora do domínio.

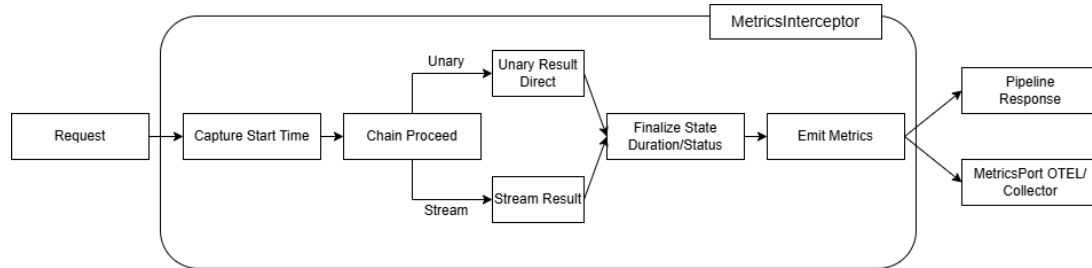


Figura 4.3: Fluxo interno de processamento e lógica de observabilidade para pedidos unários e *streaming*

Para adicionar uma métrica, a aplicação usa a DSL do **gateway-client**. O utilizador pode ativar a latência padrão, acrescentar atributos globais e definir métricas próprias com uma função **value**, que extrai o valor a partir do contexto e do resultado, e uma função **tags**, que acrescenta dimensões de análise.

```

metrics {
  metricsPort = telemetryRuntime.metricsPort
  tracer = telemetryRuntime.tracer

  attributes {
    include(ClientIpMetadataKey, alias = "client.ip")
    attribute("sdk.version") { _, _ -> "1.0.0" }
  }

  defaultLatency {
    enabled = true
    name = "gateway.inference.request.duration"
  }

  customMetrics {
    counter(
      name = "gateway.llm.tokens",
      description = "Total de tokens consumidos",
      unit = "{tokens}"
    ) {
      value { _, result ->
        (result as? PipelineResult.Unary)?.response?.usage?.totalTokens
      }
    }
  }
}

```

Listing 4.1: Configuração de telemetria e métricas personalizadas na DSL

Este desenho separa três responsabilidades: a definição da métrica na DSL, a recolha no *interceptor* e a exportação no *adapter* de telemetria. Também facilita testes, porque **NoOpMetricsPort** pode ser usado

quando a observabilidade está desligada.

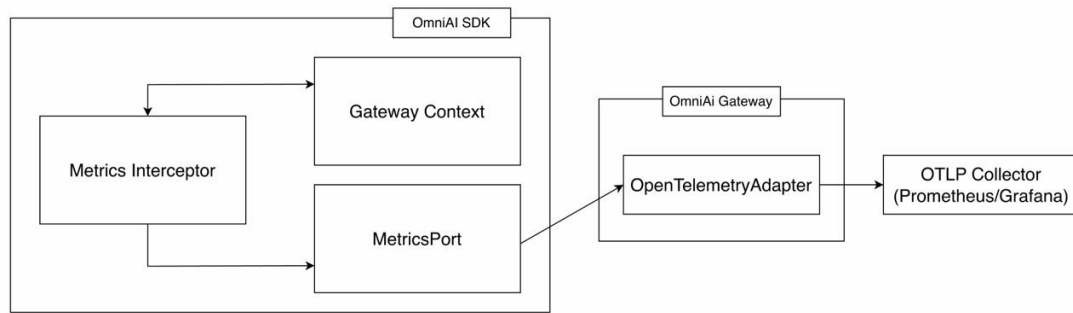


Figura 4.4: Observabilidade e métricas operacionais da **OmniAI Gateway**

4.11.2 Interceptor de Autenticação

A autenticação é realizada pelo **AuthenticationInterceptor**. A **Gateway** atua como *Resource Server*: recebe pedidos para recursos protegidos, extrai o *access token* do cabeçalho **Authorization: Bearer <token>** e valida se esse token foi emitido por um servidor de autorização confiável [56–58]. Esta responsabilidade é essencial numa **AI Gateway**, porque a camada central passa a proteger acesso a modelos, custos, quotas e dados operacionais.

A **Gateway** não deve atuar como *Authorization Server*: não emite *tokens*, não gere credenciais dos clientes e não substitui sistemas como **Logto** ou **Keycloak**. O seu papel é o de API protegida. Por isso, valida *tokens* recebidos, confirma se foram emitidos para a audiência esperada e só depois permite que o pedido avance na *pipeline* para os próximos *interceptors*. Esta separação reduz responsabilidade de segurança dentro do **SDK** e permite integrar diferentes fornecedores de identidade sem alterar o contrato dos clientes.

O fluxo inicia-se no *adapter HTTP*, que propaga o token para o domínio através de metadados. O *interceptor* classifica o token: se tiver três segmentos, é tratado como **JWT**; caso contrário, como token opaco. Esta distinção permite escolher entre validação local por assinatura e validação remota por introspeção.

No modo *OIDC Discovery* (RFC 8414), a **Gateway** obtém dinamicamente metadados do *Authorization Server*, como *issuer*, *jwks_uri* e *endpoint* de introspeção [59]. Todo este fluxo encontra-se ilustrado na Figura 5.4. Para JWTs, valida o *kid*, obtém a chave pública no **JWKS**, verifica assinatura e confirma *claims* como *exp*, *nbf*, *iss* e *aud* [60–62]. Para *tokens* opacos, usa introspeção **OAuth2** [63].

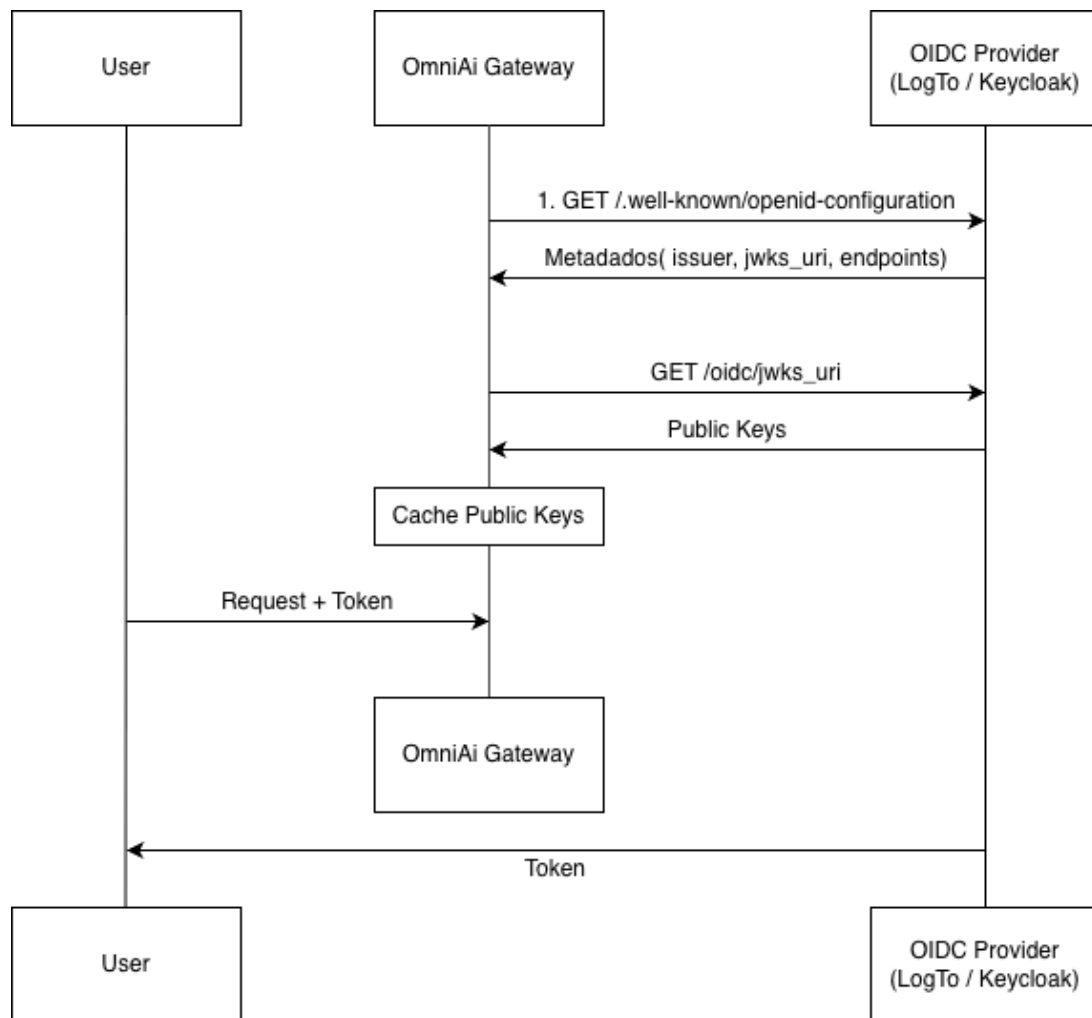


Figura 4.5: Sequência **OIDC**: descoberta, **JWKS** e cache de chaves

Para reduzir latência e chamadas repetidas ao servidor de autorização, a implementação inclui `InMemoryKeyCache`, rotação de chaves em *background* e *cooldown* para evitar consultas excessivas quando uma chave não é encontrada. No caso de *tokens* opacos, o `IntrospectionCache` usa *positive caching* para *tokens* ativos e *negative caching* para *tokens* inválidos. Os *tokens* opacos são guardados em cache através de *hash*, evitando manter credenciais em texto claro em memória.

O resultado é encapsulado em `AuthValidationResult` e guardado na `TypedMap` sob a chave `AUTH_RESULT_KEY`. Isto permite que autorização, métricas e *logging* reutilizem a identidade já validada.

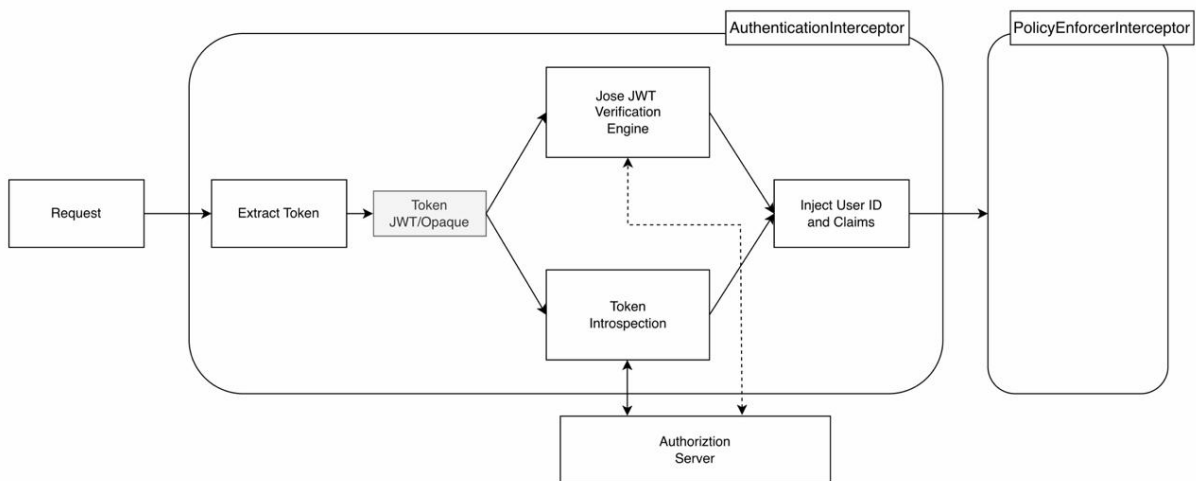


Figura 4.6: Fluxo interno do *interceptor* de autenticação

A **Gateway** final usa **Logto** como *Authorization Server* local [47], executado em **Docker** juntamente com **PostgreSQL**, **OpenTelemetry Collector**, **Prometheus** e **Grafana**. Foram também realizados testes exploratórios com **Keycloak** [64]. A escolha final do **Logto** no POC deveu-se à simplicidade de configuração local, suporte a aplicações *Machine-to-Machine* e facilidade de gerir API Resources durante a validação.

A integração do processo de autenticação é declarada na montagem do *SDK*:

```

security {
  authentication {
    discovery {
      discoveryUrl = "https://auth.example.com/.well-known/openid-configuration"
      expectedAudience = "https://gateway.example.com/api"
      clientId = "gateway-client"
      clientSecret = "secret"
    }
  }
}

```

Listing 4.2: DSL de configuração do *interceptor* de autenticação com **OIDC** Discovery

4.11.3 Interceptor de Autorização

A autorização é separada da autenticação porque responder a “quem é o utilizador?” não é o mesmo que responder a “o que este utilizador pode fazer?”. A autenticação valida identidade; a autorização decide permissões sobre uma ação e um recurso. Esta separação é importante numa **Gateway** multi-cliente, onde um token válido pode não ter permissão para usar todos os modelos, fornecedores ou operações. A distinção entre autenticação e autorização é também a base dos modelos modernos de identidade: a primeira prova identidade, enquanto a segunda concede permissão sobre dados ou ações protegidas [65].

No contexto do projeto, esta separação evita dois problemas. Primeiro, impede que a validação de token seja confundida com autorização total: um cliente autenticado pode estar limitado a um subconjunto de modelos, fornecedores ou quotas. Segundo, permite trocar a origem das decisões de autorização sem alterar a autenticação. Embora numa fase inicial a decisão possa operar de forma local e simples,

o sistema está desenhado para que, numa iteração futura, essa mesma decisão possa ser assegurada por um PDP externo, tal como o AuthZen [66] ou o OPA [67].

No **OmniAI SDK**, a autorização é centralizada no `PolicyEnforcerInterceptor`. O componente atua como *Policy Enforcement Point* (PEP): intercepta o pedido, constrói um `AuthorizationInput` e delega a decisão a um *Policy Decision Point* (PDP) através da porta `PolicyDecisionPointPort`. Podemos ver todo este fluxo na figura 5.6.

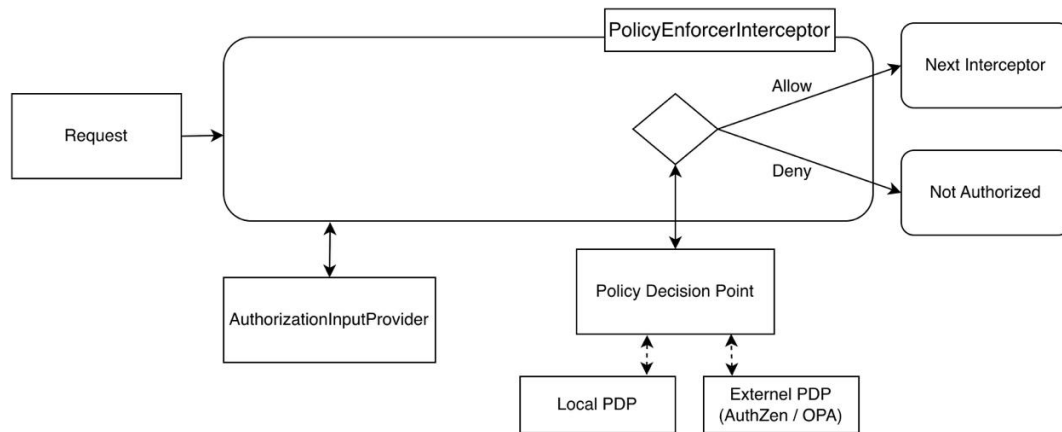


Figura 4.7: Arquitetura do *interceptor* de autorização: separação entre PEP e PDP

O `AuthorizationInput` segue o padrão **Subject-Action-Resource (SAR)**. O **Subject** representa o utilizador ou aplicação, incluindo identificador, roles e atributos vindos do token. A **Action** representa a operação, como `chat_completion`. O **Resource** identifica o alvo, como fornecedor ou modelo. O **Context** acrescenta atributos ambientais, como IP, tempo ou metadados do pedido. A porta do PDP permite integrar motores diferentes, incluindo políticas internas, AuthZen [66], OPA [67] ou outro serviço externo. Se a decisão for *Deny*, a *pipeline* é interrompida antes de qualquer chamada ao fornecedor [68].

A configuração de autorização permite estender o avaliador por omissão:

```

security {
  authorization {
    pdp = customPolicyDecisionPoint
    inputProvider = DefaultAuthorizationInputProvider
  }
}

```

Listing 4.3: DSL de configuração do *interceptor* de autorização

4.11.4 Interceptor de *Rate Limiting*

O `RateLimitInterceptor` aplica limites de utilização antes da chamada ao fornecedor. Numa **AI Gateway**, este mecanismo tem dois objetivos principais: proteger o sistema contra abuso ou tráfego excessivo e controlar custos associados a chamadas de inferência. A limitação de taxa é uma estratégia comum para impor um teto de ações repetidas num intervalo temporal, reduzindo abuso de APIs, força bruta e carga excessiva [69].

A importância deste *interceptor* é maior do que numa API tradicional, porque cada pedido pode representar custo financeiro direto, consumo de quotas externas e ocupação prolongada de ligações em *streaming*. Sem *rate limiting*, um cliente mal configurado, uma automação em ciclo ou um utilizador

abusivo pode saturar a **Gateway**, esgotar quotas nos fornecedores e degradar o serviço para clientes legítimos. O *interceptor* permite aplicar justiça operacional: cada cliente, plano, modelo ou fornecedor pode ter limites próprios.

A lógica é orientada por políticas. Cada `RateLimitPolicy` extrai dados do `GatewayContext` e devolve `RateLimitTarget`. Um *target* contém uma chave, como cliente/modelo/plano, e uma `RateLimitDefinition`, composta por limite, tipo e janela temporal. Isto permite políticas diferentes por cliente, plano, modelo ou fornecedor.

O store atual em memória usa buckets protegidos por `Mutex`. Quando um pedido chega, o *interceptor* tenta consumir uma unidade em todos os buckets aplicáveis. Se todos permitirem consumo, a cadeia continua. Se algum limite estiver esgotado, devolve `TooManyRequestsError` com `retryAfter`. Esta resposta permite que clientes bem comportados reduzam ritmo em vez de insistirem. Para desenvolvimento e instância única, este modelo é suficiente; em escalamento horizontal, o contrato `RateLimitStore` permite substituir a implementação por **Redis** ou outro *backend* partilhado. Essa substituição é necessária quando várias instâncias da **Gateway** devem partilhar a mesma visão de quota.

A configuração deste mecanismo recorre ao seguinte bloco:

```
interceptors {
  rateLimiting {
    store = InMemoryRateLimitStore()
    addPolicy(rateLimitPolicy)
  }
}
```

Listing 4.4: DSL de configuração do *interceptor* de *rate limiting*

4.11.5 Interceptor de *Circuit Breaker*

O `CircuitBreakerInterceptor` protege a **Gateway** contra chamadas repetidas a um *outbound* em falha. O padrão de *circuit breaker* é usado para lidar com falhas em serviços remotos, evitando insistir numa dependência que está indisponível e dando tempo para recuperação [70–72]. Numa **Gateway** de IA, esta proteção é especialmente útil porque fornecedores externos podem falhar por indisponibilidade, *timeouts*, quotas ou erros 5xx.

A alternativa simples seria repetir pedidos até obter sucesso. Essa abordagem é perigosa quando o fornecedor está realmente indisponível: as tentativas acumulam latência, ocupam *threads* ou corrotinas, aumentam custos e podem agravar a falha no serviço externo. O *circuit breaker* foi criado para evitar esse comportamento. Quando a falha ultrapassa um limiar, o circuito abre e a **Gateway** passa a falhar rapidamente aquele destino, libertando recursos e permitindo que mecanismos como *fallback* escolham outro *outbound*.

Para cada combinação de fornecedor/modelo, o *interceptor* identifica o `outbound.key` e consulta o `CircuitBreakerStore`. Se o circuito estiver `OPEN`, o pedido falha rapidamente com `ApiDownError` e o *outbound* é acrescentado ao conjunto de *outbounds* negados no contexto. Esta falha rápida reduz latência, evita consumo de recursos e dá ao *fallback* informação para não tentar novamente o mesmo destino.

Quando a chamada prossegue, o *interceptor* observa o resultado. Erros incrementam o contador de falhas; ao atingir `failureThreshold`, o circuito passa para `OPEN`. Após um período de recuperação, o estado `HALF_OPEN` permite testar um número limitado de pedidos antes de voltar ao estado `CLOSED`, evitando inundar um fornecedor que ainda está a recuperar [70]. A configuração inclui `failureThreshold`, `resetTimeoutMs` e `halfOpenTestRequests`. A implementação atual cobre a abertura e fecho básico do circuito; a evolução temporal mais completa do estado `HALF_OPEN` é uma melhoria natural.

```

interceptors {
  circuitBreaker {
    store = InMemoryCircuitBreakerStore()
    config = CircuitBreakerConfig(
      failureThreshold = 5,
      resetTimeoutMs = 10000L,
      halfOpenTestRequests = 2
    )
  }
}

```

Listing 4.5: DSL de configuração do *interceptor* de *circuit breaker*

4.11.6 Interceptor de *Fallback*

O `FallbackInterceptor` permite tentar alternativas quando a chamada principal falha. Em sistemas de IA, *fallback* ao nível de modelo ou fornecedor é relevante porque falhas nas APIs de inferência fornecidas pelas LLMs são um cenário frequente em produção: *rate limits*, indisponibilidade, quotas, *timeouts* ou modelos descontinuados podem impedir a resposta do fornecedor principal [73,74]. Um *gateway* é o ponto certo para esta política, porque conhece os *outbounds* configurados e consegue trocar de destino sem exigir alterações no cliente.

A decisão de colocar *fallback* na **Gateway**, e não nos clientes, reduz duplicação e melhora controlo operacional. Se cada aplicação cliente implementasse a sua própria lógica de *fallback*, seria necessário repetir regras de prioridade, tratamento de erros, mapeamento de contratos e métricas em vários pontos. Ao centralizar a política, o **SDK** consegue manter o mesmo contrato de entrada e decidir internamente que fornecedor ou modelo alternativo deve ser tentado. Isto é particularmente útil quando o cliente usa um contrato compatível com **Anthropic** ou **OpenAI**, mas a **Gateway** pode executar a chamada noutra *provider* configurado.

A implementação atual usa a lista de *outbounds* disponíveis. Primeiro tenta o *outbound* correspondente ao fornecedor e modelo pedidos; se falhar, acrescenta esse *outbound* ao conjunto de negados e tenta alternativas. Para cada tentativa, cria um novo `GatewayContext` com os mesmos atributos, mas com `provider` e `model` atualizados para o *outbound* alternativo.

A integração com o *circuit breaker* é feita através do conjunto partilhado de *outbounds* negados. Assim, se um circuito estiver aberto, o *fallback* evita esse destino e tenta outro modelo ou fornecedor. Este desenho permite construir políticas de resiliência como “tentar **Gemini**, depois **OpenAI**-compatible via **Groq**, depois **Anthropic**”, mantendo a lógica de tentativa fora dos *adapters*. Também permite distinguir degradação controlada de erro total: a resposta pode chegar por um modelo alternativo, enquanto métricas e *tracing* registam que ocorreu *fallback*.

A evolução futura poderá acrescentar prioridades configuráveis, condições por código de erro, orçamentos de tentativa e métricas específicas de *fallback*. Estas regras são importantes porque nem todo erro justifica *fallback*: erros de autenticação ou autorização devem falhar imediatamente, enquanto *timeouts*, 429 e 5xx podem justificar uma alternativa.

A sua ativação é extremamente simples, não requerendo configuração intrincada:

```
interceptors {
    fallback()
}
```

Listing 4.6: DSL de configuração do *interceptor* de *fallback*

4.11.7 Interceptor de *Logging* e *Tracing*

O `RequestLoggingInterceptor` registra fornecedor, modelo, sucesso, erro e conclusão de streams através da abstração `GatewayLogger`. No *target JVM* pode ser ligado a **SLF4J**; no *target JavaScript* pode usar consola ou outra integração. O objetivo é garantir visibilidade mínima mesmo quando a telemetria completa não está ativa.

O *logging* foi mantido separado das métricas porque responde a perguntas diferentes. Métricas mostram tendências agregadas, como latência média ou número de erros; *logs* ajudam a explicar um pedido concreto. Numa **Gateway** de IA, esta distinção também tem impacto de privacidade: o objetivo é registar metadados operacionais, e não despejar *prompts*, respostas completas ou *tokens* sensíveis nos *logs*.

O `TracingInterceptor` envolve o pedido num *span* chamado `gateway.request.process`. Quando combinado com **OpenTelemetry**, este *span* permite correlacionar latência, erros, chamadas externas e métricas agregadas. Em cenários de *fallback* ou *circuit breaker*, *tracing* é particularmente útil porque permite observar o percurso real do pedido e distinguir falhas do fornecedor, decisões da **Gateway** e erros de cliente. Também permite perceber se uma resposta lenta resulta do fornecedor principal, de uma tentativa alternativa, de validação de token ou de uma política interna.

A instalação processa-se em dois blocos da DSL, um explícito e outro decorrente da configuração de métricas:

```
interceptors {
    // Instalacao manual do Logging Interceptor
    use(RequestLoggingInterceptor(logger))

    // O TracingInterceptor e instalado automaticamente ao fornecer o tracer as metricas
    metrics {
        tracer = telemetryRuntime.tracer
    }
}
```

Listing 4.7: Instalação dos *interceptors* de *logging* e *tracing*

4.12 Distribuição Multiplataforma

Um dos objetivos do **OmniAI SDK** é permitir que a lógica central da abstração de modelos seja reutilizada em diferentes ambientes de execução. O ecossistema **Kotlin Multiplatform** permite separar o código agnóstico das implementações específicas: o domínio, os contratos, as portas, os adaptadores principais, a DSL e parte dos *interceptors* residem em *source sets* comuns; por sua vez, os detalhes de infraestrutura como o cliente **HTTP**, a reflexão de prioridades, a integração com o servidor **Ktor** e as pontes de plataforma ficam isolados em *source sets* específicos.

No *target JVM*, o **OmniAI SDK** pode ser consumido nativamente por aplicações **Kotlin** ou Java. Este é o ambiente mais maduro no estado atual do projeto, sendo a fundação sobre a qual a nossa instância de demonstração (POC) do **OmniAI Gateway** foi construída. A implementação **JVM** inclui

o cliente **HTTP Ktor**, a integração nativa com o servidor **Ktor**, a autenticação baseada em descoberta **OIDC**, a telemetria e a execução real dos *outbounds*. A futura distribuição via **Maven** permitirá que qualquer aplicação adicione o **SDK** como uma dependência versionada clássica, eliminando a necessidade de recorrer a *composite builds*.

Relativamente ao *target* **JavaScript/Node.js**, o projeto prepara a disponibilização de uma API consumível a partir do ecossistema **TypeScript**. A utilização da anotação **@JsExport** atua como uma fachada que encapsula a configuração em tipos exportáveis, garantindo que as estruturas internas do **Kotlin** não transbordam de forma inadequada para a fronteira JS. Esta camada estabelece as fundações para o consumo via **NPM**, embora ainda não apresente paridade total de funcionalidades com o ambiente **JVM**.

A principal premissa desta arquitetura multiplataforma é a de que nem todos os componentes devem ser partilhados. Enquanto o domínio e os contratos devem permanecer estritamente comuns e agnósticos, as camadas de transporte **HTTP**, servidor, observabilidade e segurança podem e devem ter implementações otimizadas para cada plataforma. Esta fronteira arquitetural evita a duplicação de código sem forçar uma abstração demasiado rígida ou ineficiente. A linha de evolução natural passa por consolidar a API pública do **OmniAI SDK**, estabilizar o versionamento semântico, automatizar a publicação **Maven/NPM** através de *Continuous Delivery* (CD) e documentar exemplos mínimos de consumo para a **JVM** e **Node.js**.

Capítulo 5

MCP Broker e Ferramentas Externas

O **MCP Broker** surge como uma extensão natural da **Gateway**: deixar de a tratar apenas como um ponto de entrada para inferência e aproximá-la de uma *gateway* de IA mais completa. Em muitos cenários, um modelo só consegue produzir uma resposta útil se puder consultar recursos externos, chamar ferramentas ou executar ações controladas. É nesse espaço que entra o **Model Context Protocol (MCP)**.

O que foi concretamente implementado neste módulo é uma **MCP Gateway** funcional: um servidor MCP autónomo que lê ficheiros de configuração **YAML** de forma declarativa, regista automaticamente as ferramentas neles descritas, e expõe-as a qualquer cliente MCP compatível, como o **MCP Inspector**. Adicionalmente, o **MCP Broker** é capaz de se ligar a outros servidores MCP externos e de expor as suas ferramentas como proxies locais, funcionando como um ponto de agregação centralizado. Com isto, a Gateway deixa de ser apenas um *proxy* de inferência e passa a ser uma infraestrutura de IA mais completa.

A implementação assenta em duas bibliotecas principais: `io.modelcontextprotocol:kotlin-sdk` (versão 0.13.0) [36], que fornece o servidor MCP, os tipos do protocolo e os transportes suportados (*stdio*, SSE, Streamable HTTP, WebSocket); e `net.mamoe.yamlkt` (versão 0.13.0) [75], que é responsável por desserializar os ficheiros YAML de configuração em objetos Kotlin de forma declarativa. O cliente HTTP utilizado para comunicar com as APIs externas é o **Ktor** [39].

5.1 Visão Geral do Sistema

A arquitetura global do sistema prevê que o MCP Broker opere em conjunto com um **Authorization Server** (AS) para garantir a autenticação e autorização via OAuth 2.1. Contudo, uma vez que este mecanismo de segurança já foi integralmente implementado e comprovado na Gateway de Inferência, a sua integração direta no MCP Broker foi omitida nesta Prova de Conceito (PoC), encontrando-se assinalada a cinza no diagrama.

A Figura 5.1 ilustra o papel do MCP Broker neste módulo: recebe ligações de clientes MCP, expõe ferramentas e contextos, e encarrega-se de as concretizar, seja através de chamadas HTTP a APIs externas, seja fazendo proxy de ferramentas descobertas noutros servidores MCP.

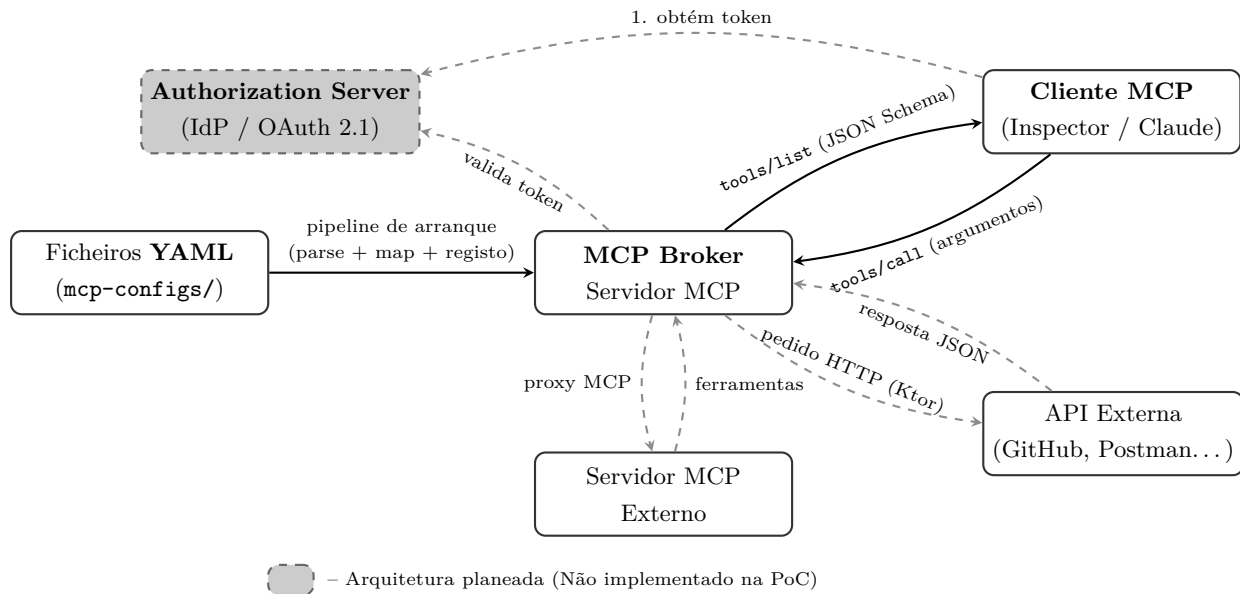


Figura 5.1: Visão geral do MCP Broker: integração planeada com o servidor de autorização, registo de ferramentas via YAML, execução de pedidos HTTP a APIs externas e proxy de servidores MCP externos.

5.2 Modelos de Domínio e DTOs

O MCP Broker organiza a sua configuração em duas camadas distintas: os **DTOs** (Data Transfer Objects), que representam a estrutura dos ficheiros YAML, e os **modelos de domínio**, que são os objetos internos sobre os quais a lógica do MCP Broker efetivamente opera. Esta separação evita que o código fique dependente do formato de serialização.

5.2.1 DTOs de Configuração

Os DTOs representam diretamente a estrutura dos ficheiros YAML e são anotados com `@Serializable`, a anotação da biblioteca `kotlinx.serialization`, para que o `yamlkt` os consiga desserializar automaticamente. O `BrokerConfigDto` é o objeto de entrada, agregando as três listas de configuração possíveis.

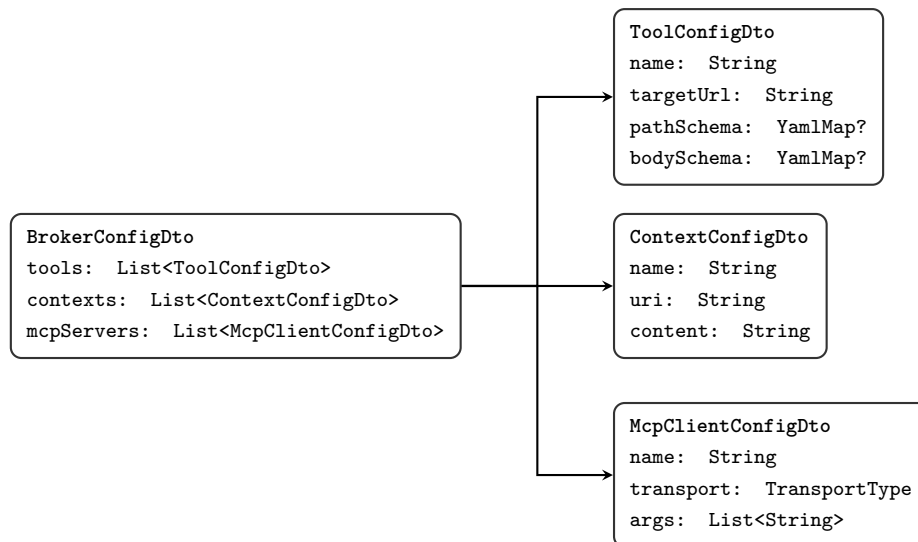


Figura 5.2: Hierarquia dos DTOs de configuração.

5.2.2 Modelos de Domínio

Após a fase de mapeamento, os DTOs dão lugar aos objetos de domínio. Estes objetos são independentes do formato de serialização e contêm os tipos já convertidos, facilitando o seu uso no resto da aplicação.

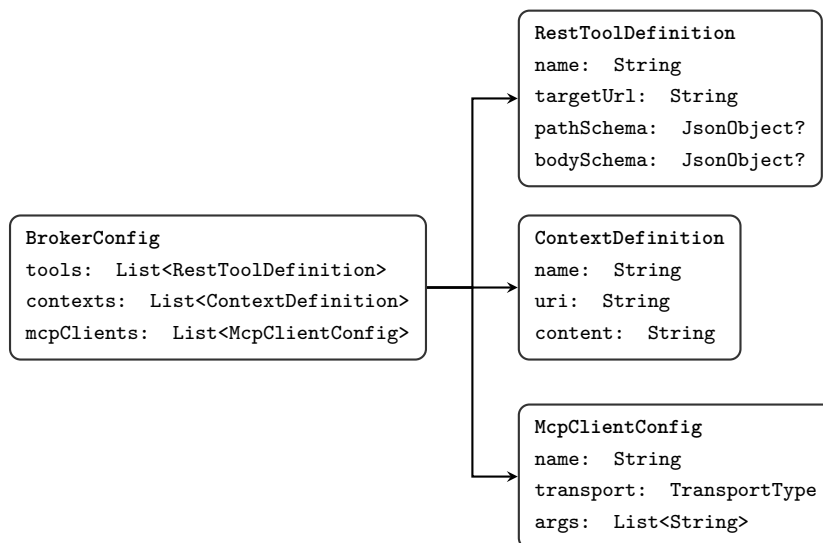


Figura 5.3: Modelos de domínio e a substituição das estruturas YAML por equivalentes em JSON.

A diferença mais relevante entre os DTOs e os modelos de domínio está na representação dos schemas de entrada. No DTO, estes campos são deserializados como `YamlMap`, preservando a estrutura em árvore do YAML. No modelo de domínio, essa estrutura é convertida para `JsonObject`, o tipo nativo da biblioteca `kotlinx.serialization`, garantindo que os tipos de cada valor (inteiro, booleano, objeto aninhado, etc.) são preservados corretamente.

5.3 Configuração Declarativa em YAML

Toda a configuração do MCP Broker é feita através de ficheiros **YAML** colocados numa pasta de configuração. Cada ficheiro pode declarar três secções independentes:

- **tools:** ferramentas REST que o MCP Broker expõe nativamente.
- **contexts:** conteúdo estático exposto na forma de recursos.
- **mcpServers:** servidores MCP externos aos quais o MCP Broker se liga.

5.3.1 Descrição de Ferramentas REST

A configuração de uma ferramenta REST é feita de forma declarativa, sem escrever código. Basta indicar o nome, a descrição, o URL de destino, o método HTTP, os cabeçalhos, e as seções de entrada de dados. Para permitir um roteamento preciso dos argumentos, a validação de entrada divide-se em três partes:

- **pathSchema:** parâmetros que são substituídos diretamente no URL, através da sintaxe de chavetas no formato {chave}.
- **querySchema:** parâmetros que são acrescentados ao URL como variáveis de interrogação.
- **bodySchema:** parâmetros que são enviados no corpo do pedido HTTP como JSON, suportando tipos complexos como objetos aninhados, arrays, números e booleanos.

O exemplo seguinte ilustra uma ferramenta que utiliza simultaneamente os três tipos referidos:

tools:

```

- name: "test_url_vs_body"
  description: "Ferramenta de teste com path, query e body."
  targetUrl: "https://postman-echo.com/post?id={test_id}"
  method: "POST"
  headers:
    Accept: "application/json"
  pathSchema:
    test_id: "string"
  querySchema:
    filtro: "string"
  bodySchema:
    mensagem: "string"
    idade: "number"
  opcoes:
    type: "object"
    properties:
      notificar:
        type: "boolean"

```

Listing 5.1: Exemplo de ferramenta REST configurada via YAML.

5.3.2 Contextos Estáticos como Recursos

Para além das ferramentas REST, o MCP Broker permite declarar **contextos estáticos** que são expostos como recursos MCP. Um contexto é um bloco de texto associado a um URI fixo, que os clientes MCP podem ler diretamente. É útil para partilhar, por exemplo, documentação de APIs ou instruções de sistema que o modelo deve ter em conta.

```

contexts:
- name: "github-api-schema"
  uri: "context://github-api-schema"
  description: "Esquema da API GitHub para auxiliar o modelo."
  mimeType: "text/plain"
  content: |
    A API GitHub usa autenticação via token Bearer.
    Endpoints relevantes: GET /repos/{owner}/{repo}

```

Listing 5.2: Exemplo de contexto estático exposto como recurso MCP.

5.3.3 Proxy de Servidores MCP Externos

O MCP Broker pode também ligar-se a outros servidores MCP no papel de cliente e expor as ferramentas desses servidores como se fossem locais. Esta funcionalidade de *proxy* é parametrizada na seção `mcpServers`, onde se indica o nome, o tipo de transporte e os parâmetros de inicialização:

```

mcpServers:
- name: "filesystem-mcp"
  transport: STDIO
  command: "npx"
  args:
    - "-y"
    - "@modelcontextprotocol/server-filesystem"
    - "/workspace"

```

Listing 5.3: Exemplo de configuração de proxy para um servidor MCP externo.

Quando o MCP Broker arranca, liga-se a cada servidor externo configurado e descobre as suas ferramentas. Essas ferramentas são então registadas localmente com o prefixo do nome do servidor (por exemplo, `filesystem-mcp_read_file`), ficando disponíveis para qualquer cliente MCP ligado ao MCP Broker como se fossem ferramentas locais.

5.4 Arranque e Inicialização do MCP Broker

Quando o Broker arranca, percorre uma sequência de passos bem definida antes de estar pronto a receber ligações:

1. **Leitura do YAML:** O `YamlConfigParser` lê todos os ficheiros presentes na pasta de configuração e desserializa cada um num `BrokerConfigDto`, usando a biblioteca `yamlkt`. Os schemas de entrada ficam nesta fase representados como `YamlMap`.
2. **Mapeamento para o Domínio:** O `ConfigMapper` converte cada DTO no modelo de domínio correspondente. Os `YamlMap` dos schemas são transformados recursivamente em `JsonObject`, preservando os tipos nativos de cada valor.
3. **Registo de Ferramentas REST:** O `ToolRegistrar` percorre a lista de `RestToolDefinition` e regista cada ferramenta no servidor MCP, usando o Kotlin MCP SDK. Para cada uma, gera automaticamente um JSON Schema unificado a partir dos três schemas parciais (`pathSchema`, `querySchema` e `bodySchema`), que é o que o cliente recebe ao fazer `tools/list`.

4. **Registo de Contextos:** O `ContextRegistrar` publica cada `ContextDefinition` como um recurso MCP, usando igualmente o Kotlin MCP SDK. Cada recurso fica acessível pelo URI declarado no YAML.
5. **Ligação a Servidores Externos:** O `McpClientManager` estabelece ligação a cada servidor MCP externo configurado. O `ProxyToolRegistrar` descobre as ferramentas disponíveis em cada um e regista-as localmente, de forma a que o MCP Broker possa encaminhar as chamadas.
6. **Arranque do Servidor:** Por fim, o `BrokerServer` inicia os transportes configurados (Stdio, SSE, Streamable HTTP ou WebSocket) e fica a aguardar ligações de clientes MCP.

5.5 Hot-Reload de Configuração

O MCP Broker suporta **hot-reload** dos ficheiros YAML em tempo de execução, sem necessidade de reiniciar. Existe uma rotina dedicada que monitoriza de forma contínua o diretório de configuração. Sempre que uma modificação é detetada pelo sistema operativo, despoleta-se um novo ciclo de recarga. O serviço reconstrói a árvore de dependências, aplica o mapeamento e atualiza a representação exposta no servidor, procedendo de maneira transparente para quem já se encontre ligado.

5.6 DSL de Configuração Programática

O MCP Broker expõe uma **DSL Kotlin** para ser instanciado programaticamente, através da função `mcpBroker{}`. Esta abordagem é útil quando o MCP Broker é integrado numa aplicação maior, como um servidor Ktor já existente:

```
val broker = mcpBroker {
    info(name = "omni-mcp-broker", version = "1.0.0")

    configDirectory("./mcp-configs")

    httpClient(
        HttpClient(CIO) {
            install(ContentNegotiation) { json() }
        }
    )

    clientTransportFactory(McpClientTransportFactory())

    serverTransport(
        ServerTransportConfig.Stdio(
            input = SystemFileSystem.source(Path("/dev/stdin")),
            output = SystemFileSystem.sink(Path("/dev/stdout")),
        )
    )
}

broker.start()
```

Listing 5.4: Exemplo de configuração do MCP Broker via DSL Kotlin.

5.7 Fluxo de Execução

Depois de inicializado, o Broker comporta-se como qualquer servidor MCP: os clientes comunicam com ele usando o protocolo MCP padrão. Quando um cliente realiza o pedido `tools/list`, recebe a lista completa de ferramentas disponíveis, incluindo tanto as ferramentas REST locais como as que foram descobertas nos servidores externos. Quando invoca uma ferramenta com `tools/call`, o Broker sabe exatamente o que fazer com base no registo feito no arranque:

- Se for uma ferramenta REST, o `RestToolExecutor` constrói e executa o pedido HTTP correspondente usando o `HttpClient`, a interface genérica de cliente HTTP injetada no Broker. Os argumentos são distribuídos pelo URL (path e query) e pelo corpo do pedido, conforme o declarado nos schemas.
- Se for uma ferramenta em proxy, o Broker reencaminha a chamada para o servidor MCP externo e devolve a resposta tal como a recebeu.
- Se for um contexto, o cliente pode lê-lo diretamente via `resources/read`, recebendo o conteúdo estático associado ao URI.

Capítulo 6

Provas de Conceito e Validação

6.1 Validação do *Proof of Concept* da OmniAI Gateway

A validação principal da arquitetura foi efetuada através da aplicação **OmniAiGateway**, nomeadamente a instanciação de uma **OmniAiGateway** como um *Proof of Concept* (PoC) que integra o **OmniAI SDK** via *composite build*, conforme detalhado anteriormente. Esta aplicação é responsável por carregar o ficheiro de configuração **application.conf**, resolver as variáveis de ambiente necessárias, instanciar o **KtorHttpClient** e configurar os respetivos *outbounds*. Adicionalmente, assegura a ativação dos módulos de segurança e telemetria, a inicialização do *runtime* do *Software Development Kit* e a exposição de *endpoints* **HTTP** compatíveis com as APIs da **OpenAI** [1], **Anthropic** [2] e **Gemini** [3].

É importante clarificar que o comportamento validado não configura um mecanismo de "roteamento inteligente". A implementação atual baseia-se numa seleção direta por configuração, em que o pedido original já especifica o fornecedor e o modelo pretendidos, cabendo ao *dispatcher* padrão direcionar o fluxo para o *outbound* correspondente. Embora este desenho garanta o desacoplamento efetivo entre o cliente e o fornecedor, a seleção dinâmica por critérios como custo, latência, qualidade ou plano de subscrição ainda não está implementada. Contudo, os mecanismos de *fallback* e *circuit breaker* [72] [71] [70] já existentes estabelecem a infraestrutura necessária para evoluir a solução em direção a um roteamento semântico ou otimizado no futuro.

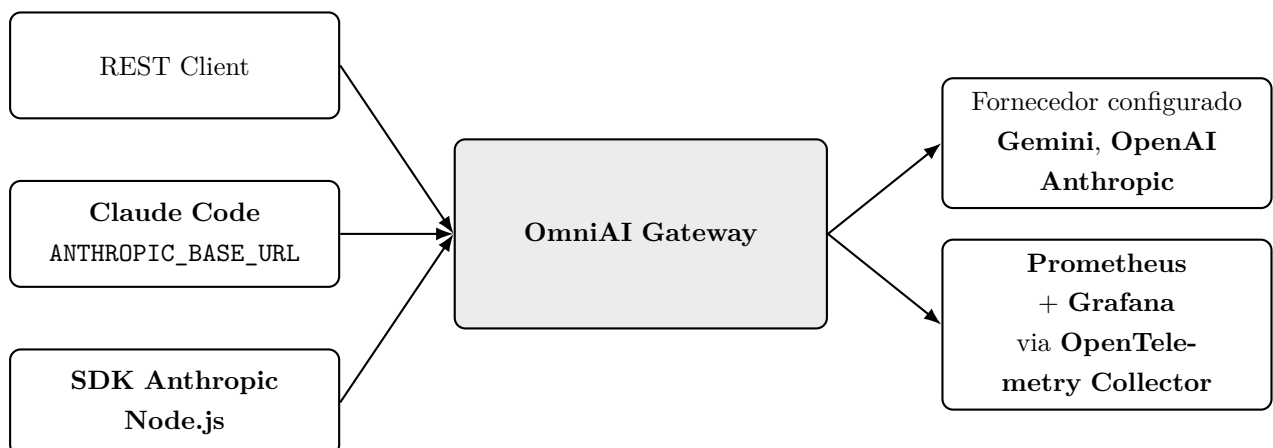


Figura 6.1: Cenários de validação do *Proof of Concept*

6.2 Validação com REST Client

O ficheiro `gateway-requests.http` foi utilizado para a validação manual dos *endpoints* expostos pela **OmniAI Gateway**. Este ficheiro agrupa pedidos direcionados a `/v1/chat/completions`, `/v1/messages` e `/v1beta/models/model:generateContent`, cobrindo cenários de respostas unárias, *streaming*, geração baseada em parâmetros e integração com autenticação via token Bearer.

A relevância deste cenário de testes fundamenta-se em dois pilares principais. Em primeiro lugar, viabilizou a validação rigorosa dos contratos **HTTP** recebidos pela **OmniAI Gateway**, prescindindo do acoplamento a aplicações externas. Em segundo lugar, proporcionou a testagem rápida e integral do fluxo de dados: desde a emissão do token pelo **Logto** e a respetiva autenticação do pedido, passando pela tradução *inbound*, conversão para o *outbound* e invocação do fornecedor configurado, até à devolução da resposta no formato esperado pelo cliente.

6.3 Validação de Segurança com Logto

A camada de segurança foi validada com **Logto** como *Authorization Server* local. O ambiente **Docker** inclui **Logto** [47], **PostgreSQL** [48], **OpenTelemetry Collector** [55], **Prometheus** [50] e **Grafana** [51]. A configuração do *Proof of Concept* usa **OIDC** Discovery [76] para obter metadados do servidor de autorização e validar *tokens* com audiência configurada para o recurso da API.

Durante o desenvolvimento foram realizados testes exploratórios com **Keycloak** [64], mas a configuração final mudou para **Logto** [47] por reduzir a complexidade operacional. O **Logto** facilitou a criação de API Resources, aplicações *Machine-to-Machine* e fluxos **OIDC** adequados à validação da **OmniAI Gateway** como *Resource Server* [77].

O fluxo também foi validado com `run_claude.py`. Este script abre o *browser* para autenticação no **Logto**, usa **PKCE** [78], troca o código por um token e inicia o **Claude Code** com `ANTHROPIC_BASE_URL` a apontar para a **OmniAI Gateway**. O cabeçalho `Authorization` é injetado através de `ANTHROPIC_CUSTOM_HEADERS`, permitindo testar um cliente real compatível com **Anthropic** a passar pela nossa **Gateway**.

6.4 Validação de Observabilidade

A observabilidade foi validada com **OpenTelemetry Collector**, **Prometheus** e **Grafana**. A **OmniAIGateway** ativa métricas na DSL do **OmniAI SDK**, incluindo latência padrão, contagem de *tokens* e contagem de erros. Também acrescenta atributos como IP do cliente, versão do **OmniAI SDK**, audiência e dados derivados do resultado de autenticação, como `sub`, `email` ou `preferred_username` quando disponíveis.

O objetivo desta validação foi confirmar que a **OmniAI Gateway** consegue produzir telemetria e garantir observabilidade em tempo real durante o processamento de pedidos. A utilização de **Prometheus** [50] e **Grafana** [51] permitiu monitorizar as métricas operacionais da aplicação, comprovando que o módulo de telemetria corre de forma isolada da lógica de inferência sendo definido via DSL e exposto por *adapters* dedicados.

6.5 Validação com Aplicações Cliente

Para validar a compatibilidade da **OmniAI Gateway** com aplicações existentes, foram estudados e testados clientes que permitem configurar *endpoints* alternativos para APIs de inferência. Esta capacidade

é particularmente relevante porque permite introduzir a **Gateway** numa aplicação já existente sem necessidade de modificar a sua lógica interna.

Um dos casos analisados foi o **Claude Code**, que disponibiliza a variável de ambiente `ANTHROPIC_BASE_URL` para redefinir o *endpoint* utilizado nos pedidos à API da **Anthropic**. Para complementar esta validação, foi também desenvolvido um teste utilizando o **SDK** oficial da **Anthropic**, implementado em `node-tests/anthropicSDKTest.js`. Neste cenário, o cliente foi configurado com um `baseUrl` local e uma chave fictícia, demonstrando que aplicações construídas sobre o **SDK** conseguem comunicar com a **OmniAI Gateway** quando esta expõe um contrato compatível com a API original.

6.5.1 Execução do Claude Code com autenticação Logto

Para além dos testes realizados através de pedidos diretos e do **SDK** oficial, foi desenvolvido o script `run_claude.py` com o objetivo de validar um cenário de utilização mais próximo de um ambiente real. Este script automatiza a configuração necessária para que o **Claude Code** utilize a **OmniAI Gateway** como ponto de entrada para os pedidos de inferência, sem qualquer alteração ao código do cliente.

O processo inicia-se com a autenticação através do **Logto**. Durante a execução, o script gera os parâmetros necessários para o fluxo **PKCE**, abre o navegador para o processo de autenticação **OIDC** e disponibiliza temporariamente um servidor **HTTP** local em `localhost:8080` para receber o código de autorização devolvido pelo fornecedor de identidade. Após a autenticação do utilizador, o código obtido é trocado por um *access token* associado ao recurso da API configurado no **Logto**.

Concluída a autenticação, o script prepara automaticamente o ambiente de execução do **Claude Code**. Para tal, define uma chave fictícia em `ANTHROPIC_API_KEY`, configura a variável `ANTHROPIC_BASE_URL` para apontar para a instância local da **Gateway** e injeta o *access token* obtido através do cabeçalho `Authorization`, utilizando a variável `ANTHROPIC_CUSTOM_HEADERS`. Finalmente, o comando `claude` é iniciado já com toda a configuração necessária.

Esta validação permitiu demonstrar que uma aplicação desenvolvida para comunicar com a API da **Anthropic** pode ser integrada com a **OmniAI Gateway** apenas através de configuração. O cenário exercita simultaneamente a autenticação federada através do **Logto**, a validação de *tokens* na **Gateway** e o encaminhamento dos pedidos para os fornecedores configurados, constituindo assim uma validação de ponta a ponta da arquitetura proposta.

Para além do **Claude Code**, foram ainda analisadas plataformas como *OpenCode* [79], *LibreChat* [80] e *Dify* [81], uma vez que também suportam a configuração de *endpoints* compatíveis com APIs existentes. Estes casos reforçam uma das principais vantagens da arquitetura proposta: quando uma aplicação já utiliza um contrato amplamente adotado, a introdução da **OmniAI Gateway** pode ser realizada através de configuração, evitando alterações na lógica da aplicação cliente e reduzindo significativamente o esforço de integração.

6.6 Limites da Validação

A validação realizada confirma o consumo do **OmniAI SDK** pelo Proof of Concept, a exposição **HTTP** com **Ktor**, a integração com **Logto**, o uso de **Prometheus/Grafana** e a compatibilidade com clientes reais ou semi-reais. No entanto, existem limites importantes. O Proof of Concept ainda não demonstra invocação de ferramentas via **MCP Broker**, nem políticas avançadas de roteamento por custo, latência ou qualidade. Estes pontos dependem de trabalho futuro sobre o módulo `mcp-broker`, sobre políticas de *dispatcher* e sobre *stores* distribuídos para cenários multi-instância.

Capítulo 7

Conclusões e Trabalho Futuro

No momento em que o desenvolvimento deste projeto teve início, a adoção de componentes dedicados ao intermédio entre aplicações e APIs de inferência era ainda uma prática não muito utilizada. A falta de arquiteturas e de produtos consolidados forçava a que a maioria das aplicações integrasse as APIs dos fornecedores de forma direta, o que limitava severamente a escalabilidade, a segurança e a gestão centralizada como já identificamos o problema anteriormente.

Contudo, à medida que o uso de modelos de linguagem em ambientes de produção cresceu exponencialmente foram surgindo cada vez mais gateways no mercado levando ao aparecimento de várias novas ferramentas consolidadas. Esta evolução e o rápido aparecimento de soluções semelhantes funcionam como um indicador claro de validação da nossa proposta. O facto de a indústria ter adotado de uma forma rápida e massiva este tipo de arquitetura prova não só a pertinência do problema escolhido no início deste projeto, mas também que a centralização e a intermediação do tráfego de IA representam o caminho definitivo pretendido por este setor.

A principal diferença introduzida por este projeto é o facto de a fundação da **OmniAI** ser um **SDK** (*Software Development Kit*). Assume a forma de uma biblioteca completamente abstrata e customizável, desenvolvida com recurso a **Kotlin Multiplatform**. Esta abordagem não força as equipas de engenharia a levantar um novo microserviço na sua infraestrutura, permitindo antes que toda a inteligência da *gateway* seja embebida de forma direta em sistemas e arquiteturas já existentes.

Devido à natureza multiplataforma do **SDK**, este nível profundo de integração é possível tanto em ecossistemas **Node.js** como em aplicações empresariais *backend* nativas da **JVM**. Esta flexibilidade arquitetural permite que os utilizadores apliquem as suas próprias políticas, criem novas rotas, desenhem adaptadores específicos ou façam a gestão de tráfego internamente sem qualquer latência de rede adicional e sem saírem das suas plataformas habituais. Neste sentido, a **OmniAI** destaca-se não apenas como um serviço, mas como uma ferramenta altamente programável construída à medida das necessidades de cada engenheiro e de cada projeto.

7.1 Conclusões

O trabalho desenvolvido permitiu concretizar o **OmniAI SDK** como uma base reutilizável para a instanciação de *gateways* de IA em **Kotlin Multiplatform**. A solução estabilizou um domínio comum para pedidos, respostas, eventos de *streaming* e ferramentas, suportado por contratos específicos para **OpenAI** [1], **Anthropic** [2] e **Gemini** [3]. Através dos *inbound adapters*, dos *outbound adapters* e do **DispatcherPort**, foi possível normalizar fluxos que, nas APIs originais, apresentam estruturas JSON, ciclos **SSE** e modelos de *tool calling* distintos, mantendo a lógica central desacoplada dos detalhes de cada um dos fornecedores.

A separação entre **OmniAI SDK** e **OmniAI Gateway** revelou-se uma decisão estrutural importante. O **OmniAI SDK** concentra o domínio, as portas, os tradutores, a *pipeline* e a DSL de configuração enquanto que a **OmniAI Gateway** desenvolvida neste projeto funciona como uma instância concreta dessa infraestrutura, construída sobre **Ktor** no *target JVM*. O *Proof of Concept* validou a montagem de *outbounds* configuráveis, a exposição de contratos compatíveis com clientes existentes, a integração com **Logto** [47] via **OIDC** [76], a recolha de telemetria com **OpenTelemetry** [55], **Prometheus** [50] e **Grafana** [51], e a aplicação de *interceptors* para autenticação, autorização, métricas, *rate limiting*, *circuit breaker*, *fallback*, *logging* e *tracing*.

Um dos principais desafios foi lidar com as diferenças reais entre os fornecedores sem reduzir a abstração a uma equivalência superficial de campos. **OpenAI**, **Anthropic** e **Gemini** convergem em conceitos como mensagens, papéis, ferramentas e consumo de *tokens*, mas divergem na forma como organizam o histórico, representam chamadas de ferramentas, ativam *streaming* e emitem eventos. No caso da **Gemini**, as restrições sobre *schemas* e o uso de estruturas próprias, como `functionDeclarations`, `generationConfig` e eventos parciais com formato de resposta completa, exigiram um tratamento específico. De forma semelhante, a fragmentação dos eventos **SSE** obrigou à criação de um modelo comum de eventos capaz de preservar deltas de texto, argumentos parciais de ferramentas, utilização de *tokens* e estados de conclusão.

Apesar destes resultados, o projeto não deve ser lido como uma solução finalizada para todos os cenários de produção. A validação demonstrou uma base estável do **OmniAI SDK** e da **OmniAI Gateway** como *Proof of Concept* na **JVM**, mas identificou pontos que dependem de continuação. O **MCP Broker** deve evoluir de uma fundação inicial para um componente operacional, com carregamento declarativo de ferramentas, recursos e *prompts* através de **YAML**, registo efetivo de *handlers* e suporte robusto a *stdio*, **SSE** e **WebSocket**. Será também necessário completar a paridade do *target JavaScript/Node.js*, consolidar a API exportada e preparar a publicação do **SDK** em **Maven** e **NPM** com versionamento semântico e documentação de consumo.

O balanço final é positivo e realista. A arquitetura construída reduz o acoplamento entre aplicações cliente e fornecedores de LLMs, centraliza preocupações operacionais que normalmente ficam dispersas e oferece uma base extensível para integrações futuras. Para além do resultado técnico, o desenvolvimento em dupla permitiu transformar um problema inicialmente amplo num conjunto de decisões arquiteturais justificadas, implementadas e validadas. Esse processo foi uma parte essencial da aprendizagem alcançada no projeto, tanto ao nível da engenharia de software como da compreensão prática das limitações atuais das APIs de inferência.

7.2 Melhorias Futuras e Escalabilidade

Como trabalho futuro, existem várias linhas de evolução. A primeira é consolidar a distribuição pública do **SDK** via **Maven** e **NPM**, garantindo documentação de instalação, versionamento semântico e exemplos de consumo. A segunda é completar a paridade entre *targets JVM* e **JavaScript**, sobretudo na montagem de *runtime* e transporte **HTTP**.

Outra melhoria importante é aprofundar o **MCP Broker**, incluindo carregamento efetivo de ferramentas via **YAML**, execução completa dos *handlers MCP* e suporte robusto a **SSE** e **WebSocket**. Também será relevante melhorar as políticas de roteamento, permitindo decisões com base em custo, latência, disponibilidade, qualidade do modelo, plano do cliente e limites de utilização.

Por fim, a camada de segurança pode evoluir para políticas de autorização mais expressivas, integração com PDPs externos, introspeção real de *tokens* opacos e mapeamento mais fino entre *claims*, planos e quotas. Estas melhorias permitiriam aproximar a **OmniAI Gateway** de uma solução pronta para ambientes *multi-tenant* e produção.

Apêndice A

Exemplos JSON das APIs de Inferência

Este apêndice reúne os exemplos completos dos corpos JSON utilizados durante o estudo das APIs de inferência. Os exemplos incluem pedidos e respostas de geração de texto, chamadas de ferramentas (*tool calling*), *streaming* e configuração de raciocínio (*thinking*), permitindo ao leitor consultar as estruturas integrais que foram analisadas e comparadas no Capítulo 2.

A.1 Pedidos (*Requests*)

A.1.1 OpenAI (formato compatível)

```
{
  "model": "llama3.2:1b",
  "messages": [
    {
      "role": "system",
      "content": "Es um assistente local a correr em Docker."
    },
    {
      "role": "user",
      "content": "Ola! Estas a funcionar?"
    }
  ],
  "temperature": 0.7
}
```

Listing A.1: Pedido básico à API da **OpenAI** (geração com *system prompt*)

```
{
  "model": "llama3.2:1b",
  "messages": [
    {
      "role": "user",
      "content": "Como esta o preco das acoes da Apple (AAPL)?"
    }
  ],
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "get_stock_price",
        "description": "Obtem o preco atual de uma acao",
        "parameters": {
          "type": "object",
          "properties": {
            "symbol": {
              "type": "string",
              "description": "O ticker da empresa, ex: AAPL"
            }
          },
          "required": ["symbol"]
        }
      }
    }
  ],
  "tool_choice": "auto"
}
```

Listing A.2: Pedido à API da **OpenAI** com ferramentas (*tool calling*)

```
{
  "model": "llama-3.3-70b-versatile",
  "messages": [
    {
      "role": "system",
      "content": "Es um assistente tecnico que responde apenas em topicos."
    },
    {
      "role": "user",
      "content": "Explica a diferenca entre CPU e GPU."
    }
  ],
  "temperature": 0.5,
  "max_tokens": 1024,
  "top_p": 1,
  "stream": false,
  "stop": ["FIM", "TERMINAR"],
  "presence_penalty": 0.0,
  "frequency_penalty": 0.0,
  "seed": 42,
  "user": "utilizador_123",
  "response_format": { "type": "text" },
  "logit_bias": null
}
```

Listing A.3: Pedido à API da **OpenAI** com todos os parâmetros de geração

```
{
  "model": "llama3.2:1b",
  "messages": [
    {
      "role": "user",
      "content": "O que é Kubernetes?"
    }
  ],
  "stream": true
}
```

Listing A.4: Pedido *stream* à API da **OpenAI**

A.1.2 Anthropic

```
{
  "model": "claude-3-5-haiku-20241022",
  "max_tokens": 1024,
  "system": "You are a cat. Your name is Neko.",
  "messages": [
    {
      "role": "user",
      "content": "Hello there"
    }
  ]
}
```

Listing A.5: Pedido básico à API da **Anthropic** (geração com *system prompt*)

```
{
  "model": "claude-3-5-haiku-20241022",
  "max_tokens": 1024,
  "messages": [
    {
      "role": "user",
      "content": "What is the weather like in Lisbon?"
    }
  ],
  "tools": [
    {
      "name": "get_weather",
      "description": "Get the current weather in a given location",
      "input_schema": {
        "type": "object",
        "properties": {
          "location": {
            "type": "string",
            "description": "The city and country, e.g. Lisbon, Portugal"
          }
        }
      },
      "required": ["location"]
    }
  ]
}
```

Listing A.6: Pedido à API da **Anthropic** com ferramentas (*tool calling*)

```
{
  "model": "claude-3-5-haiku-20241022",
  "max_tokens": 1024,
  "temperature": 1.0,
  "top_p": 0.8,
  "top_k": 10,
  "stop_sequences": ["Title"],
  "messages": [
    {
      "role": "user",
      "content": "Explain how AI works"
    }
  ]
}
```

Listing A.7: Pedido à API da **Anthropic** com parâmetros de amostragem

```
{
  "model": "claude-3-7-sonnet-20250219",
  "max_tokens": 4096,
  "thinking": {
    "type": "enabled",
    "budget_tokens": 2048
  },
  "messages": [
    {
      "role": "user",
      "content": "How does AI work?"
    }
  ]
}
```

Listing A.8: Pedido à API da **Anthropic** com raciocínio (*thinking*)

```
{
  "model": "claude-3-5-haiku-20241022",
  "max_tokens": 200,
  "messages": [
    {
      "role": "user",
      "content": "Explain AI."
    }
  ],
  "stream": true
}
```

Listing A.9: Pedido *stream* à API da **Anthropic**

A.1.3 Google Gemini

```
{
  "system_instruction": {
    "parts": [
      { "text": "You are a cat. Your name is Neko." }
    ]
  },
  "contents": [
    {
      "parts": [
        { "text": "Hello there" }
      ]
    }
  ]
}
```

Listing A.10: Pedido básico à API do **Gemini** (geração com *system instruction*)

```
{
  "contents": [
    {
      "role": "user",
      "parts": [
        {
          "text": "Schedule a meeting with Bob for tomorrow."
        }
      ]
    }
  ],
  "tools": [
    {
      "functionDeclarations": [
        {
          "name": "schedule_meeting",
          "description": "Schedules a meeting with attendees.",
          "parameters": {
            "type": "object",
            "properties": {
              "attendees": {
                "type": "array",
                "items": { "type": "string" }
              },
              "date": { "type": "string" },
              "time": { "type": "string" },
              "topic": { "type": "string" }
            },
            "required": ["attendees", "date", "time", "topic"]
          }
        }
      ]
    }
  ]
}
```

Listing A.11: Pedido à API do **Gemini** com ferramentas (*tool calling*)

```
{
  "contents": [
    {
      "parts": [
        { "text": "Explain how AI works" }
      ]
    }
  ],
  "generationConfig": {
    "stopSequences": ["Title"],
    "temperature": 1.0,
    "topP": 0.8,
    "topK": 10,
    "thinkingConfig": {
      "include_thoughts": true,
      "thinkingLevel": "low"
    }
  }
}
```

Listing A.12: Pedido à API do **Gemini** com configuração de geração e *thinking*

```
{
  "contents": [
    {
      "parts": [
        { "text": "Explain how AI works" }
      ]
    }
  ]
}
```

Listing A.13: Pedido *stream* à API do **Gemini** (URI com `?alt=sse`)

A.2 Respostas (*Responses*)

A.2.1 OpenAI (formato compatível)


```

{
  "id": "chatcmpl-31",
  "object": "chat.completion",
  "created": 1773531116,
  "model": "llama3.2:1b",
  "system_fingerprint": "fp_ollama",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "Desculpe, mas posso tentar ajuda-lo!"
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 46,
    "completion_tokens": 40,
    "total_tokens": 86
  }
}

```

Listing A.14: Resposta completa da API da **OpenAI** (geração básica)

```

{
  "id": "chatcmpl-53",
  "object": "chat.completion",
  "created": 1773531604,
  "model": "llama3.2:1b",
  "system_fingerprint": "fp_ollama",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "",
        "tool_calls": [
          {
            "id": "call_9fasug2c",
            "index": 0,
            "type": "function",
            "function": {
              "name": "get_stock_price",
              "arguments": "{\"symbol\":\"AAPL\"}"
            }
          }
        ]
      },
      "finish_reason": "tool_calls"
    }
  ],
  "usage": {
    "prompt_tokens": 171,
    "completion_tokens": 19,
    "total_tokens": 190
  }
}

```

Listing A.15: Resposta da API da **OpenAI** (chamada de ferramenta)

```

data: {"id":"chatcmpl-308","object":"chat.completion.chunk",
      "created":1773532270,"model":"llama3.2:1b",
      "choices":[{"index":0,
        "delta":{"role":"assistant","content":"K"},
        "finish_reason":null}]}

data: {"id":"chatcmpl-308","object":"chat.completion.chunk",
      "created":1773532270,"model":"llama3.2:1b",
      "choices":[{"index":0,
        "delta":{"content":"ubernetes "},
        "finish_reason":null}]}

data: {"id":"chatcmpl-308","object":"chat.completion.chunk",
      "created":1773532272,"model":"llama3.2:1b",
      "choices":[{"index":0,
        "delta":{},
        "finish_reason":"stop"}]}

data: [DONE]

```

Listing A.16: Resposta *stream* da **OpenAI** (eventos SSE)

A.2.2 Anthropic

```
{
  "id": "msg_01Xf...",
  "type": "message",
  "role": "assistant",
  "model": "claude-3-5-haiku-20241022",
  "content": [
    {
      "type": "text",
      "text": "Artificial intelligence (AI) is a complex field..."
    }
  ],
  "stop_reason": "end_turn",
  "usage": {
    "input_tokens": 14,
    "output_tokens": 350
  }
}
```

Listing A.17: Resposta completa da API da **Anthropic** (geração básica)

```
{
  "id": "msg_01Cd...",
  "type": "message",
  "role": "assistant",
  "content": [
    {
      "type": "text",
      "text": "I can help you check the weather in Lisbon."
    },
    {
      "type": "tool_use",
      "id": "toolu_01A02B03C",
      "name": "get_weather",
      "input": { "location": "Lisbon, Portugal" }
    }
  ],
  "stop_reason": "tool_use"
}
```

Listing A.18: Resposta da API da **Anthropic** (chamada de ferramenta)

```
{
  "id": "msg_01xyz...",
  "type": "message",
  "role": "assistant",
  "model": "claude-3-7-sonnet-20250219",
  "content": [
    {
      "type": "thinking",
      "thinking": "Here I need to explain how AI works...",
      "signature": "zxcvbnm1234567890..."
    },
    {
      "type": "text",
      "text": "Artificial Intelligence (AI) works fundamentally by..."
    }
  ],
  "stop_reason": "end_turn",
  "stop_sequence": null,
  "usage": {
    "input_tokens": 13,
    "output_tokens": 850
  }
}
```

Listing A.19: Resposta da API da **Anthropic** (raciocínio *thinking*)

```
event: message_start
data: {"type": "message_start",
  "message": {"id": "msg_d6d...", "role": "assistant",
    "model": "claude-3-5-haiku-20241022"}}

event: content_block_start
data: {"type": "content_block_start", "index": 0,
  "content_block": {"type": "text"}}

event: content_block_delta
data: {"type": "content_block_delta", "index": 0,
  "delta": {"type": "text_delta", "text": "Artificial"}}

event: content_block_delta
data: {"type": "content_block_delta", "index": 0,
  "delta": {"type": "text_delta", "text": " intelligence"}}

event: content_block_stop
data: {"type": "content_block_stop", "index": 0}

event: message_stop
data: {"type": "message_stop"}
```

Listing A.20: Resposta *stream* da **Anthropic** (eventos **SSE**)

```
{
  "role": "user",
  "content": [
    {
      "type": "tool_result",
      "tool_use_id": "toolu_01A02B03C",
      "content": "The weather is 18C and sunny."
    }
  ]
}
```

Listing A.21: Devolução de resultado de ferramenta na API da **Anthropic**

A.2.3 Google Gemini

```
{
  "candidates": [
    {
      "content": {
        "parts": [
          {
            "text": "At its simplest level, AI works by...",
            "thoughtSignature": "..."
          }
        ],
        "role": "model"
      },
      "finishReason": "STOP",
      "index": 0
    }
  ],
  "usageMetadata": {
    "promptTokenCount": 5,
    "candidatesTokenCount": 969,
    "totalTokenCount": 1480,
    "promptTokensDetails": [
      { "modality": "TEXT", "tokenCount": 5 }
    ],
    "thoughtsTokenCount": 506
  },
  "modelVersion": "gemini-3-flash-preview",
  "responseId": "feKyafqAEJ_VvdIP5s7TSA"
}
```

Listing A.22: Resposta completa da API do **Gemini** (geração básica)

```

{
  "candidates": [
    {
      "content": {
        "parts": [
          {
            "functionCall": {
              "name": "schedule_meeting",
              "args": {
                "time": "10:00",
                "date": "2025-03-27",
                "attendees": ["Bob", "Alice"],
                "topic": "Q3 planning"
              },
              "id": "fcaog9xf"
            },
            "thoughtSignature": "..."
          }
        ],
        "role": "model"
      },
      "finishReason": "STOP",
      "index": 0,
      "finishMessage": "Model generated function call(s)."
    }
  ],
  "usageMetadata": {
    "promptTokenCount": 203,
    "candidatesTokenCount": 54,
    "totalTokenCount": 423,
    "promptTokensDetails": [
      { "modality": "TEXT", "tokenCount": 203 }
    ],
    "thoughtsTokenCount": 166
  },
  "modelVersion": "gemini-3-flash-preview",
  "responseId": "AueyadTFGvKlvdIPrv74QA"
}

```

Listing A.23: Resposta da API do **Gemini** (chamada de ferramenta)

```

data: {"candidates": [
  {"content": {"parts": [
    {"text": "AI works by "}
  ], "role": "model"}, "index": 0}
], "usageMetadata": {
  "promptTokenCount": 4,
  "candidatesTokenCount": 25,
  "totalTokenCount": 608,
  "thoughtsTokenCount": 579
}, "modelVersion": "gemini-3-flash-preview"}

data: {"candidates": [
  {"content": {"parts": [
    {"text": "processing data."}
  ], "role": "model"}, "index": 0}
], "usageMetadata": {
  "promptTokenCount": 4,
  "candidatesTokenCount": 51,
  "totalTokenCount": 634,
  "thoughtsTokenCount": 579
}, "modelVersion": "gemini-3-flash-preview"}

data: {"candidates": [
  {"content": {"parts": [
    {"text": "", "thoughtSignature": "..."}
  ], "role": "model"},
  "finishReason": "STOP", "index": 0}
], "usageMetadata": {
  "promptTokenCount": 4,
  "candidatesTokenCount": 853,
  "totalTokenCount": 1436,
  "thoughtsTokenCount": 579
}, "modelVersion": "gemini-3-flash-preview"}

```

Listing A.24: Resposta *stream* do **Gemini** (padrão SSE)

Apêndice B

Histórico de Planeamento e Riscos

Durante a fase de planeamento foram identificados riscos técnicos e operacionais que continuam relevantes para a evolução do projeto.

B.1 Riscos

Durante o desenvolvimento do projeto foram identificados vários riscos técnicos e operacionais. Estes riscos influenciaram decisões de desenho, prioridades de implementação e estratégias de validação.

- **Curva de aprendizagem tecnológica:** O projeto exigiu a utilização de tecnologias e conceitos com elevada complexidade, nomeadamente **Kotlin Multiplatform**, arquitetura hexagonal, APIs de inferência de diferentes fornecedores, autenticação baseada em **OIDC/OAuth2** e o **Model Context Protocol**. Esta diversidade tecnológica representou um risco inicial, porque aumentou o tempo necessário para compreender os contratos externos, configurar o ambiente de desenvolvimento e definir uma arquitetura estável. A mitigação passou pela realização de protótipos incrementais, pelo estudo isolado das APIs de **OpenAI**, **Anthropic** e **Gemini**, e pela separação progressiva do sistema em módulos com responsabilidades claras.
- **Instabilidade de serviços externos:** A **Gateway** depende de serviços de terceiros, como APIs de inferência, servidores de autorização e ferramentas externas. Estes serviços podem apresentar indisponibilidade temporária, alterações nos contratos, limites de utilização ou comportamentos diferentes dos documentados. Este risco poderia comprometer a validação da solução e tornar os testes dependentes de fatores externos. Para reduzir este impacto, foram usados testes locais, servidores simulados, *mocks* e abstrações sobre o transporte **HTTP**. Além disso, a arquitetura inclui mecanismos de resiliência, como *fallback* e *circuit breaker*, que permitem lidar melhor com falhas de fornecedores.
- **Custos de utilização:** A execução de testes com modelos pagos pode gerar custos inesperados, especialmente em cenários de *streaming*, chamadas repetidas ou testes de integração. Este risco é relevante porque a **Gateway** incentiva a experimentação com múltiplos fornecedores e modelos. A mitigação passou por limitar o número de chamadas reais, utilizar fornecedores com quotas gratuitas sempre que possível, recorrer a testes locais e validar grande parte da lógica através de contratos, *adapters* e servidores simulados.
- **Complexidade de normalização entre fornecedores:** As APIs de **OpenAI**, **Anthropic** e **Gemini** partilham conceitos semelhantes, como mensagens, modelos, parâmetros de geração e ferramentas, mas representam esses conceitos de formas diferentes. Alguns elementos, como *tool*

calling, *streaming*, multimodalidade e metadados específicos, não têm correspondência direta entre fornecedores. Este risco poderia levar a uma abstração demasiado rígida ou à perda de informação específica de cada API. Para o mitigar, foi definido um domínio comum extensível, complementado por campos de opções específicas do fornecedor. Desta forma, o **SDK** consegue representar o núcleo comum sem impedir a passagem de características particulares de cada API.

- **Segurança e dados sensíveis:** A centralização dos pedidos numa **Gateway** aumenta a responsabilidade sobre autenticação, autorização, gestão de *tokens*, *logs*, métricas e chaves de API. Um erro nesta camada poderia expor dados sensíveis ou permitir acesso indevido a modelos e recursos. A mitigação passou pela introdução de uma *pipeline* de segurança com autenticação e autorização separadas, validação de *tokens*, integração com um *Authorization Server* e cuidado na recolha de métricas e *logs*. Ainda assim, este risco continua relevante para uma evolução futura em direção a ambientes de produção e cenários *multi-tenant*.
- **Âmbito e complexidade funcional:** O domínio das Gateways de IA inclui várias funcionalidades possíveis, como roteamento por custo, políticas avançadas de autorização, execução automática de ferramentas, suporte completo a **MCP**, cache semântica e gestão de quotas por utilizador. Implementar todas estas capacidades numa primeira versão aumentaria significativamente o risco de atrasos e instabilidade. A mitigação passou pela definição de um âmbito inicial mais controlado, focado na arquitetura base, nos *adapters* principais, na *pipeline* de *interceptors* e numa **Gateway** funcional construída sobre o **SDK**. As funcionalidades mais avançadas foram identificadas como trabalho futuro.

B.2 Planeamento

Semana	Período	Descrição
1	18 fev – 22 fev (parcial)	Estudo das APIs de inferência e protocolo MCP .
2	23 fev – 1 mar	Continuação da semana anterior e início da proposta.
3	2 mar – 8 mar	Entrega Proposta do projeto
4	9 mar – 15 mar	Configuração inicial da biblioteca, definição da arquitetura base e das interfaces.
5	16 mar – 22 mar	Integração com as APIs de inferência. Foco inicial em dois fornecedores estáveis.
6	23 mar – 29 mar	Integração do MCP no OmniAI Core . Invocação e registo dinâmico de ferramentas, gerindo o ciclo de <i>tool calling</i> .
7	30 mar – 5 abr	Implementação de despacho configurável de modelos e mecanismos de redundância (<i>fallback</i> e <i>retry</i>).
8	6 abr – 12 abr	Implementação dos mecanismos de segurança, como controlo de acesso, preparação do contexto e remoção de dados sensíveis.
9	13 abr – 19 abr	Conclusão das implementações anteriores.
10	20 abr – 26 abr	Apresentação de Progresso
11	27 abr – 3 mai	Início do desenvolvimento da aplicação final OmniAI Gateway . Exposição da Web API.
12	4 mai – 10 mai	Continuação da semana anterior e preparação das abstrações MCP para integração futura.
13	11 mai – 17 mai	Término de todas as APIs de comunicação para o exterior.
14	18 mai – 24 mai	Testes de integração do Core e da Gateway .
15	25 mai – 31 mai	Entrega da Versão beta
16	1 jun – 7 jun	Refinamento de testes a todos os módulos.
17	8 jun – 14 jun	Refinamento de código, desempenho e limpeza de bugs do sistema.
18	15 jun – 21 jun	Continuação da semana anterior e refinamento do relatório final.
19	22 jun – 28 jun 2026	Conclusão do relatório final.
20	29 jun – 5 jul 2026	Vistoria completa a todo o sistema e polimento de falhas para a entrega da Versão final.
21	6 jul – 11 jul 2026 (parcial)	Entrega Final do Projeto e Relatório

Tabela B.1: Planeamento semanal do projeto

Referências

- [1] OpenAI, “Openai api documentation.” [Online]. Available: <https://platform.openai.com/docs/>
- [2] Anthropic, “Anthropic messages api documentation.” [Online]. Available: <https://docs.anthropic.com/en/api/messages>
- [3] Google, “Gemini api documentation.” [Online]. Available: <https://ai.google.dev/docs>
- [4] IBM, “What is tool calling?” [Online]. Available: <https://www.ibm.com/think/topics/tool-calling>
- [5] DAIR.AI, “Function calling and tool use in LLM agents.” [Online]. Available: <https://www.promptingguide.ai/agents/function-calling>
- [6] KMP, “Kmp documentation.” [Online]. Available: <https://kotlinlang.org/multiplatform/>
- [7] A. D. Blog, “Hexagonal architecture: Understanding ports and adapters.” [Online]. Available: <https://medium.com/@akdevblog/hexagonal-architecture-understanding-ports-4020009e1aad>
- [8] GeeksforGeeks, “Java servlet filter.” [Online]. Available: <https://www.geeksforgeeks.org/java/java-servlet-filter/>
- [9] —, “Servlet - filterchain.” [Online]. Available: <https://www.geeksforgeeks.org/java/servlet-filterchain/>
- [10] Model Context Protocol, “Model context protocol specification.” [Online]. Available: <https://modelcontextprotocol.io>
- [11] Claude, “Get started with claude code.” [Online]. Available: <https://claude.com/product/claude-code>
- [12] Model Context Protocol, “Architecture overview.” [Online]. Available: <https://modelcontextprotocol.io/docs/learn/architecture>
- [13] IBM, “What are large language models (LLMs)?” [Online]. Available: <https://www.ibm.com/think/topics/large-language-models>
- [14] NVIDIA, “Ai tokens explained.” [Online]. Available: <https://blogs.nvidia.com/blog/ai-tokens-explained/>
- [15] Groq, “Groq api documentation.” [Online]. Available: <https://console.groq.com/docs>
- [16] IBM, “What are agentic workflows?” [Online]. Available: <https://www.ibm.com/think/topics/agentic-workflows>
- [17] CloudZero, “Ai vendor lock-in: How ai is creating a new dependency problem.” [Online]. Available: <https://www.cloudzero.com/blog/ai-vendor-lock-in/>

- [18] S. P, “System design of api gateway.” [Online]. Available: <https://medium.com/@lazygeek78/system-design-of-api-gateway-e8924b71b7fb>
- [19] Vasanthan, “Ai gateway pattern: Centralized model access layer.” [Online]. Available: <https://medium.com/@vasanthancomrads/ai-gateway-pattern-centralized-model-access-layer-c5049e4f151f>
- [20] K. Stoney, “Building an ai gateway.” [Online]. Available: <https://karlstoney.com/building-an-ai-gateway/>
- [21] Berri AI, “LiteLLM: Call all llm apis using the openai format.” [Online]. Available: <https://www.litellm.ai/>
- [22] OpenRouter, “OpenRouter: A unified interface for llms.” [Online]. Available: <https://openrouter.ai/>
- [23] JetBrains, “Kotlin multiplatform documentation.” [Online]. Available: <https://www.jetbrains.com/help/kotlin-multiplatform-dev/get-started.html>
- [24] OpenAI, “Openai api documentation: Structured outputs and function calling (json schema).” [Online]. Available: <https://platform.openai.com/docs/guides/structured-outputs>
- [25] Anthropic, “Anthropic docs: Define tools.” [Online]. Available: <https://platform.claude.com/docs/en/agents-and-tools/tool-use/define-tools>
- [26] JSON Schema, “Json schema specification.” [Online]. Available: <https://json-schema.org/specification>
- [27] OpenAPI Initiative, “Openapi specification v3.0.3.” [Online]. Available: <https://spec.openapis.org/oas/v3.0.3>
- [28] Google, “Gemini api reference: Schema object (openapi 3.0 subset).” [Online]. Available: <https://ai.google.dev/api/caching?#Schema>
- [29] Ollama, “Ollama documentation.” [Online]. Available: <https://ollama.com/>
- [30] Google, “Google ai studio documentation.” [Online]. Available: <https://aistudio.google.com/app/docs>
- [31] Model Context Protocol, “Introduction.” [Online]. Available: <https://modelcontextprotocol.io/docs/getting-started/intro>
- [32] —, “Server concepts.” [Online]. Available: <https://modelcontextprotocol.io/docs/learn/server-concepts>
- [33] —, “Client concepts.” [Online]. Available: <https://modelcontextprotocol.io/docs/learn/client-concepts>
- [34] JSON-RPC, “Json-rpc 2.0 specification.” [Online]. Available: <https://www.jsonrpc.org/specification>
- [35] Model Context Protocol, “Mcp inspector.” [Online]. Available: <https://modelcontextprotocol.io/docs/tools/inspector>
- [36] —, “Kotlin sdk for model context protocol.” [Online]. Available: <https://github.com/modelcontextprotocol/kotlin-sdk>
- [37] LMSYS Org, “Routellm: Learning to route llms with preference data.” [Online]. Available: <https://github.com/lm-sys/RouteLLM>

- [38] Cloudflare, “Cloudflare ai gateway.” [Online]. Available: <https://developers.cloudflare.com/ai-gateway/>
- [39] Ktor, “Ktor serialization documentation.” [Online]. Available: <https://ktor.io/docs/welcome.html>
- [40] GeeksforGeeks, “Hexagonal architecture - system design.” [Online]. Available: <https://www.geeksforgeeks.org/system-design/hexagonal-architecture-system-design/>
- [41] Netflix Technology Blog, “Ready for changes with hexagonal architecture,” 2020. [Online]. Available: <https://netflixtechblog.com/ready-for-changes-with-hexagonal-architecture-b315ec967749>
- [42] OPA, “Medium documentation.” [Online]. Available: <https://medium.com/@deutnerg/arquitetura-backend-a-arquitetura-em-camadas-01368fc13797>
- [43] H. Graça, “Ddd, hexagonal, onion, clean, cqrs, ... how i put it all together,” 2017. [Online]. Available: <https://herbertograca.com/2017/11/16/explicit-architecture-01-ddd-hexagonal-onion-clean-cqrs-how-i-put-it-all-together/>
- [44] Gradle, “Composite builds.” [Online]. Available: https://docs.gradle.org/current/userguide/composite_builds.html
- [45] Kotlin Serialization , “Kotlin serialization documentation.” [Online]. Available: <https://kotlinlang.org/docs/serialization.html>
- [46] Spring, “Spring documentation.” [Online]. Available: <https://docs.spring.io/spring-framework/reference/index.html>
- [47] Logto, “Logto documentation.” [Online]. Available: <https://docs.logto.io/>
- [48] PostgreSQL, “Postgresql documentation.” [Online]. Available: <https://www.postgresql.org/docs/>
- [49] OpenTelemetry, “What is opentelemetry?” [Online]. Available: <https://opentelemetry.io/docs/what-is-opentelemetry/>
- [50] Prometheus, “Prometheus documentation.” [Online]. Available: https://prometheus.io/docs/prometheus/latest/getting_started/
- [51] Grafana, “Grafana documentation.” [Online]. Available: <https://grafana.com/docs/>
- [52] GitLab, “Pipelines ci documentation.” [Online]. Available: <https://docs.gitlab.com/ci/pipelines/>
- [53] Redis, “Redis documentation.” [Online]. Available: <https://redis.io/docs/latest/>
- [54] Medium, “God-object.” [Online]. Available: <https://maj-ahd.medium.com/god-object-a3aa6d077312>
- [55] OpenTelemetry, “Opentelemetry protocol specification.” [Online]. Available: <https://opentelemetry.io/docs/specs/otlp/>
- [56] D. Hardt, “RFC 6749: The oauth 2.0 authorization framework.” [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc6749>
- [57] M. Jones and D. Hardt, “RFC 6750: The oauth 2.0 authorization framework: Bearer token usage.” [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc6750>
- [58] OAuth.com, “The resource server.” [Online]. Available: <https://www.oauth.com/oauth2-servers/the-resource-server/>

- [59] M. Jones, N. Sakimura, and J. Bradley, “RFC 8414: Oauth 2.0 authorization server metadata.” [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc8414>
- [60] M. Jones, J. Bradley, and N. Sakimura, “RFC 7519: Json web token (JWT).” [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7519>
- [61] M. Jones, “RFC 7517: Json web key (JWK).” [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7517>
- [62] M. Jones, J. Bradley, and N. Sakimura, “RFC 7515: Json web signature (JWS).” [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7515>
- [63] J. Richer, “RFC 7662: Oauth 2.0 token introspection.” [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7662>
- [64] Keycloak, “Keycloak documentation.” [Online]. Available: <https://www.keycloak.org/documentation>
- [65] Microsoft, “Authentication vs. authorization.” [Online]. Available: <https://learn.microsoft.com/en-us/entra/identity-platform/authentication-vs-authorization>
- [66] AuthZen, “Authzen documentation.” [Online]. Available: <https://openid.net/wg/authzen/specifications/>
- [67] OPA, “Opa documentation.” [Online]. Available: <https://www.openpolicyagent.org/docs>
- [68] Identity Beyond Borders, “The four horsemen of authorization: The p’s, pep, pdp, pap, and pip.” [Online]. Available: <https://medium.com/identity-beyond-borders/the-four-horsemen-of-authorization-the-p-ps-pep-pdp-pap-and-pip-42717e445ce7>
- [69] Cloudflare, “O que e limitacao de taxa?” [Online]. Available: <https://www.cloudflare.com/pt-br/learning/bots/what-is-rate-limiting/>
- [70] Microsoft, “Circuit breaker pattern.” [Online]. Available: <https://learn.microsoft.com/azure/architecture/patterns/circuit-breaker>
- [71] Itau Tecnologia, “Compreendendo o conceito de circuit breaker e sua aplicacao em arquitetura de microservicos.” [Online]. Available: <https://medium.com/@itautecnologia/compreendendo-o-conceito-de-circuit-breaker-e-sua-aplicacao-em-arquitetura-de-microservicos-108dc2d1be2c>
- [72] TreinaWeb, “O que e circuit breaker?” [Online]. Available: <https://www.treinaweb.com.br/blog/o-que-e-circuit-breaker>
- [73] T. Tombas, “Building resilient ai systems: Understanding model-level fallback mechanisms.” [Online]. Available: <https://medium.com/@tombastaner/building-resilient-ai-systems-understanding-model-level-fallback-mechanisms-436cf636045f>
- [74] Portkey, “Fallbacks.” [Online]. Available: <https://portkey.ai/docs/product/ai-gateway/fallbacks>
- [75] Him188, “yamlkt: Yaml parser and serializer for kotlin.” [Online]. Available: <https://github.com/Him188/yamlkt>
- [76] OIDC discovery, “Oidc discovery documentation.” [Online]. Available: <https://cloud.ibm.com/docs/appid?topic=appid-discovery>

- [77] Resource Server, “Resource server.” [Online]. Available: <https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/index.html>
- [78] PKCE, “Pkce documentation.” [Online]. Available: <https://auth0.com/docs/get-started/authentication-and-authorization-flow/authorization-code-flow-with-pkce>
- [79] Open Code, “Open code documentation.” [Online]. Available: <https://opencode.ai/docs/>
- [80] LibreChat, “Librechat documentation.” [Online]. Available: <https://www.librechat.ai/docs/documentation>
- [81] Dify, “Dify documentation.” [Online]. Available: <https://docs.dify.ai/en/use-dify/getting-started/introduction>